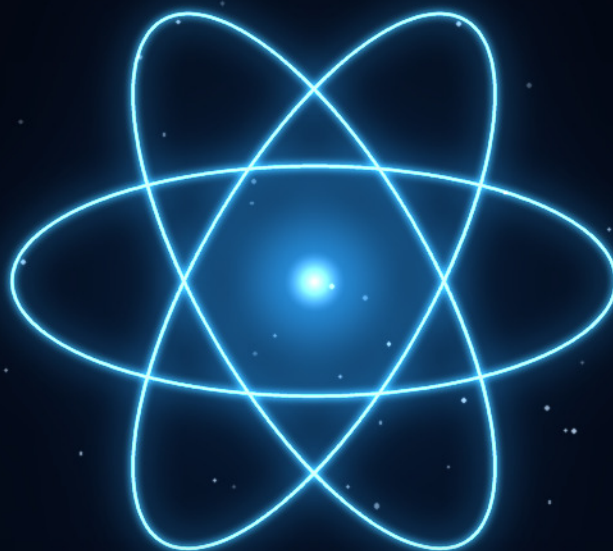




P E R S I A N D E V E L O P E R H A N D B O O K

راهنمای جامع و حرفه‌ای

React 19.2

از مبانی تا معماری Production-Level



React 19.2



TypeScript



Vite



React Compiler

راهنمایی برای جامعه توسعه‌دهندگان فارسی‌زبان، به‌سوی یادگیری مدل ذهنی درست، الگوهای مدرن و استانداردهای واقعی صنعت در ساخت رابط‌های کاربری با React.

۳۷ فصل کاربردی • مینی‌پروژه عملی • آمادگی مصاحبه

محمدرضا رضائیان

Front-End Developer • گردآوری و تدوین

با هدف کمک به جامعه توسعه‌دهندگان Front-End فارسی‌زبان



فهرست مطالب

بخش یکم · بنیادها و مفاهیم اصلی

۴	01 معرفی React و تازه‌های نسخه ۱۹
۸	02 شروع به کار و ساختار پروژه
۱۵	03 نقشه راه یادگیری
۱۸	04 JSX و رندر عناصر
۲۳	05 کامپوننت‌ها و Props
۲۹	06 State و چرخه رندر
۳۴	07 رویدادها و تعامل کاربر
۳۹	08 قوانین Hooks و مرور همه هوک‌ها
۴۳	09 فرم‌ها و Actions در React 19
۴۹	10 useOptimistic و تجربه کاربری آنی
۵۳	11 useEffect، useRef و useEffectEvent
۶۰	12 واکنشی داده، Suspense و use
۶۷	13 Context و useReducer
۷۳	14 هوک‌های سفارشی
۸۱	15 React در TypeScript
۸۸	16 روتینگ با React Router v8
۹۵	17 استایل‌دهی، Tailwind v4 و Metadata
۱۰۱	18 پروژه عملی: مدیریت وظایف کامل

بخش دوم · پیشرفته، معماری و Production

۱۱۲	19 کارایی، Concurrent Rendering و React Compiler
۱۱۹	20 مدیریت state در مقیاس: Zustand، Jotai و Redux Toolkit
۱۲۶	21 معماری پروژه و Clean Code در React

۱۳۲	الگوهای پیشرفته کامپوننت	22
۱۴۱	فرم‌ها و اعتبارسنجی پیشرفته	23
۱۵۰	مدیریت خطا – مرجع کامل	24
۱۵۶	دسترسی‌پذیری، RTL و بین‌المللی‌سازی	25
۱۶۴	امنیت اپلیکیشن React	26
۱۶۹	تست‌نویسی: Vitest، Testing Library و Playwright	27
۱۷۶	Server Functions و Server Components	28
۱۸۳	انتخاب فریم‌ورک: TanStack Start و Next.js، React Router	29
۱۸۸	Build، استقرار و Web Vitals	30
۱۹۶	مهاجرت و ارتقا به React 19	31
۲۰۲	انیمیشن، View Transitions و حس «نرم بودن»	32

بخش سوم · کارگاه عملی و نکات طلایی

۲۱۲	نکات و ترفندهای طلایی	33
۲۱۶	کارگاه مینی‌پروژه‌ها	34
۲۲۰	بهترین شیوه‌ها، اشتباهات رایج و منابع	35

بخش چهارم · مرجع سریع و آمادگی مصاحبه

۲۲۵	پرسش‌های مصاحبه (Junior تا Senior)	36
۲۳۰	واژه‌نامه فارسی-انگلیسی	37

01 معرفی React و تازه‌های نسخه ۱۹

در این فصل می‌بینیم React دقیقاً چیست، چرا در دنیای Front-End به استاندارد تبدیل شد، و چرا نسخه ۱۹ (و به‌ویژه ۱۹.۲) یک نقطه عطف در نحوه نوشتن اپلیکیشن‌های وب محسوب می‌شود.

React چیست و چه مشکلی را حل می‌کند؟

React یک کتابخانه جاوااسکریپت برای ساخت رابط کاربری است که در سال ۲۰۱۳ توسط Meta (فیسبوک) متن‌باز شد. ایده اصلی آن ساده اما انقلابی بود: به جای این‌که به مرورگر بگویید «چطور» DOM را تغییر دهد، شما «چه چیزی» را می‌خواهید توصیف می‌کنید و React خودش تفاوت‌ها را محاسبه و اعمال می‌کند.

سه ایده بنیادی که React را متمایز می‌کند:

- ♦ **کامپوننت‌محوری (Component-Based):** رابط کاربری از قطعات کوچک، مستقل و قابل استفاده مجدد ساخته می‌شود. هر کامپوننت منطق و نمای خودش را در یک جا نگه می‌دارد.
- ♦ **اعلانی بودن (Declarative):** شما وضعیت (state) را توصیف می‌کنید و React رابط را با آن همگام نگه می‌دارد؛ نه `document.querySelector` و نه دست‌کاری دستی DOM.
- ♦ **جریان داده یک‌طرفه:** داده از والد به فرزند از طریق props جریان می‌یابد؛ این قابلیت، پیش‌بینی‌پذیری و دیباگ را بسیار ساده‌تر می‌کند.

i مدل ذهنی « $UI = f(state)$ »

مهم‌ترین جمله‌ای که باید از این کتاب به خاطر بسپارید: **رابط کاربری تابعی از وضعیت است.** هر بار که state تغییر می‌کند، React کامپوننت را دوباره اجرا می‌کند (render) و خروجی جدید را با قبلی مقایسه می‌کند. اگر این جمله را درک کنید، ۸۰٪ باگ‌های رایج React را از قبل پیش‌بینی خواهید کرد.

چرا React همچنان انتخاب اول است؟

در سال ۲۰۲۶ گزینه‌های زیادی وجود دارد: Vue، Svelte، Solid، Angular. اما React همچنان بزرگ‌ترین سهم بازار کار، غنی‌ترین اکوسیستم و بیشترین سرمایه‌گذاری صنعتی را دارد:

معیار	وضعیت React در ۲۰۲۶
بازار کار	بیشترین آگهی شغلی Front-End در ایران و جهان
اکوسیستم	Expo، TanStack، React Router، Next.js (موبایل)، React Three Fiber (سه‌بعدی)
پایداری API	تغییرات شکننده بسیار کم؛ کد React 16 هنوز روی ۱۹ اجرا می‌شود
نوآوری	React Compiler، Actions، Server Components، <Activity>
پشتیبانی	تیم تمام‌وقت در Vercel + Meta + جامعه عظیم

چه چیزی در نسخه ۱۹ تغییر کرد؟

React 19 (دسامبر ۲۰۲۴) بزرگ‌ترین به‌روزرسانی از زمان معرفی Hooks در نسخه ۱۶.۸ است. نسخه‌های ۱۹.۱ (مارس ۲۰۲۵) و ۱۹.۲ (اکتبر ۲۰۲۵) این مسیر را تکمیل کردند. این کتاب بر پایه **React 19.2** نوشته شده است.

قابلیت	توضیح کوتاه
Actions	توابع async در transition؛ مدیریت خودکار pending، خطا و optimistic update
هوک‌های جدید	useActionState، useFormStatus، useOptimistic، useEffectEvent
API جدید use	خواندن Promise و Context در حین رندر؛ حتی داخل شرط و حلقه
ref به‌عنوان prop	دیگر نیازی به forwardRef نیست؛ ref مثل هر prop دیگری پاس داده می‌شود
Metadata داخلی	<title>، <meta> و <link> را در هر کامپوننتی بنویسید؛ React خودش به <head> منتقل می‌کند
Server Components	پایدار شد؛ کامپوننت‌هایی که فقط روی سرور اجرا می‌شوند و JS به کلاینت نمی‌فرستند
<Activity>	(۱۹.۲) پنهان کردن بخشی از UI با حفظ state و بدون هزینه رندر
React Compiler 1.0	(اکتبر ۲۰۲۵) memoization خودکار؛ خداحافظی با useCallback / useMemo دستی
حذف‌ها	defaultProps، propTypes برای توابع، string refs، Legacy Context و ReactDOM.render حذف شدند

⚠️ مهم‌ترین تغییر ذهنی

در React 18 برای مدیریت فرم‌ها، وضعیت loading، خطا و به‌روزرسانی خوش‌بینانه باید چندین `useState` و `try/catch` می‌نوشتید. در React 19 این‌ها **مفهوم درجه‌یک** شده‌اند: یک تابع `async` را به `<form action>` می‌دهید و React بقیه را انجام می‌دهد. اگر از ۱۸ می‌آیید، باید یاد بگیرید «کمتر بنویسید و بیشتر به React اعتماد کنید».

نقشه بزرگ: React در سال ۲۰۲۶

فول‌استک (فریم‌ورک)

Next.js، React Router v8 (حالت Framework)، TanStack Start، Streaming Server Components، و Server Functions بهره می‌برند. بعد از تسلط بر مبانی این کتاب، مهاجرت به آن‌ها آسان است.

فقط کلاینت (SPA)

Vite + React + React Router مناسب داشبوردهای داخلی، ابزارهای ادمین و اپ‌هایی که SEO اهمیت ندارد. ساده، سریع و کاملاً تحت کنترل شما. **این کتاب روی همین مدل تمرکز دارد** تا خود React را عمیق یاد بگیرید.

خود تیم React توصیه می‌کند برای پروژه جدید از یک فریم‌ورک استفاده کنید، اما **بدون فهم عمیق React خالص**، هیچ فریم‌ورکی را نمی‌توان درست به کار برد. به همین دلیل ابتدا React را با Vite یاد می‌گیریم و در فصل‌های پایانی به فریم‌ورک‌ها و Server Components می‌رسیم.

پیش‌نیازهای این آموزش

- ♦ تسلط نسبی به **JavaScript مدرن** (+ES2020): `const/let`، `arrow function`، `destructuring`، `spread`، `async/await`، `map`، `filter`، `reduce`.
- ♦ آشنایی پایه با **HTML و CSS**: عناصر، ویژگی‌ها، Flexbox.
- ♦ آشنایی مقدماتی با **TypeScript** مفید است اما الزامی نیست؛ فصل ۱۵ به‌طور کامل به آن می‌پردازد و تمام مثال‌ها TS هستند چون عملاً استاندارد پروژه‌های جدی شده‌اند.
- ♦ **Node.js نسخه ۲۰.۱۹ یا بالاتر** و یک ویرایشگر مثل VS Code.

“ سؤالات مصاحبه (با پاسخ)

React – Junior کتابخانه است یا فریمورک؟ رسماً کتابخانه است: فقط لایه View را مدیریت می‌کند و روتینگ، مدیریت داده و ساخت پروژه را به ابزارهای دیگر می‌سپارد. اما در عمل «اکوسیستم React» (با Next.js یا React Router) نقش فریمورک را بازی می‌کند.

Virtual DOM – Mid چیست و آیا هنوز مهم است؟ یک نمایش سبک از DOM در حافظه که React از آن برای مقایسه (diff) خروجی قبلی و جدید استفاده می‌کند تا کمترین تغییر را روی DOM واقعی اعمال کند. در React 19 اصطلاح رسمی کمتر استفاده می‌شود و تمرکز روی «Fiber tree» و «reconciliation» است؛ اما مفهوم یکسان است.

Senior – چرا React 19 «Actions» را معرفی کرد و چه مشکل معماری‌ای را حل می‌کند؟ مدیریت async state (loading/error/optimistic) در هر پروژه‌ای به صورت دست‌ساز و ناسازگار پیاده می‌شد. Actions با ادغام transitions و فرم‌های بومی HTML، این الگوها را استاندارد کرد، امکان Progressive Enhancement داد و پایه‌ای مشترک بین کلاینت و Server Functions ساخت.

02 شروع به کار و ساختار پروژه

در این فصل یک پروژه React 19 + TypeScript با Vite می‌سازیم، آناتومی فایل‌ها را می‌فهمیم و ابزارهایی که هر توسعه‌دهنده حرفه‌ای باید نصب داشته باشد را راه‌اندازی می‌کنیم.

چرا Vite؟

سال‌ها `create-react-app` ابزار رسمی شروع پروژه بود، اما در سال ۲۰۲۵ به‌طور رسمی بازنشسته شد. امروز **Vite** استاندارد صنعت برای پروژه‌های React خالص است: شروع سرد زیر یک ثانیه، HMR فوری و خروجی بهینه. نسخه ۸ Vite (مارس ۲۰۲۶) با باندلر Rust-محور **Rolldown** عرضه شد و سرعت build را چند برابر کرد.

```
Terminal

# ساخت پروژه با قالب React + TypeScript
npm create vite@latest my-app -- --template react-ts

cd my-app
npm install
npm run dev
```

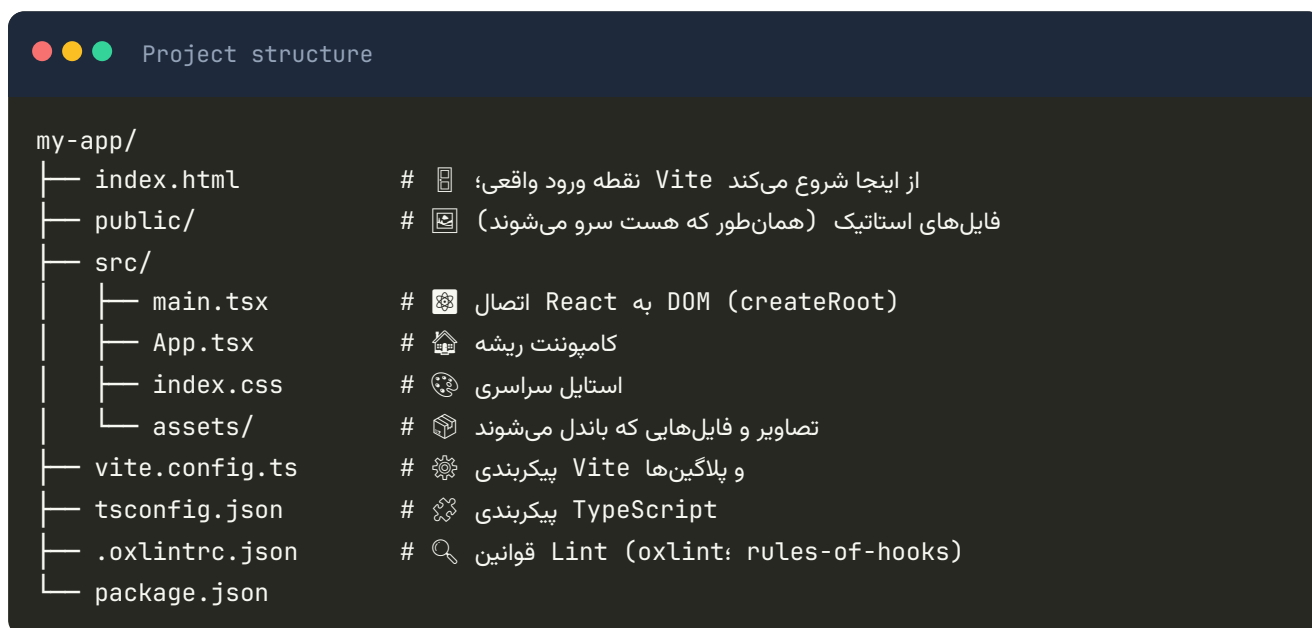
پس از اجرا، آدرس `http://localhost:5173` را در مرورگر باز کنید.

انتخاب پکیج‌منیجر

`npm` کاملاً کافی است، اما در پروژه‌های تیمی `pnpm` (سریع‌تر، فضای دیسک کمتر) رایج‌تر شده است: `pnpm create vite my-app --template react-ts`. در طول کتاب از `npm` استفاده می‌کنیم؛ دستورات معادل `pnpm/yarn/bun` تقریباً یکسان‌اند.

دستورهای اصلی

دستور	کاربرد
<code>npm run dev</code>	سرور توسعه با HMR
<code>npm run build</code>	ساخت نسخه بهینه Production در پوشه <code>dist/</code>
<code>npm run preview</code>	اجرای محلی خروجی build برای تست نهایی
<code>npx tsc --noEmit</code>	بررسی صحت تایپ‌ها بدون تولید فایل
<code>npm run lint</code>	اجرای linter قالب (oxlint) – یا ESLint اگر آن را نصب کنید (بخش بعد)



قلب اپلیکیشن دو فایل است. اول `index.html` که برخلاف ابزارهای قدیمی، در ریشه پروژه قرار دارد و Vite آن را نقطه شروع می‌داند:



و دوم `main.tsx` که React را به آن `div` متصل می‌کند:

```
src/main.tsx

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

StrictMode چیست؟

`<StrictMode>` فقط در حالت توسعه فعال است و کامپوننت‌ها را دو بار رندر می‌کند و `Effect`‌ها را دو بار اجرا می‌کند تا باگ‌های ناشی از ناخالصی (impure render) یا `cleanup` فراموش‌شده را زودتر آشکار کند. در `Production` هیچ اثری ندارد. هرگز آن را برای «رفع» باگ حذف نکنید؛ باگ را رفع کنید.

اولین کامپوننت

فایل `App.tsx` را با این محتوا جایگزین کنید:

```
src/App.tsx

import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>کلیک تعداد: {count}</button>
  );
}

export default function App() {
  return (
    <main>
      <h1>سلام React 19! 🙌</h1>
      <Counter />
    </main>
  );
}
```

مرورگر بدون رفرش به روز می‌شود؛ این همان **Hot Module Replacement** است. سه چیز را از همین حالا به خاطر بسپارید: نام کامپوننت با حرف بزرگ شروع می‌شود، هر کامپوننت یک تابع است که JSX برمی‌گرداند، و state با `useState` تعریف می‌شود.

پیکربندی Vite و فعال کردن React Compiler

React Compiler در نسخه ۱.۰ (اکتبر ۲۰۲۵) پایدار شد و memoization را خودکار می‌کند. در قالب پیش فرض Vite به دلیل تأثیر جزئی روی سرعت build غیرفعال است؛ برای پروژه واقعی توصیه می‌کنیم فعالش کنید:

```
Terminal

npm install -D babel-plugin-react-compiler@latest @rolldown/plugin-babel
```

```
vite.config.ts

import { defineConfig } from 'vite';
import react, { reactCompilerPreset } from '@vitejs/plugin-react';
import babel from '@rolldown/plugin-babel';
import path from 'node:path';

export default defineConfig({
  plugins: [
    react(),
    babel({ presets: [reactCompilerPreset()] }),
  ],
  resolve: {
    // import Button from '@components/Button'
    alias: { '@': path.resolve(__dirname, 'src') },
  },
});
```

برای این‌که alias در TypeScript هم شناخته شود، به `tsconfig.app.json` اضافه کنید (بدون `baseUrl` – این گزینه در TypeScript 6 منسوخ شده و مسیرها نسبت به خود فایل `tsconfig` حل می‌شوند):

```
tsconfig.app.json

{
  "compilerOptions": {
    "paths": { "@/*": ["../src/*"] }
  }
}
```

⚠ ترتیب پلاگین‌ها مهم است

React Compiler باید **اولین** پلاگین Babel باشد چون به کد اصلی و دست‌نخورده نیاز دارد. اگر از نسخه‌های قدیمی‌تر `@vitejs/plugin-react` (قبل از ۶) استفاده می‌کنید، به جای `@rolldown/plugin-babel` گزینه `react({ babel: { plugins: ['babel-plugin-react-compiler'] } })` را به کار ببرید.

Lint: قوانین Hooks و کامپایلر

قالب رسمی Vite از نسخه ۹ به جای ESLint، **oxlint** (نوشته شده با Rust، ده ها برابر سریع تر) را با قانون `react/rules-of-hooks` می آورد و برای شروع کافی است. اما قوانین کامل «Rules of React» و بررسی های React Compiler (mutation، purity، setState در Effect و...) فقط در `eslint-plugin-react-hooks` (نسخه ۶ به بعد) هستند؛ برای پروژه واقعی توصیه می کنیم ESLint را با Flat Config کنار آن اضافه کنید:

Terminal

```
npm install -D eslint @eslint/js typescript-eslint \
  eslint-plugin-react-hooks eslint-plugin-react-refresh
```

eslint.config.js

```
import js from '@eslint/js';
import tseslint from 'typescript-eslint';
import reactHooks from 'eslint-plugin-react-hooks';
import reactRefresh from 'eslint-plugin-react-refresh';
import { defineConfig } from 'eslint/config';

export default defineConfig([
  js.configs.recommended,
  ...tseslint.configs.recommended,
  reactHooks.configs.flat.recommended, // شامل قوانین کامپایلر
  reactRefresh.configs.vite,
  { files: ['**/*.ts', '**/*.tsx'] },
]);
```

ابزارهای ضروری توسعه دهنده

ابزار	چرا؟
React Developer Tools	افزونه مرورگر؛ درخت کامپوننت ها، props/state، پروفایلر و نشان ✨ Memo برای کامپوننت های بهینه شده توسط کامپایلر
VS) ES7+ React Snippets (Code	rfc → کامپوننت آماده، us → useState
Prettier	فرمت خودکار؛ بحث های بی فایده روی استایل کد تمام می شود
(VS Code) Error Lens	نمایش خطای TypeScript/ESLint در همان خط
Chrome Performance Tracks	ردهای اختصاصی React در تب Performance کروم (React 19.2)

متغیرهای محیطی

Vite فقط متغیرهایی با پیشوند `VITE_` را در کد کلاینت قابل دسترس می‌کند:

```
.env.local
```

```
VITE_API_URL=https://api.example.com
```

```
src/lib/config.ts
```

```
export const API_URL = import.meta.env.VITE_API_URL;
```

✗ هرگز اسرار را در کلاینت نگذارید

هر چیزی با `VITE_` در باندل نهایی و قابل مشاهده برای همه است. کلید API خصوصی، توکن پرداخت یا رمز دیتابیس هیچ وقت نباید در پروژه کلاینتی React قرار بگیرد. این‌ها فقط در بک‌اند (یا Server Components / Functions) جای دارند.

“سؤالات مصاحبه (با پاسخ)”

Junior – تفاوت `public/` و `src/assets/` چیست؟ فایل‌های `public/` بدون پردازش و با همان نام سرو می‌شوند (مثل `favicon.ico` یا `robots.txt`). فایل‌های `src/assets/` توسط Vite باندل، hash گذاری و بهینه می‌شوند و از طریق `import` استفاده می‌شوند.

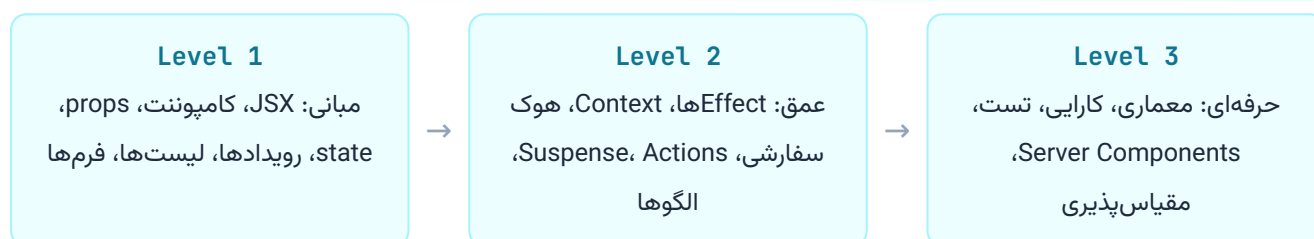
Mid – چرا Vite در `dev` سریع‌تر از Webpack است؟ چون در `dev` کد را باندل نمی‌کند؛ از ES Modules بومی مرورگر استفاده می‌کند و هر فایل را در لحظه درخواست تبدیل می‌کند. تغییر یک فایل یعنی فقط همان مازول دوباره ارسال می‌شود.

Senior – React Compiler چه کدی را نمی‌تواند بهینه کند؟ کدی که «قوانین React» را نقض می‌کند: تغییر `props/state (mutate)`، فراخوانی هوک به صورت شرطی، `side effect` در بدنه رندر. کامپایلر چنین کامپوننت‌هایی را رد می‌کند (skip) نه این‌که بشکند؛ ESLint دقیقاً به شما می‌گوید کدام کامپوننت و چرا.

03 نقشه راه یادگیری

React را نمی‌توان با حفظ کردن API یاد گرفت؛ باید مدل ذهنی درست ساخت. این فصل نقشه‌ای می‌دهد که بدانید هر مفهوم کجای پازل قرار می‌گیرد و چطور از این کتاب بیشترین بهره را ببرید.

سه سطح تسلط



بخش یکم کتاب (فصل‌های ۱ تا ۱۸) سطح ۱ و بخشی از سطح ۲ را پوشش می‌دهد. بخش دوم (فصل‌های ۱۹ تا ۳۲) عمیقاً وارد سطح ۲ و ۳ می‌شود. بخش‌های سوم و چهارم برای تثبیت، تمرین و آمادگی مصاحبه هستند.

نقشه راه پیشنهادی (۸ هفته)

هفته	تمرکز	فصل‌ها	خروجی عملی
۱	مبانی و JSX	۱-۵	کارت پروفایل، لیست محصولات
۲	State و رویدادها	۶-۸	شمارنده پیشرفته، فیلتر جستجو
۳	فرم‌ها و Actions	۹-۱۰	فرم ثبت‌نام با اعتبارسنجی
۴	Effect و داده	۱۱-۱۲	لیست کاربران از API با Suspense
۵	Context، Reducer، هوک سفارشی	۱۳-۱۴	تم تاریک/روشن، سبد خرید
۶	TypeScript، روتینگ، استایل	۱۵-۱۸	مینی‌بلاگ چندصفحه‌ای
۷	کارایی و معماری	۱۹-۲۳	ری‌فکتور پروژه با ساختار feature-based
۸	تست، Production، RSC	۲۴-۳۲	پروژه نهایی + استقرار

اشتباهات رایج در مسیر یادگیری

✓ مسیر درست

- ◆ تسلط بر JavaScript مدرن قبل از React
- ◆ فهم «رندر = اجرای مجدد تابع»
- ◆ ساخت مینی پروژه بعد از هر فصل
- ◆ خواندن پیامهای خطای React (بسیار گویا هستند)
- ◆ استفاده از TypeScript از روز اول

✗ مسیر اشتباه

- ◆ پرش مستقیم به Next.js بدون فهم React خالص
- ◆ حفظ کردن هوکها بدون درک چرخه رندر
- ◆ کپی کردن کد از هوش مصنوعی بدون فهم
- ◆ استفاده از `useEffect` برای هر چیزی
- ◆ یادگیری Redux قبل از فهم `useState` و `Context`

نحوه استفاده از این کتاب

- ◆ هر فصل یک واحد مستقل است: با یک مقدمه، مفاهیم، مثالهای واقعی، جعبه‌های «نکته» و «اشتباه رایج» و در پایان پرسش‌های مصاحبه.
- ◆ کدها را تایپ کنید، کپی نکنید: تایپ کردن کد باعث می‌شود با خطاها روبه‌رو شوید و یاد بگیرید آنها را حل کنید.
- ◆ جعبه‌های «سؤالات مصاحبه» را جدی بگیرید: سه سطح Junior، Mid و Senior دارند و دقیقاً همان چیزهایی‌اند که در مصاحبه‌های واقعی پرسیده می‌شوند.
- ◆ مینی پروژه‌ها را انجام دهید: فصل ۳۴ هشت پروژه کوچک دارد که مفاهیم چند فصل را ترکیب می‌کند.

i راهنمای نمادها

در سراسر کتاب از جعبه‌های رنگی استفاده شده است: آبی برای نکات مفهومی، سبز برای ترفندهای حرفه‌ای، نارنجی برای هشدارها، قرمز برای اشتباهات رایج، بنفش برای سؤالات مصاحبه و فیروزه‌ای برای مینی پروژه‌ها.

منابع رسمی که باید بشناسید

منبع	آدرس	کاربرد
مستندات رسمی	react.dev	مرجع اصلی؛ بخش Learn برای مفاهیم، بخش Reference برای API
وبلاگ React	react.dev/blog	اعلام نسخه‌ها و RFCها
React Compiler	react.dev/learn/react-compiler	راهنمای نصب و «قوانین React»
Vite	vite.dev	پیکربندی ابزار build
TypeScript Cheatsheet	react-typescript-cheatsheet.netlify.app	الگوهای تایپ‌دهی کامپوننت‌ها

✦ پیش‌نیاز صادقانه

اگر با `filter / map`، `destructuring`، `async/await` و `spread` راحت نیستید، ابتدا ۲ تا ۳ روز روی JavaScript مدرن وقت بگذارید. React خودش ساده است؛ اکثر سردرگمی‌ها از JavaScript می‌آیند، نه از React.

04 JSX و رندر عناصر

JSX زبانی است که با آن رابط کاربری را «توصیف» می‌کنیم. این فصل قواعد آن، تفاوتش با HTML و الگوهای رایج رندر شرطی و لیست را با دقت آموزش می‌دهد.

JSX چیست؟

JSX یک افزونه نحوی برای JavaScript است که اجازه می‌دهد ساختار شبیه HTML را مستقیماً داخل کد بنویسید. مرورگر JSX را نمی‌فهمد؛ ابزار build (Vite) آن را به فراخوانی‌های تابع تبدیل می‌کند:

آنچه اجرا می‌شود

```
import { jsx }
  from 'react/jsx-runtime';
const el = jsx('h1', {
  className: 'title',
  children: ['سلام ', name],
});
```

آنچه می‌نویسید

```
const el = (
  <h1 className="title">سلام {name}</h1>
);
```

نکته کلیدی: خروجی JSX یک شیء جاوااسکریپت ساده (React Element) است، نه DOM واقعی. به همین دلیل می‌توانید آن را در متغیر بریزید، از تابع برگردانید یا در آرایه نگه دارید.

قواعد طلایی JSX

قاعده	مثال درست	چرا
یک عنصر ریشه	<code><><A/></></code>	تابع فقط یک مقدار برمی‌گرداند
بستن همه تگ‌ها	<code><input /></code> ، <code></code>	JSX به XML نزدیک‌تر است تا HTML
camelCase برای ویژگی‌ها	<code>tabIndex</code> ، <code>onClick</code> ، <code>className</code>	ویژگی‌ها property جاوااسکریپت هستند
عبارت داخل <code>{}</code>	<code>{a + b}</code> ، <code>{user.name}</code>	فقط عبارت، نه <code>for</code> / <code>if</code>
استایل به صورت شیء	<code>style={{ color: 'red' }}</code>	دو آکولاد: یکی JSX و یکی شیء
کامنت	<code>{/* توضیح */}</code>	کامنت HTML کار نمی‌کند

✖ اشتباه رایج: `class` به جای `className`

کلمه رزرو شده جاوااسکریپت است؛ در JSX باید `className` بنویسید. همچنین `for` در label به `htmlFor` تبدیل می‌شود. React 19 در صورت اشتباه هشدار واضحی در کنسول می‌دهد.

```
src/components/Greeting.tsx

type GreetingProps = { user: { name: string; age: number; isPro: boolean } };

export function Greeting({ user }: GreetingProps) {
  const greeting = new Date().getHours() < 12 ? 'صبح بخیر' : 'عصر بخیر';

  return (
    <section>
      <h2>{greeting}, {user.name}!</h2>
      <p>سن: {user.age.toLocaleString('fa-IR')}</p>
      <p>وضعیت: {user.isPro ? '☆ حرفه‌ای' : 'رایگان'}</p>
    </section>
  );
}
```

مقادیر مختلف چگونه رندر می‌شوند؟

مقدار	خروجی
number ، string	متن
undefined ، null ، false ، true	هیچ چیز (همین ویژگی پایه رندر شرطی است)
آرایه	تک تک عناصر پشت سرهم
شیء ساده	✗ خطا: «Objects are not valid as a React child»

سه الگوی اصلی وجود دارد و انتخاب بین آن‌ها به خوانایی بستگی دارد:

```
src/components/OrderStatus.tsx

type Props = { status: 'pending' | 'paid' | 'failed'; items: number };

export function OrderStatus({ status, items }: Props) {
  // ۱) return زود هنگام برای حالت‌های کاملاً متفاوت
  if (status === 'failed') {
    return <p className="error">پرداخت ناموفق</p>;
  }

  // ۲) عملگر && برای «نمایش یا هیچ»
  // ۳) عملگر سه‌تایی برای «یا این یا آن»
  return (
    <div>
      {items > 0 && <p>سفارش در کالا {items}</p>}
      <p>{status === 'paid' ? 'پرداخت شد' : 'در انتظار پرداخت'}</p>
    </div>
  );
}
```

⚠ تله عدد صفر در `&&`

{items.length && <List/>} وقتی items.length برابر ۰ باشد، خود عدد ۰ را روی صفحه چاپ می‌کند (چون ۰ falsy است اما null نیست). همیشه شرط را boolean کنید: {items.length > 0 && <List/>}

```
src/components/ProductList.tsx

type Product = { id: string; title: string; price: number };

export function ProductList({ products }: { products: Product[] }) {
  if (products.length === 0) return <p>نشد یافت محصولی.</p>;

  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          {p.title} - {p.price.toLocaleString('fa-IR')} تومان
        </li>
      ))}
    </ul>
  );
}
```

key به React می‌گوید هر آیتم «کیست» تا هنگام تغییر لیست (افزودن، حذف، مرتب‌سازی) بتواند عناصر DOM و state آن‌ها را درست نگه دارد.

✖ هرگز از index به‌عنوان key استفاده نکنید (مگر...)

اگر لیست مرتب‌سازی، فیلتر یا آیتم از وسطش حذف می‌شود، استفاده از index باعث می‌شود state آیتم‌ها (مثل مقدار input) به آیتم اشتباه بچسبد. فقط وقتی لیست کاملاً ثابت و بدون شناسه است، index قابل‌قبول است. اگر داده id ندارد، هنگام ساخت داده با `crypto.randomUUID()` یکی بسازید — نه در زمان رندر.

وقتی نمی‌خواهید یک `div` اضافی به DOM اضافه کنید:

```
// فرم کوتاه
<>
  <dt>نام</dt>
  <dd>{name}</dd>
</>

// لازم است key فرم کامل - فقط وقتی
{rows.map((r) => (
  <Fragment key={r.id}>
    <dt>{r.label}</dt>
    <dd>{r.value}</dd>
  </Fragment>
))}
```

استایل داخلی و کلاس‌های شرطی

```
<button
  className={isActive ? 'btn btn-active' : 'btn'}
  style={{ padding: 8, opacity: disabled ? 0.5 : 1 }}
>
```

مقادیر عددی در `style` به صورت خودکار `px` می‌گیرند (`padding: 8` → `8px`). برای ترکیب کلاس‌های متعدد، کتابخانه کوچک `clsx` بسیار رایج است: `clsx('btn', { 'btn-active': isActive })`.

“سؤالات مصاحبه (با پاسخ)”

Junior – چرا در JSX باید یک عنصر ریشه داشته باشیم؟ چون JSX به یک فراخوانی تابع تبدیل می‌شود و تابع فقط یک مقدار برمی‌گرداند. Fragment (`<>...</>`) این محدودیت را بدون افزودن عنصر اضافی به DOM حل می‌کند.

Mid – key باید در کجا قرار بگیرد و آیا داخل کامپوننت قابل دسترسی است؟ `key` روی عنصری که مستقیماً داخل `map` برمی‌گردد قرار می‌گیرد (نه روی عنصر داخلی‌تر). `key` به عنوان `prop` به کامپوننت پاس داده نمی‌شود؛ اگر به مقدار نیاز دارید، آن را با نام دیگری هم ارسال کنید.

Senior – تغییر key یک کامپوننت چه اثری دارد و کجا عمداً از آن استفاده می‌کنیم؟ تغییر `key` یعنی «این یک عنصر کاملاً جدید است»: React نمونه قبلی را `unmount` و `state` آن را دور می‌ریزد و از نو `mount` می‌کند. الگوی عمدی: ریست کردن فرم هنگام تغییر کاربر با `<ProfileForm key={userId} />` به جای `useEffect` برای پاک کردن `state`.

05 کامپوننت‌ها و Props

کامپوننت واحد سازنده هر اپلیکیشن React است. در این فصل یاد می‌گیرید چطور کامپوننت‌های قابل‌ترکیب و قابل‌استفاده‌مجدد طراحی کنید و با تغییر مهم React 19 در مورد ref آشنا می‌شوید.

کامپوننت یعنی چه؟

یک کامپوننت تابعی است که props می‌گیرد و React Element برمی‌گرداند. همین. هیچ جادوی دیگری در کار نیست:

```
src/components/Avatar.tsx

type AvatarProps = {
  src: string;
  name: string;
  size?: number; // اختیاری با مقدار پیش‌فرض
};

export function Avatar({ src, name, size = 40 }: AvatarProps) {
  return (
    <img
      src={src}
      alt={name}
      width={size}
      height={size}
      style={{ borderRadius: '50%' }}
    />
  );
}
```

i `defaultProps` حذف شد

در React 19، `defaultProps` برای کامپوننت‌های تابعی حذف شده است. مقدار پیش‌فرض را همان‌طور که بالا می‌بینید با **destructuring پیش‌فرض** جاوااسکریپت تعریف کنید. `propTypes` هم حذف شده و جای آن را TypeScript گرفته است.

قواعد Props

- ◆ **Props فقط خواندنی هستند.** هرگز `props.value = x` ننویسید؛ کامپوننت باید نسبت به ورودی‌هایش «خالص» (pure) باشد.
- ◆ **جریان یک‌طرفه:** والد به فرزند داده می‌دهد. اگر فرزند باید چیزی را تغییر دهد، والد یک تابع **callback** پاس می‌دهد.
- ◆ **هر مقدار جاوااسکریپت مجاز است:** رشته، عدد، شیء، آرایه، تابع، حتی JSX.

```
src/components/Toolbar.tsx

type ToolbarProps = {
  title: string;
  onSave: () => void; // callback
  onDelete?: (id: string) => void; // اختیاری
  actions?: React.ReactNode; // دلخواه JSX
};

export function Toolbar({ title, onSave, actions }: ToolbarProps) {
  return (
    <header className="toolbar">
      <h2>{title}</h2>
      <div>
        {actions}
        <button onClick={onSave}>ذخیره</button>
      </div>
    </header>
  );
}
```


ترکیب با children

`children` یک prop ویژه است: هر چیزی که بین تگ باز و بسته کامپوننت بنویسید. این مهم‌ترین ابزار ترکیب (Composition) در React است و جایگزین وراثت می‌شود:

```
src/components/Card.tsx

type CardProps = {
  title?: string;
  children: React.ReactNode;
  footer?: React.ReactNode;
};

export function Card({ title, children, footer }: CardProps) {
  return (
    <div className="card">
      {title && <h3 className="card-title">{title}</h3>}
      <div className="card-body">{children}</div>
      {footer && <div className="card-footer">{footer}</div>}
    </div>
  );
}

// استفاده
<Card title="پرو فایل" footer=<button>ویرایش</button>>
  <Avatar src={user.avatar} name={user.name} />
  <p>{user.bio}</p>
</Card>
```

♦ الگوی «Slot»

وقتی یک کامپوننت چند ناحیه قابل سفارشی‌سازی دارد (هدر، بدنه، فوتر)، به جای دهها prop رشته‌ای، برای هر ناحیه یک prop از نوع `ReactNode` تعریف کنید. این الگو «Slot» نام دارد و کامپوننت را انعطاف‌پذیر و API آن را کوچک نگه می‌دارد.

ref به عنوان prop (تغییر بزرگ React 19)

در نسخه‌های قبل برای دسترسی به DOM داخلی یک کامپوننت سفارشی باید از `forwardRef` استفاده می‌کردید. در React 19 `ref` یک prop معمولی است:

حالا (React 19)

```
function Input({
  ref,
  ...props
}: InputProps & {
  ref?: React.Ref<HTMLInputElement>;
}) {
  return <input ref={ref} {...props} />;
}
```

قبل (React 18)

```
const Input = forwardRef<
  HTMLInputElement,
  InputProps
>((props, ref) => (
  <input ref={ref} {...props} />
));
```

`forwardRef` هنوز کار می‌کند اما منسوخ (deprecated) است و در نسخه‌های آینده حذف خواهد شد. یک `codemod` رسمی برای مهاجرت خودکار وجود دارد.

پخش props با Spread

src/components/Button.tsx

```
import type { ComponentProps } from 'react';

type ButtonProps = ComponentProps<'button'> & {
  variant?: 'primary' | 'ghost';
};

export function Button({ variant = 'primary', className, ...rest }: ButtonProps) {
  return (
    <button
      className={`btn btn-${variant} ${className ?? ''}`}
      {...rest}
    />
  );
}
```

`ComponentProps<'button'>` تمام ویژگی‌های بومی `<button>` (از جمله `disabled`، `onClick`، `type` و حتی `ref`) را به ارث می‌برد. این الگو استاندارد ساخت Design System است.

⚠️ **spread** را کورکورانه استفاده نکنید

`{...rest}` راحت است اما اگر روی عنصر DOM قرار بگیرد و prop ناشناخته‌ای در آن باشد، React هشدار می‌دهد. همیشه prop‌های سفارشی (مثل `variant`) را قبل از `spread` جدا کنید.

Render Props و کامپوننت‌های چندشکلی

گاهی می‌خواهید **منطق** را به اشتراک بگذارید ولی **نما** را به مصرف‌کننده بسپارید:

```
src/components/DataList.tsx

type DataListProps<T> = {
  items: T[];
  renderItem: (item: T, index: number) => React.ReactNode;
  emptyMessage?: string;
};

export function DataList<T extends { id: string }>({
  items,
  renderItem,
  emptyMessage = 'موردی وجود ندارد',
}: DataListProps<T>) {
  if (items.length === 0) return <p>{emptyMessage}</p>;
  return <ul>{items.map((it, i) => <li key={it.id}>{renderItem(it, i)}</li>)}</ul>;
}

// خودکار استنتاج می‌شود item استفاده - کامپوننت جنریک است و تایپ
<DataList items={users} renderItem={(u) => <b>{u.name}</b>} />
```

امروز هوک‌های سفارشی (فصل ۱۴) بیشتر جای Render Props را گرفته‌اند، اما این الگو برای کامپوننت‌های لیست، جدول و ویرچوالایز هنوز رایج است.

اصول طراحی API کامپوننت

اصل	توضیح
کوچک و متمرکز	هر کامپوننت یک مسئولیت. اگر بیش از ۱۵۰ خط شد، بشکنید
نام‌گذاری boolean	<code>open</code> ، <code>canEdit</code> ، <code>hasError</code> ، <code>isOpen</code> — نه <code>open</code>
callbacks با <code>on</code>	<code>onClose</code> ، <code>onSubmit</code> ، <code>onChange</code>
کنترل‌شده در برابر کنترل‌نشده	اگر <code>value</code> می‌گیرید، <code>onChange</code> هم بگیرید؛ اگر نه <code>defaultValue</code>
ترکیب به جای پیکربندی	<code><Modal><Modal.Header/></Modal></code> بهتر از <code><Modal headerText=... headerIcon=... /></code>

“سؤالات مصاحبه (با پاسخ)”

Junior – تفاوت props و state چیست؟ props از بیرون (والد) می‌آید و فقط خواندنی است؛ state داخل کامپوننت است و کامپوننت خودش آن را تغییر می‌دهد. تغییر هرکدام باعث رندر مجدد می‌شود.

Mid – چرا نباید کامپوننت را داخل کامپوننت دیگر تعریف کرد؟ چون در هر رندر والد، یک «نوع» کامپوننت جدید ساخته می‌شود؛ React آن را عنصری متفاوت می‌بیند، کل زیردرخت را `unmount/mount` می‌کند و state از دست می‌رود. همیشه کامپوننت‌ها را در سطح مازول تعریف کنید.

Senior – Composition چگونه مشکل «prop drilling» را قبل از رسیدن به Context حل می‌کند؟ با پاس‌دادن JSX به‌عنوان `children` یا `slot`، کامپوننت میانی دیگر نیازی به دانستن داده ندارد؛ والد بالایی مستقیماً کامپوننت نهایی را با داده می‌سازد و به پایین می‌فرستد. این «بالا کشیدن ترکیب» اغلب Context را غیرضروری می‌کند.

06 State و چرخه رندر

state حافظه کامپوننت است و رندر مکانیزمی که آن حافظه را به رابط کاربری تبدیل می‌کند. درک دقیق این دو مفهوم، تفاوت بین «کسی که React می‌نویسد» و «کسی که React را می‌فهمد» است.

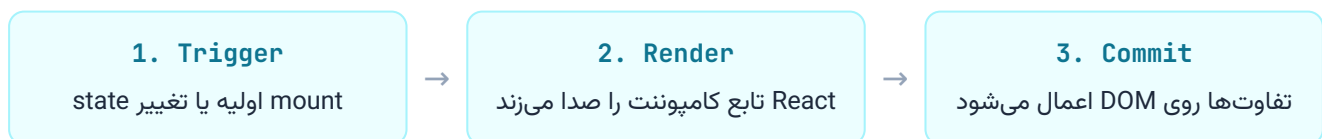
چرا متغیر معمولی کافی نیست؟

```
function Counter() {  
  let count = 0; // ❌ با هر رندر دوباره می‌شود  
  return <button onClick={() => count++}>{count}</button>;  
}
```

دو مشکل: اول، تغییر `count` به React نمی‌گوید که باید دوباره رندر کند. دوم، حتی اگر رندر شود، تابع از نو اجرا و `count` دوباره می‌شود. `useState` هر دو مشکل را حل می‌کند: مقدار را بین رندها حفظ می‌کند و تغییر آن رندر مجدد را زمان‌بندی می‌کند.

```
src/components/Counter.tsx  
  
import { useState } from 'react';  
  
export function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

مدل ذهنی رندر: سه مرحله



نکته حیاتی: «رندر» یعنی اجرای تابع شما، نه لمس React. DOM ممکن است کامپوننت را رندر کند ولی چون خروجی تغییری نکرده، هیچ چیزی در DOM عوض نشود. به همین دلیل تابع کامپوننت باید خالص باشد: با ورودی یکسان، خروجی یکسان بدهد و هیچ side effect نداشته باشد.

state یک «عکس فوری» است

```
src/components/SnapshotDemo.tsx

export function SnapshotDemo() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    // ✗ count فقط ۱ واحد اضافه می‌شود
    console.log(count); // هنوز مقدار قدیمی را چاپ می‌کند
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

چرا؟ چون `count` در طول یک رندر ثابت است؛ متغیری است که مقدارش در لحظه رندر «عکس‌برداری» شده. هر سه `setCount(0 + 1)` هستند. اگر مقدار جدید به مقدار قبلی وابسته است، از تابع به‌روزرسان (**updater**) استفاده کنید:

```
setCount((c) => c + 1);
setCount((c) => c + 1);
setCount((c) => c + 1); // ✓ حالا ۳ واحد اضافه می‌شود
```

i Batching خودکار

React چند `setState` را که در یک رویداد (یا حتی در `setTimeout` ، `Promise` و...) رخ می‌دهند، دسته‌بندی می‌کند و فقط یک بار رندر می‌کند. از React 18 به بعد این رفتار همه‌جا فعال است. بنابراین نگران کارایی چند `setState` پشت‌سرهم نباشید.

ناتغییرپذیری (Immutability)

React برای تشخیص تغییر state از مقایسه مرجع (`Object.is`) استفاده می‌کند. اگر شیء یا آرایه را مستقیماً تغییر دهید، مرجع همان است و React متوجه تغییر نمی‌شود:

کپی جدید ✓

```
setUser({ ...user, name: 'علی' });

setItems([...items, newItem]);
setItems(items.filter(i => !i.done));
setItems(items.map(i =>
  i.id === id ? { ...i, done: true } : i
));
```

Mutation ✗

```
user.name = 'علی';
setUser(user); // رندر نمی‌شود!

items.push(newItem);
setItems(items); // رندر نمی‌شود!
```

عملیات	متد ناتغییرپذیر
افزودن	<code>arr.concat(item)</code> یا <code>[...arr, item]</code>
حذف	<code>arr.filter(x => x.id !== id)</code>
به‌روزرسانی	<code>arr.map(x => x.id === id ? {...x, ...changes} : x)</code>
مرتب‌سازی	<code>[...arr].sort(cmp)</code> یا <code>arr.toSorted(cmp)</code> (ES2023)
جای‌گذاری	<code>arr.with(index, value)</code> (ES2023)
شیء تودرتو	<code>{...s, address: {...s.address, city}}</code>

♦ Immer برای state پیچیده

اگر state چند سطح تودرتو دارد، کتابخانه `immer` (یا `use-immer`) اجازه می‌دهد کد را «به‌شکل mutation» بنویسید ولی خروجی ناتغییرپذیر بگیرید: `.setUser(draft => { draft.address.city = 'تهران' })`. Redux Toolkit هم درونش از Immer استفاده می‌کند.

مقدار اولیه تنبل (Lazy Initializer)

```
// ✗ در هر رندر اجرا می‌شود (فقط نتیجه اولی استفاده می‌شود)
const [data, setData] = useState(expensiveParse(raw));

// ✓ فقط در رندر اول اجرا می‌شود
const [data, setData] = useState(() => expensiveParse(raw));
```

ساختاردهی state

- گروه‌بندی مرتبط‌ها: اگر دو state همیشه با هم تغییر می‌کنند (x و y)، یک شیء باشند.
- پرهیز از تناقض: به‌جای `isLoading` و `isError` جداگانه، یک `status: 'idle' | 'loading' | 'error' | 'success'`
- پرهیز از تکرار: اگر مقداری از state دیگری محاسبه می‌شود (مثل `fullName` از `first` و `last`)، آن را state نکنید؛ در رندر محاسبه کنید.
- پرهیز از تودرتویی عمیق: ساختار normalize شده (`{ byId, allIds }`) برای داده‌های رابطه‌ای.

✖ اشتباه رایج: کپی state در props

```
function Profile({ user }) {  
  const [name, setName] = useState(user.name); // ✖  
}
```

اگر `user.name` از والد تغییر کند، این state به‌روز نمی‌شود چون `useState` مقدار اولیه را فقط بار اول می‌خواند. یا مستقیماً از prop استفاده کنید، یا اگر واقعاً «مقدار اولیه قابل‌ویرایش» می‌خواهید، نامش را صریح کنید (`initialName`) و با `key` ریست کنید.

State کجا باید زندگی کند؟ (Lifting State Up)

وقتی دو کامپوننت خواهر به یک داده نیاز دارند، state را به نزدیک‌ترین والد مشترک منتقل کنید و از طریق props پایین بفرستید:

```
src/features/search/SearchPage.tsx  
  
export function SearchPage() {  
  const [query, setQuery] = useState('');  
  
  return (  
    <>  
      <SearchInput value={query} onChange={setQuery} />  
      <SearchResults query={query} />  
    </>  
  );  
}
```


کامپوننت کنترل شده در برابر کنترل نشده

کنترل نشده	کنترل شده	
DOM	state React	منبع حقیقت
<code>ref + defaultValue</code>	<code>onChange + value</code>	props
فرم‌های ساده، فایل آپلود	اعتبارسنجی لحظه‌ای، فرمت‌دهی	کاربرد
با Actions و <code>FormData</code> بسیار راحت‌تر شد	همچنان پرکاربرد	React 19

«سؤالات مصاحبه (با پاسخ)»

Junior – چرا نباید state را مستقیماً تغییر داد؟ چون React با مقایسه مرجع تغییر را تشخیص می‌دهد؛ mutation مرجع را عوض نمی‌کند و رندر رخ نمی‌دهد. علاوه بر آن، ناتغییرپذیری امکان بازگشت (undo)، دیباگ و بهینه‌سازی با memo را فراهم می‌کند.

Mid – تفاوت `setCount(count + 1)` و `setCount(c => c + 1)` چیست؟ اولی از مقدار «عکس فوری» رندر فعلی استفاده می‌کند؛ دومی از آخرین مقدار در صف به‌روزرسانی‌ها. وقتی به‌روزرسانی به مقدار قبلی وابسته است یا چند بار پشت‌سرهم اتفاق می‌افتد، فرم تابعی درست است.

Senior – «رندر» و «کامیت» چه تفاوتی دارند و چرا این تمایز برای Concurrent React مهم است؟ رندر محاسبه خروجی (اجرای تابع) است و قابل توقف، تکرار یا دورانداختن؛ کامیت اعمال نتیجه روی DOM است و همیشه همگام و یکپارچه. چون رندر ممکن است چند بار بدون کامیت اجرا شود (مثلاً در transition‌ها)، side effect در بدنه رندر ممنوع است و باید در Effect یا event handler باشد.

07 رویدادها و تعامل کاربر

تعامل کاربر قلب هر اپلیکیشن است. React سیستم رویداد خودش را دارد که روی DOM ساخته شده اما تفاوت‌های مهمی دارد. این فصل الگوهای صحیح و تله‌های رایج را پوشش می‌دهد.

مبانی event handler

```
src/components/LikeButton.tsx

import { useState, type MouseEvent } from 'react';

export function LikeButton() {
  const [liked, setLiked] = useState(false);

  function handleClick(e: MouseEvent<HTMLButtonElement>) {
    e.preventDefault();
    setLiked((v) => !v);
  }

  return (
    <button onClick={handleClick} aria-pressed={liked}>
      {liked ? '💔 پسندیدم' : '💖 پسندیدن'}
    </button>
  );
}
```

سه قاعده:

- ♦ تابع را پاس بدهید، صدا نزنید: ☒ `onClick={handleClick}` – ☐ `onClick={handleClick()}` (این یکی در لحظه رندر اجرا می‌شود).
- ♦ برای پاس دادن آرگومان از arrow استفاده کنید: `onClick={() => deleteItem(id)}`.
- ♦ نام‌گذاری: handlerهای داخلی با `handle` و propهای callback با `on` شروع شوند.

SyntheticEvent چیست؟

React رویدادها را در یک شیء `SyntheticEvent` می‌پیچد که در همه مرورگرها رفتار یکسان دارد و همان API استاندارد (`currentTarget` ، `target` ، `stopPropagation` ، `preventDefault`) را ارائه می‌دهد. از React 17 به بعد، رویدادها به **root container** (نه `document`) متصل می‌شوند و pooling حذف شده؛ یعنی می‌توانید `e` را در `async` هم استفاده کنید.

رویداد React	نوع TypeScript	نکته
<code>onClick</code>	<code>MouseEvent<HTMLButtonElement></code>	روی هر عنصر
<code>onChange</code>	<code>ChangeEvent<HTMLInputElement></code>	در هر keystroke اجرا می‌شود (برخلاف HTML)
<code>onSubmit</code>	<code>FormEvent<HTMLFormElement></code>	حتماً <code>preventDefault</code> مگر با Actions
<code>onKeyDown</code>	<code>KeyboardEvent<HTMLElement></code>	<code>e.key === 'Enter'</code>
<code>onBlur</code> / <code>onFocus</code>	<code>FocusEvent</code>	در React حباب می‌کنند (bubble)
<code>onPointerDown</code>	<code>PointerEvent</code>	موس + لمس + قلم یکجا

Delegation و Propagation

src/components/ProductCard.tsx

```
export function ProductCard({ product, onOpen, onAddToCart }: Props) {
  return (
    <article onClick={() => onOpen(product.id)} className="card">
      <h3>{product.title}</h3>
      <button
        onClick={(e) => {
          e.stopPropagation(); // جلوی باز شدن کارت را می‌گیرد
          onAddToCart(product.id);
        }}
      >
        سبد به افزودن
      </button>
    </article>
  );
}
```

⚠️ `stopPropagation` را کم استفاده کنید

اغلب راه بهتر این است که کلیک‌پذیر بودن کارت را حذف و یک لینک/دکمه واضح بگذارید. حباب‌کردن رویداد، قابلیت‌های دسترسی‌پذیری و کتابخانه‌های دیگر (مثل بستن منو با کلیک بیرون) را هم مختل می‌کند.

الگوی handler مشترک برای فرم‌ها

src/components/SignupForm.tsx

```
import { useState, type ChangeEvent } from 'react';

type Form = { name: string; email: string; agree: boolean };

export function SignupForm() {
  const [form, setForm] = useState<Form>({ name: '', email: '', agree: false });

  function handleChange(e: ChangeEvent<HTMLInputElement>) {
    const { name, value, type, checked } = e.target;
    setForm((f) => ({ ...f, [name]: type === 'checkbox' ? checked : value }));
  }

  return (
    <form>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" type="email" value={form.email} onChange={handleChange} />
      <label>
        <input name="agree" type="checkbox" checked={form.agree}
          onChange={handleChange} />
        می‌پذیرم را قوانین
      </label>
    </form>
  );
}
```

در فصل ۹ خواهید دید که React 19 با **Actions** و **FormData** بسیاری از این boilerplate را حذف می‌کند؛ اما الگوی کنترل‌شده برای اعتبارسنجی لحظه‌ای همچنان لازم است.

```
<div
  role="button"
  tabIndex={0}
  onClick={open}
  onKeyDown={(e) => {
    if (e.key === 'Enter' || e.key === ' ') {
      e.preventDefault();
      open();
    }
  }}
>
```

از عنصر معنایی درست استفاده کنید

اگر چیزی کلیک می‌شود، `<button>` باشد؛ اگر به جایی می‌برد، `<a href>`. این عناصر به صورت خودکار فوکوس، کیبورد و اعلام صحیح به screen reader را دارند. کد بالا فقط برای مواقع خاص است. `<button>` بدون `type="button"` داخل فرم، فرم را submit می‌کند!

رویدادهای غیر-React (window, document)

برای رویدادهایی که به عنصر خاصی تعلق ندارند (اسکرول پنجره، resize، کلیدهای سراسری) از `useEffect` استفاده می‌شود (فصل ۱۱):

```
useEffect(() => {
  const onKey = (e: KeyboardEvent) => e.key === 'Escape' && onClose();
  window.addEventListener('keydown', onKey);
  return () => window.removeEventListener('keydown', onKey);
}, [onClose]);
```

Throttle و Debounce |

برای جستجوی زنده، هر keystroke نباید درخواست شبکه بزند. راه مدرن و بدون کتابخانه: `useDeferredValue` (فصل ۱۹) برای `UI` و یک هوک `debounce` کوچک برای شبکه:

```
src/hooks/useDebounceValue.ts

import { useEffect, useState } from 'react';

export function useDebounceValue<T>(value: T, delay = 300) {
  const [debounced, setDebounce] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounce(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا `onClick={handleClick()}` اشتباه است؟ چون تابع در لحظه رندر اجرا می‌شود و نتیجه‌اش (معمولاً `undefined`) به `onClick` داده می‌شود. اگر داخلش `setState` باشد، حلقه بی‌نهایت رندر می‌سازید.

Mid – تفاوت `e.target` و `e.currentTarget` چیست؟ `target` عنصری است که رویداد روی آن رخ داده (ممکن است فرزند باشد)؛ `currentTarget` عنصری است که handler به آن متصل است. در `TypeScript`، `currentTarget` تایپ دقیق دارد و ترجیح داده می‌شود.

React – Senior رویدادها را چطور پیاده‌سازی می‌کند و چه پیامدهایی دارد؟ `React` یک `listener` برای هر نوع رویداد روی `root container` ثبت می‌کند (`event delegation`) و خودش رویداد را به کامپوننت‌ها `dispatch` می‌کند. پیامدها: `stopPropagation` در `React` جلوی `listener`های بومی روی `document` را نمی‌گیرد؛ ترتیب اجرای `listener`های بومی و `React` ممکن است غیرمنتظره باشد؛ و چند اپ `React` مستقل روی یک صفحه با هم تداخل ندارند.

08 قوانین Hooks و مرور همه هوک‌ها

هوک‌ها به کامپوننت‌های تابعی «حافظه» و «قابلیت» می‌دهند. اما دو قانون ساده دارند که نقض‌شان باگ‌های عجیب می‌سازد. این فصل چرایی این قوانین و نقشه کامل هوک‌های نسخه ۱۹ را ارائه می‌دهد.

۱ دو قانون طلایی

۱. فقط در سطح بالای کامپوننت یا هوک سفارشی صدا بزنید – نه داخل `if`، حلقه، تابع تودرتو یا بعد از `return` زود هنگام.

۲. فقط از داخل کامپوننت `React` یا هوک سفارشی صدا بزنید – نه در تابع `جاوااسکریپت` معمولی، `کلاس` یا `event handler`.

۱ چرا این قوانین وجود دارند؟

`React` هوک‌ها را با ترتیب فراخوانی به هم مرتبط می‌کند، نه با نام. در هر رندر، `React` یک لیست از «سلول‌های حافظه» دارد و انتظار دارد هوک اول همان هوک اول رندر قبلی باشد:

```
function Profile({ userId }) {  
  const [name, setName] = useState(''); // سلول ۰  
  if (userId) {  
    const [age, setAge] = useState(0); // گاهی سلول ۱، گاهی وجود ندارد ✗  
  }  
  const [bio, setBio] = useState(''); // گاهی سلول ۱، گاهی سلول ۲ → خرابی  
}
```

وقتی `userId` از `truthy` به `falsy` تغییر کند، `useState` سوم به سلولی که قبلاً برای `age` بود می‌رسد و `state` قاطی می‌شود. ESLint با قانون `react-hooks/rules-of-hooks` این خطا را قبل از اجرا می‌گیرد.

۱ قوانین (Rules of React) React

از `React 19` با `React Compiler`، مجموعه گسترده‌تری به نام **Rules of React** رسمی شد: کامپوننت‌ها و هوک‌ها باید خالص باشند، `props/state` تغییر داده نشوند، مقادیر بازگشتی هوک‌ها `mutate` نشوند، و هوک‌ها فقط به‌عنوان هوک استفاده شوند (نه پاس‌دادن به‌عنوان مقدار). پلاگین `eslint-plugin-react-hooks@6+` این‌ها را بررسی می‌کند.

نقشه کامل هوک‌های React 19.2

◆ هوک‌های State

هوک	کاربرد	فصل
<code>useState</code>	state ساده محلی	۶
<code>useReducer</code>	state پیچیده با منطق انتقال متمرکز	۱۳
<code>useActionState</code>	(۱۹) state + pending یک Action فرم	۹
<code>useOptimistic</code>	(۱۹) نمایش خوش‌بینانه نتیجه قبل از پاسخ سرور	۱۰

◆ هوک‌های Context و Ref

هوک	کاربرد	فصل
<code>useContext</code>	خواندن Context (در ۱۹ می‌توان <code>use(Context)</code> نوشت)	۱۳
<code>useRef</code>	مقدار قابل‌تغییر بدون رندر؛ دسترسی به DOM	۱۱
<code>useImperativeHandle</code>	سفارشی‌کردن چیزی که ref والد می‌بیند	۱۱

◆ هوک‌های Effect

هوک	کاربرد	فصل
<code>useEffect</code>	همگام‌سازی با سیستم خارجی (بعد از paint)	۱۱
<code>useLayoutEffect</code>	مثل Effect ولی قبل از paint (اندازه‌گیری DOM)	۱۱
<code>useInsertionEffect</code>	فقط برای کتابخانه‌های CSS-in-JS	—
<code>useEffectEvent</code>	(۱۹.۲) جداکردن منطق «رویدادی» از وابستگی‌های Effect	۱۱

♦ هوک‌های کارایی

هوک	کاربرد	فصل
<code>useMemo</code>	کش نتیجه محاسبه سنگین (با Compiler اغلب غیرضروری)	۱۹
<code>useCallback</code>	کش هویت تابع (با Compiler اغلب غیرضروری)	۱۹
<code>useTransition</code>	علامت‌گذاری به‌روزرسانی غیرفوری + <code>isPending</code>	۱۹
<code>useDeferredValue</code>	نسخه «عقب‌افتاده» یک مقدار برای UI سنگین	۱۹

♦ هوک‌های دیگر

هوک	کاربرد	فصل
<code>useId</code>	شناسه یکتا و پایدار برای دسترسی‌پذیری (label/input)	۱۷
<code>useSyncExternalStore</code>	اتصال به store خارجی (Zustand زیر پوسته از آن استفاده می‌کند)	۲۰
<code>useDebugValue</code>	برچسب در DevTools برای هوک سفارشی	۱۴
<code>useFormStatus</code>	(۱۹, <code>react-dom</code>) وضعیت pending فرم والد	۹

♦ `use API` – هوکی که قانون اول را می‌شکند

```
import { use } from 'react';

function Comments({ commentsPromise }) {
  // ✅ می‌تواند داخل شرط باشد!
  if (!commentsPromise) return null;
  const comments = use(commentsPromise); // suspend تا resolve شود
  const theme = use(ThemeContext); // مثل useContext
  return comments.map(c => <Comment key={c.id} {...c} />);
}
```

`use` یک API جدید در React 19 است که Promise یا Context را می‌خواند و برخلاف هوک‌ها، داخل شرط و حلقه مجاز است. با Suspense یکپارچه است (فصل ۱۲).

انتخاب هوک درست



✗ اشتباه رایج: هوک بعد از return

```
function List({ items }) {  
  if (items.length === 0) return <Empty />; // ✗ return هوک قبل از  
  const [selected, setSelected] = useState(null);  
}
```

راه‌حل: همه هوک‌ها را بالای تابع، قبل از هر `return` قرار دهید.

“سؤالات مصاحبه (با پاسخ)”

Junior – چرا نمی‌توان هوک را داخل `if` صدا زد؟ چون React هوک‌ها را با ترتیب فراخوانی شناسایی می‌کند. اگر ترتیب بین رندها تغییر کند، state هوک‌ها جابه‌جا می‌شود.

Mid – `use` چه تفاوتی با `useContext` دارد؟ `useContext` همان کار را می‌کند اما داخل شرط و حلقه هم مجاز است و علاوه بر Promise، Context را هم می‌خواند (با Suspense ادغام می‌شود). تیم React توصیه می‌کند در کد جدید `use` را ترجیح دهید.

Senior – چرا React به‌جای ترتیب فراخوانی از «کلید» برای هوک‌ها استفاده نکرد؟ طراحی مبتنی بر ترتیب سه مزیت دارد: بدون نام‌گذاری اضافی (که در هوک‌های سفارشی ترکیبی تداخل می‌سازد)، بدون سربار جستجو در `map`، و امکان ترکیب آزاد هوک‌های سفارشی بدون نگرانی از برخورد نام. هزینه‌اش دو قانون است که ابزار lint به‌طور کامل بررسی می‌کند.

09 فرم‌ها و Actions در React 19

React 19 نحوه کار با فرم‌ها را متحول کرد. به جای مدیریت دستی loading و خطا، یک تابع `async` به فرم می‌دهید و React بقیه را انجام می‌دهد. این فصل مهم‌ترین تغییر عملی نسخه ۱۹ را عمیق آموزش می‌دهد.

مشکلی که Actions حل می‌کند

در React 18 یک فرم ساده «ثبت نظر» این‌طور بود:

```
const [text, setText] = useState('');
const [isPending, setIsPending] = useState(false);
const [error, setError] = useState<string | null>(null);

async function handleSubmit(e: FormEvent) {
  e.preventDefault();
  setIsPending(true);
  setError(null);
  try {
    await postComment(text);
    setText('');
  } catch (err) {
    setError((err as Error).message);
  } finally {
    setIsPending(false);
  }
}
```

سه `state`، یک `try/catch/finally` و `preventDefault` — برای هر فرم، در هر پروژه، با کیفیت متفاوت. React 19 این الگو را درون خودش آورد.

Action چیست؟

Action تابعی `async` است که داخل یک **transition** اجرا می‌شود. وقتی تابع `async` را به `<form action>`، `startTransition` یا `useActionState` می‌دهید، React به‌طور خودکار:

- ♦ وضعیت **pending** را مدیریت می‌کند،
- ♦ خطاها را به نزدیک‌ترین **Error Boundary** می‌فرستد،
- ♦ به‌روزرسانی خوش‌بینانه را با `useOptimistic` پشتیبانی می‌کند،
- ♦ فرم را پس از موفقیت ریست می‌کند (برای فرم‌های کنترل‌نشده).

<form action> – ساده‌ترین شکل

```
src/features/comments/CommentForm.tsx

export function CommentForm({ postId }: { postId: string }) {
  async function submit(formData: FormData) {
    const text = formData.get('text') as string;
    await postComment(postId, text);
    // فرم به صورت خودکار ریست می‌شود
  }

  return (
    <form action={submit}>
      <textarea name="text" required minLength={3} />
      <SubmitButton />
    </form>
  );
}
```

هیچ `useState`، هیچ `preventDefault`، هیچ `onSubmit`، `formData` همان `FormData` استاندارد وب است و مقادارها را با ویژگی `name` می‌خواند.

useFormStatus – دکمه‌ای که خودش می‌داند pending است

```
src/components/SubmitButton.tsx

import { useFormStatus } from 'react-dom';

export function SubmitButton({ children = 'ارسال' }) {
  const { pending } = useFormStatus();
  return (
    <button type="submit" disabled={pending}>
      {pending ? '...در حال ارسال' : children}
    </button>
  );
}
```

⚠️ `useFormStatus` فقط برای فرزندان فرم

این هوک وضعیت نزدیک‌ترین `<form>` والد را می‌خواند. اگر آن را در همان کامپوننتی که `<form>` را رندر می‌کند صدا بزنید، همیشه `pending: false` می‌گیرید. دکمه را به یک کامپوننت جدا منتقل کنید – همان‌طور که بالا انجام دادیم.

useActionState — وقتی به نتیجه Action نیاز دارید

برای نمایش خطای اعتبارسنجی یا پیام موفقیت:

```
src/features/auth/SignupForm.tsx

import { useActionState } from 'react';

type State = { error?: string; success?: boolean };

async function signupAction(prev: State, formData: FormData): Promise<State> {
  const email = String(formData.get('email') ?? '');
  const password = String(formData.get('password') ?? '');

  if (!email.includes('@')) return { error: 'ایمیل معتبر نیست' };
  if (password.length < 8) return { error: 'رمز باید حداقل ۸ کاراکتر باشد' };

  try {
    await api.signup({ email, password });
    return { success: true };
  } catch {
    return { error: 'خطای سرور؛ دوباره تلاش کنید' };
  }
}

export function SignupForm() {
  const [state, action, isPending] = useActionState(signupAction, {});

  if (state.success) return <p>✓☒ کنید بررسی را خود ایمیل. شد انجام ثبت نام</p>;

  return (
    <form action={action}>
      <input name="email" type="email" placeholder="ایمیل" />
      <input name="password" type="password" placeholder="رمز عبور" />
      {state.error && <p role="alert" className="error">{state.error}</p> }
      <button disabled={isPending}>{isPending ? '...' : 'ثبت نام'}</button>
    </form>
  );
}
```

بخش	توضیح
<code>useActionState(fn, initialState, permalink?)</code>	ورودی
<code>fn(prevState, formData)</code>	Action با state قبلی به عنوان آرگومان اول
<code>[state, formAction, isPending]</code>	خروجی: آخرین نتیجه، تابعی برای <code><form action></code> ، وضعیت

✦ الگوی نتیجه استاندارد

برای همه Action های پروژه یک نوع مشترک تعریف کنید:

```
type ActionResult<T> = { ok: true; data: T } | { ok: false; error: string; fieldErrors?: Record<string, string> }
```

نمایش خطاهای فیلدی را یکنواخت می کند و در فصل ۲۳ با `zod` ترکیب می شود.

نگه داشتن مقادیر فرم پس از خطا

فرم کنترل نشده بعد از Action ریست می شود؛ اگر خطا رخ داد کاربر نباید همه چیز را دوباره تایپ کند. مقادیر را در state برگردانید و به عنوان `defaultValue` بدهید:

```
async function action(prev, formData) {
  const values = Object.fromEntries(formData) as Record<string, string>;
  if (!values.email.includes('@')) return { error: 'ایمیل نامعتبر', values };
  // ...
}



```

Action روی دکمه: `formAction`

هر دکمه می تواند Action مخصوص خودش داشته باشد — مفید برای «ذخیره پیش نویس» و «انتشار» در یک فرم:

```
<form action={publish}>
  <textarea name="body" />
  <button type="submit">انتشار</button>
  <button type="submit" formAction={saveDraft}>ذخیره پیش نویس</button>
</form>
```

startTransition: Action خارج از فرم

Actions محدود به فرم نیستند. هر تابع async داخل `startTransition` یک Action است:

```
src/features/profile/DeleteAccount.tsx

import { useTransition } from 'react';

export function DeleteAccount() {
  const [isPending, startTransition] = useTransition();

  function handleDelete() {
    startTransition(async () => {
      await api.deleteAccount();
      navigate('/goodbye');
    });
  }

  return <button onClick={handleDelete} disabled={isPending}>حذف حساب</button>;
}
```

Progressive Enhancement

چون `<form action>` روی مکانیزم بومی فرم HTML سوار است، در فریم‌ورک‌هایی با Server Functions (Next.js، React Router) فرم حتی قبل از لود شدن JavaScript کار می‌کند. در یک SPA خالص این مزیت وجود ندارد، اما API یکسان است؛ یعنی مهارت شما مستقیماً به فریم‌ورک منتقل می‌شود.

➤ فرم تماس با ما

یک فرم با فیلدهای نام، ایمیل و پیام بسازید که: (۱) با `useActionState` اعتبارسنجی کند و خطای هر فیلد را زیر همان فیلد نشان دهد؛ (۲) دکمه ارسال با `useFormStatus` غیرفعال شود؛ (۳) پس از خطا مقادیر حفظ شوند؛ (۴) پس از موفقیت پیام تشکر جای فرم را بگیرد. برای شبیه‌سازی API از `await new Promise(r => setTimeout(r, 1000))` استفاده کنید.

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت `onSubmit` و `action` روی فرم چیست؟ `onSubmit` یک رویداد است که خودتان باید `preventDefault` کنید و همه چیز را دستی مدیریت کنید. `action` یک تابع (Action) می‌گیرد که `FormData` دریافت می‌کند و React به‌طور خودکار `pending`، ریست و خطا را مدیریت می‌کند.

Mid – چرا `useFormStatus` در همان کامپوننت فرم کار نمی‌کند؟ چون شبیه Context عمل می‌کند و وضعیت فرم والد را می‌خواند. کامپوننتی که خودش `<form>` را رندر می‌کند، فرزند آن فرم نیست.

Senior – Actions چگونه با Concurrent Rendering ادغام می‌شوند؟ Action داخل transition اجرا می‌شود؛ یعنی به‌روزرسانی‌های ناشی از آن غیرفوری (non-urgent) هستند و UI فعلی تا آماده‌شدن نتیجه تعاملی می‌ماند. React همچنین Action‌های متوالی را صف می‌کند تا ترتیب حفظ شود، و `useOptimistic` هنگام اتمام یا شکست transition به‌صورت خودکار به مقدار واقعی بازمی‌گردد.

10 useOptimistic و تجربه کاربری آنی

کاربران انتظار دارند رابط کاربری «فوری» واکنش نشان دهد، نه بعد از پاسخ سرور. React 19 با `useOptimistic` این الگو را که قبلاً پیچیده و خطاخیز بود، به چند خط تبدیل کرده است.

مفهوم به‌روزرسانی خوش‌بینانه

وقتی کاربر دکمه لایک را می‌زند، دو گزینه دارید: صبر کنید تا سرور پاسخ دهد (کند و سنگین) یا فرض کنید موفق می‌شود، UI را فوراً به‌روز کنید و اگر شکست خورد، برگردانید. گزینه دوم «Optimistic UI» است و تجربه‌ای مثل توئیتر و اینستاگرام را می‌سازد.

useOptimistic در عمل

```
src/features/posts/LikeButton.tsx

import { useOptimistic, useTransition } from 'react';

type Props = { postId: string; likes: number; liked: boolean };

export function LikeButton({ postId, likes, liked }: Props) {
  const [isPending, startTransition] = useTransition();
  const [optimistic, setOptimistic] = useOptimistic(
    { likes, liked },
    (current, next: boolean) => ({
      liked: next,
      likes: current.likes + (next ? 1 : -1),
    }),
  );

  function toggle() {
    startTransition(async () => {
      setOptimistic(!optimistic.liked); // فوراً در UI
      await api.toggleLike(postId);    // بعد سرور
      // جایگزین می‌شود props مقدار واقعی از transition، پس از اتمام
    });
  }

  return (
    <button onClick={toggle} disabled={isPending} aria-pressed={optimistic.liked}>
      {optimistic.liked ? '💔' : '❤️'} {optimistic.likes}
    </button>
  );
}
```

امضا: `const [optimisticValue, addOptimistic] = useOptimistic(actualValue, updateFn?)`

بخش	توضیح
<code>actualValue</code>	مقدار واقعی (از props یا state)
<code>updateFn(current, input)</code>	نحوه محاسبه مقدار خوش‌بینانه از ورودی
<code>optimisticValue</code>	آنچه رندر می‌کنید: تا پایان transition خوش‌بینانه، بعد واقعی
<code>addOptimistic(input)</code>	باید داخل transition یا Action صدا زده شود

i بازگشت خودکار (Rollback)

نیازی به کد rollback نیست. وقتی transition تمام شود (چه موفق چه با خطا)، `optimisticValue` به `actualValue` بازمی‌گردد. اگر والد پس از موفقیت داده جدید را پاس دهد، الی همان می‌ماند؛ اگر خطا رخ دهد، به مقدار قبلی برمی‌گردد. برای نمایش پیام خطا، در Action آن را catch کنید.

الگوی لیست: افزودن آنی به لیست نظرات

src/features/comments/Comments.tsx (1/2)

```
import { useOptimistic } from 'react';

type Comment = { id: string; text: string; pending?: boolean };
type Props = { comments: Comment[]; postId: string };

export function Comments({ comments, postId }: Props) {
  const [optimisticComments, addOptimistic] = useOptimistic(
    comments,
    (state, newText: string) => [
      ...state,
      { id: crypto.randomUUID(), text: newText, pending: true },
    ],
  );

  async function submit(formData: FormData) {
    const text = formData.get('text') as string;
    addOptimistic(text);
    await api.addComment(postId, text);
    // پاس می‌دهد props والد لیست جدید را واکنشی و به‌عنوان
  }
```

آیتم خوش‌بینانه با `pending: true` علامت می‌خورد تا در رندر، نیمه‌شفاف نمایش داده شود؛ وقتی والد لیست واقعی را پاس دهد، این پرچم دیگر وجود ندارد:

```
src/features/comments/Comments.tsx (2/2)

return (
  <>
    <ul>
      {optimisticComments.map((c) => (
        <li key={c.id} style={{ opacity: c.pending ? 0.5 : 1 }}>
          {c.text} {c.pending && <small>(در...ارسال حال در)</small>}
        </li>
      ))}
    </ul>
    <form action={submit}>
      <input name="text" required />
      <button>ارسال</button>
    </form>
  </>
);
}
```

فراخوانی `addOptimistic` داخل `<form action>` مجاز است چون Action خودش یک transition است.

الگوی حذف

```
const [optimisticItems, removeOptimistic] = useOptimistic(
  items,
  (state, idToRemove: string) => state.filter((i) => i.id !== idToRemove),
);

function handleDelete(id: string) {
  startTransition(async () => {
    removeOptimistic(id);
    await api.deleteItem(id);
    onRefresh(); // در فریم‌ورک revalidate یا
  });
}
```

کی از Optimistic استفاده نکنیم؟

✓ مناسب

- ♦ لایک، بوکمارک، فالو
- ♦ افزودن نظر، پیام چت
- ♦ تیک زدن تسک، مرتب‌سازی drag & drop
- ♦ تغییر تنظیمات ساده (تم، اعلان)

✗ نامناسب

- ♦ پرداخت و تراکنش مالی
- ♦ عملیات غیرقابل بازگشت (حذف حساب)
- ♦ عملیاتی که نتیجه‌اش غیرقابل پیش‌بینی است (تولید محتوا با AI)
- ♦ وقتی نرخ خطا بالاست (اتصال ضعیف شبکه)

✗ اشتباه رایج: صدا زدن خارج از transition

```
function toggle() {  
  setOptimistic(!liked); // ✗ هشدار: خارج از transition  
  api.toggleLike(id);  
}
```

`addOptimistic` باید داخل `startTransition(async () => {...})` یا Action فرم باشد؛ در غیر این صورت React هشدار می‌دهد و مقدار بلافاصله برمی‌گردد.

✦ ترکیب با `useActionState``

برای فرم‌هایی که هم به Optimistic UI و هم به پیام خطا نیاز دارند، هر دو هوک را کنار هم استفاده کنید: `useActionState` نتیجه/خطا را نگه می‌دارد و `useOptimistic` نمایش آنی را. Action مشترک اول `addOptimistic` را صدا می‌زند و بعد نتیجه را برمی‌گرداند.

“سؤالات مصاحبه (با پاسخ)”

Junior – Optimistic UI یعنی چه؟ نمایش نتیجه یک عملیات قبل از تأیید سرور، با فرض موفقیت؛ اگر شکست خورد، UI به حالت قبل برمی‌گردد.

Mid – `useOptimistic` چگونه می‌فهمد کی باید به مقدار واقعی برگردد؟ به `transition` می‌گوید که `addOptimistic` در آن صدا زده شده گوش می‌دهد. وقتی آن `transition` (شامل همه `await` ها) تمام شود، مقدار خوش‌بینانه دور ریخته می‌شود و `actualValue` رندر می‌شود.

Senior – چالش‌های Optimistic UI در لیست‌های همزمان (concurrent) چیست و React چگونه کمک می‌کند؟ چند Action هم‌زمان می‌توانند مقادیر خوش‌بینانه را روی هم انباشته کنند و ترتیب پاسخ سرور ممکن است با ترتیب ارسال متفاوت باشد. React به‌روزرسانی‌های خوش‌بینانه را روی آخرین مقدار واقعی دوباره اعمال (`rebase`) می‌کند و Action‌های یک فرم را صف می‌کند تا ترتیب حفظ شود. شناسه موقت (`crypto.randomUUID`) و فیلد `pending` به شما امکان تمایز آیتم‌های تأیید نشده را می‌دهد.

11 useEffect، useRef و useEffectEvent

`useEffect` پرکاربردترین و در عین حال بدفهمیده‌شده‌ترین هوک React است. این فصل مدل ذهنی درست (همگام‌سازی، نه چرخه حیات)، الگوهای `cleanup`، و هوک جدید `useEffectEvent` در نسخه ۱۹.۲ را آموزش می‌دهد.

Effect چیست و چه زمانی به آن نیاز داریم؟

Effect راهی است برای همگام‌کردن کامپوننت با چیزی بیرون از **React**: اتصال `WebSocket`، اشتراک در رویداد `window`، کتابخانه نمودار غیر-React، `document.title`، یا تایمر. Effect بعد از **commit** و پس از نقاشی صفحه اجرا می‌شود.

✗ Effect برای این کارها نیست

- ♦ تبدیل داده برای رندر: محاسبه `fullName` از `last + first` → مستقیماً در بدنه رندر.
 - ♦ واکنش به رویداد کاربر: ارسال فرم، خرید → در `event handler`.
 - ♦ ریست `state` با تغییر `props`: از `key` استفاده کنید.
 - ♦ واکنشی داده (**data fetching**): در اپ واقعی → `Suspense`/کتابخانه (فصل ۱۲). Effect خام برای `fetch` فقط برای یادگیری یا پروژه بسیار کوچک.
- اگر هیچ «سیستم خارجی» درگیر نیست، احتمالاً به Effect نیاز ندارید. مستندات رسمی یک صفحه کامل با عنوان **You Might Not Need an Effect** دارد.

```
src/features/chat/ChatRoom.tsx

import { useEffect, useState } from 'react';

export function ChatRoom({ roomId }: { roomId: string }) {
  const [messages, setMessages] = useState<string[]>([]);

  useEffect(() => {
    // ۱) راه اندازی (setup)
    const conn = createConnection(roomId);
    conn.on('message', (m) => setMessages((prev) => [...prev, m]));
    conn.connect();

    // ۲) unmount قبل از اجرای مجدد و هنگام پاک سازی (cleanup):
    return () => conn.disconnect();
  }, [roomId]); // ۳) هر بار roomId تغییر کند وابستگی‌ها: cleanup → setup

  return <ul>{messages.map((m, i) => <li key={i}>{m}</li>)}</ul>;
}
```

مدل ذهنی درست: Effect «چرخه حیات کامپوننت» نیست؛ بلکه هر Effect یک فرآیند همگام‌سازی مستقل را توصیف می‌کند: «چطور شروع کن» و «چطور متوقف کن». React هر وقت لازم بداند این چرخه را تکرار می‌کند – از جمله دو بار در StrictMode برای اطمینان از صحت cleanup.

آرایه وابستگی‌ها

رفتار	آرایه
بعد از هر رندر (تقریباً همیشه اشتباه)	بدون آرایه
فقط پس از mount (+ cleanup در unmount)	[]
پس از mount و هر بار a یا b تغییر کند	[a, b]

قاعده: هر مقدار reactive (state، props، متغیرهای محاسبه‌شده از آن‌ها) که داخل Effect استفاده می‌شود باید در آرایه باشد. ESLint این را بررسی می‌کند. **آرایه را دروغ نگویند؛** اگر می‌خواهید Effect به چیزی واکنش نشان ندهد، کد را طوری بنویسید که نیازی به آن نداشته باشد.

useEffectEvent — راه حل رسمی وابستگی‌های ناخواسته (React 19.2)

مشکل کلاسیک: می‌خواهید هنگام اتصال، یک notification با تم فعلی نشان دهید، اما تغییر `theme` نباید باعث reconnect شود:

✓ راه حل

```
const onConnected = useEffectEvent(
  () => showToast('متصل شد', theme),
);

useEffect(() => {
  const conn = createConnection(roomId);
  conn.on('connected', onConnected);
  conn.connect();
  return () => conn.disconnect();
}, [roomId]);
// ✓ Effect Event در وابستگی‌ها نیست
```

✗ مشکل

```
useEffect(() => {
  const conn = createConnection(roomId);
  conn.on('connected', () => {
    showToast('متصل شد', theme);
  });
  conn.connect();
  return () => conn.disconnect();
}, [roomId, theme]);
// !تغییر تم → قطع و وصل مجدد
```

`useEffectEvent` تابعی می‌سازد که همیشه آخرین `props/state` را می‌بیند اما reactive نیست. قوانین: فقط داخل Effect همان کامپوننت صدا زده شود، به کامپوننت دیگر پاس داده نشود، و در آرایه وابستگی‌ها قرار نگیرد. `eslint-plugin-react-hooks@6+` این‌ها را می‌داند.

✦ چه زمانی از `useEffectEvent` استفاده کنیم؟

فقط برای منطقی که مفهوماً یک «رویداد» است اما از داخل Effect شلیک می‌شود (مثل «وقتی متصل شد»، «وقتی صفحه دیده شد»). آن را برای ساکت کردن lint استفاده نکنید؛ اگر Effect واقعاً باید به مقداری واکنش نشان دهد، آن مقدار باید در وابستگی‌ها باشد.

واکشی داده با Effect (فقط برای یادگیری)

```
src/features/users/UserList.tsx

useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/users?q=${query}`, { signal: controller.signal })
    .then((r) => r.json())
    .then(setUsers)
    .catch((err) => {
      if (err.name !== 'AbortError') setError(err);
    });

  return () => controller.abort(); // می‌گیرد race condition جلوی
}, [query]);
```

بدون `AbortController`، اگر کاربر سریع تایپ کند، پاسخ درخواست قدیمی ممکن است بعد از جدید برسد و UI اشتباه شود. این «race condition» رایج‌ترین باگ `fetch` در `Effect` است. در فصل ۱۲ راه مدرن (`Suspense` + کتابخانه) را می‌بینید که این مشکلات را ندارد.

useRef — حافظه بدون رندر

`useRef` یک «جعبه» با ویژگی `current` است که بین رندها زنده می‌ماند اما تغییرش رندر نمی‌سازد. تفاوت آن با `useState` را یک‌بار برای همیشه به خاطر بسپارید:

useRef	useState	
✗	✓	تغییر باعث رندر می‌شود؟
✓	✓	مقدار بین رندها حفظ می‌شود؟
✗ (فقط در handler/Effect)	✓	خواندن در حین رندر
تایمر، مقدار قبلی، فلگ	داده UI	کاربرد

نمونه‌ی کلاسیک: شناسه‌ی تایمر و لحظه‌ی شروع، داده‌ی ال نیستند و نباید رندر بسازند – فقط `elapsed` باید state باشد:

```
src/components/Stopwatch.tsx

import { useRef, useState } from 'react';

export function Stopwatch() {
  const [elapsed, setElapsed] = useState(0);
  const intervalRef = useRef<number | null>(null); // مقدار قابل تغییر (۱)
  const startRef = useRef(0);

  function start() {
    startRef.current = Date.now() - elapsed;
    intervalRef.current = window.setInterval(() => {
      setElapsed(Date.now() - startRef.current);
    }, 50);
  }

  function stop() {
    if (intervalRef.current) clearInterval(intervalRef.current);
  }

  return (
    <>
      <p>{(elapsed / 1000).toFixed(2)} ثانیه</p>
      <button onClick={start}>شروع</button>
      <button onClick={stop}>توقف</button>
    </>
  );
}
```

```
src/components/SearchBox.tsx

export function SearchBox() {
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    inputRef.current?.focus(); // فوکوس خودکار پس از mount
  }, []);

  return <input ref={inputRef} placeholder="...جستجو" />;
}
```

React 19 دو قابلیت جدید به `ref` داد: `cleanup function` در `ref callback` `ref={e => { ...; return ... }}` و `ref` به عنوان `prop` معمولی (فصل ۵).

useLayoutEffect — وقتی باید قبل از paint اندازه بگیرید

```
useLayoutEffect(() => {
  const { height } = tooltipRef.current!.getBoundingClientRect();
  setPosition(height > spaceBelow ? 'top' : 'bottom');
}, []);
```

مثل `useEffect` است اما همگام و قبل از نقاشی اجرا می‌شود؛ برای جلوگیری از پرش بصری هنگام اندازه‌گیری DOM. چون مرورگر را بلاک می‌کند، فقط در همین موارد استفاده کنید.

جمع‌بندی فصل

- ◆ Effect = همگام‌سازی با بیرون React، نه چرخه حیات.
- ◆ هر Effect باید `cleanup` متناظر داشته باشد اگر چیزی راه می‌اندازد.
- ◆ وابستگی‌ها را صادقانه بنویسید؛ برای منطق رویدادی از `useEffectEvent` استفاده کنید.
- ◆ `useRef` برای مقادیری که رندر لازم ندارند و برای DOM.
- ◆ برای داده، به `Suspense` و کتابخانه‌ها بروید (فصل بعد).

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا Effect در StrictMode دو بار اجرا می‌شود؟ React عمداً `setup→cleanup→setup` را شبیه‌سازی می‌کند تا مطمئن شود `cleanup` درست نوشته شده. اگر دوبار اجرا شدن مشکل می‌سازد، `cleanup` شما ناقص است.

Mid – تفاوت `useEffect` و `useLayoutEffect` چیست؟ `useEffect` بعد از `paint` و به صورت غیرهمگام اجرا می‌شود؛ `useLayoutEffect` بعد از تغییر DOM ولی قبل از `paint` و همگام. دومی فقط برای اندازه‌گیری/تنظیم `layout` که باید قبل از دیده شدن انجام شود.

Senior – `useEffectEvent` چه مشکلی را حل می‌کند که `useRef` + `useEffect` به طور کامل حل نمی‌کرد؟ الگوی «latest ref» (ذخیره `callback` در `ref` و به‌روزرسانی در `Effect`) یک پنجره زمانی دارد که `ref` هنوز مقدار قدیمی است، و همچنین `linter` نمی‌تواند صحت آن را بررسی کند. `useEffectEvent` تضمین می‌کند تابع همیشه آخرین مقادیر `commit` شده را ببیند، قوانین استفاده را قابل `lint` می‌کند و به‌صراحت بیان می‌کند این منطق `reactive` نیست.

12 واکنشی داده، Suspense و use

واکنشی داده در React 19 یک پارادایم جدید دارد: به جای «loading state دستی»، کامپوننت هنگام نبود داده «تعليق» می‌شود و Suspense نمایش fallback را مدیریت می‌کند. این فصل مدل ذهنی، API use و راه‌حل production با TanStack Query را آموزش می‌دهد.

مشکل رویکرد سنتی

با `useState` + `useEffect` هر کامپوننت سه حالت (`loading/error/data`) را دستی مدیریت می‌کند، «آبشار درخواست‌ها» (`waterfall`) می‌سازد (فرزند تا `mount` نشود `fetch` نمی‌کند)، و هیچ کش، `retry` یا `deduplication` ندارد. Suspense این مسئولیت‌ها را از کامپوننت به لایه بالاتر منتقل می‌کند.

مدل ذهنی Suspense

```
<Suspense fallback={<Skeleton />}>
  <UserProfile />    { /* می‌شود "suspend" اگر داده‌اش آماده نباشد */ }
  <UserPosts />      { /* هر دو با هم منتظر می‌مانند */ }
</Suspense>
```

وقتی کامپوننتی داخل Suspense هنوز داده‌اش آماده نیست، React رندر آن زیردرخت را متوقف و `fallback` را نشان می‌دهد. به محض آماده‌شدن، محتوای واقعی جایگزین می‌شود. Suspense خودش داده `fetch` نمی‌کند؛ فقط به «سیگنال تعليق» واکنش نشان می‌دهد.

```
src/features/users/UserProfile.tsx

import { use, Suspense } from 'react';

type User = { id: string; name: string; bio: string };

function UserDetails({ userPromise }: { userPromise: Promise<User> }) {
  const user = use(userPromise); // می‌شود suspend شدن تا resolve شدن
  return (
    <article>
      <h2>{user.name}</h2>
      <p>{user.bio}</p>
    </article>
  );
}

export function UserProfile({ id }: { id: string }) {
  // ⚠ Promise باید پایدار باشد؛ نه ساخته‌شده در هر رندر
  const userPromise = useMemo(() => fetchUser(id), [id]);

  return (
    <Suspense fallback=<p>...بارگذاری حال در</p>>
      <UserDetails userPromise={userPromise} />
    </Suspense>
  );
}
```

⚠ تله بزرگ: Promise ساخته‌شده در رندر

`use(fetchUser(id))` را مستقیم ننویسید. هر رندر یک Promise جدید می‌سازد، React دوباره suspend می‌شود، رندر می‌شود، Promise جدید... حلقه بی‌نهایت. Promise باید یا از والد بیاید (بهترین: از Server Component یا loader روتر)، یا کش شده باشد (`useMemo`، کتابخانه، یا `cache()` در RSC). این محدودیت باعث شده در SPA خالص، کتابخانه‌هایی مثل TanStack Query راه‌حل عملی باشند.

| Error Boundary – مدیریت خطای Suspense

اگر `Promise reject` شود، خطا به نزدیک‌ترین **Error Boundary** می‌رسد. React هنوز API هوکی برای آن ندارد؛ از پکیج کوچک `react-error-boundary` استفاده کنید:

```
src/app/DataSection.tsx

import { ErrorBoundary } from 'react-error-boundary';
import { Suspense } from 'react';

export function DataSection() {
  return (
    <ErrorBoundary
      fallbackRender={({ error, resetErrorBoundary }) => (
        <div role="alert">
          <p>آمد پیش مشکلی: {error.message}</p>
          <button onClick={resetErrorBoundary}>مجدد تلاش</button>
        </div>
      )}
    >
      <Suspense fallback=<Skeleton />>
        <UserProfile id="42" />
      </Suspense>
    </ErrorBoundary>
  );
}
```

الگوی استاندارد: `ErrorBoundary` بیرون، `Suspense` داخل. هر «بخش مستقل» صفحه یک جفت از این دو داشته باشد تا خطای یک ویجت کل صفحه را ن خواباند.

چیدمان مرزهای Suspense

مرزهای تودرتو

```
<Header />
<Suspense fallback={<FeedSkeleton />}>
  <Feed />
</Suspense>
<Suspense fallback={<SidebarSkeleton />}>
  <Sidebar />
</Suspense>
```

هر بخش مستقل ظاهر می‌شود؛ تجربه «پیش‌رونده».

یک مرز بزرگ

```
<Suspense fallback={<PageSkeleton />}>
  <Header />
  <Feed />
  <Sidebar />
</Suspense>
```

همه یک‌جا ظاهر می‌شوند؛ کندترین بخش، همه را نگه می‌دارد.

React 19.2 در SSR مرزهای Suspense را **دسته‌ای آشکار می‌کند** (batched reveal) تا از ظاهرشدن پشت‌سرهم و آزاردهنده جلوگیری شود؛ رفتار کلاینت مشابه است.

راه‌حل Production: TanStack Query

برای یک SPA واقعی به کش، refetch، retry در فوکوس، pagination و mutation نیاز دارید. **TanStack Query** (v5) استاندارد صنعت است و از Suspense پشتیبانی کامل دارد:

Terminal

```
npm install @tanstack/react-query
```

src/main.tsx

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry: 1 } },
});

createRoot(root).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>,
);
```

src/features/products/ProductList.tsx

```
import { useSuspenseQuery } from '@tanstack/react-query';

export function ProductList({ category }: { category: string }) {
  const { data } = useSuspenseQuery({
    queryKey: ['products', category], // کلید کش
    queryFn: () => api.getProducts(category), // تابع واکنشی
  });

  return <ul>{data.map((p) => <li key={p.id}>{p.title}</li>)}</ul>;
}

// والد: <Suspense fallback={<Skeleton/>}><ProductList category="books"/></Suspense>
```

`useSuspenseQuery` تضمین می‌کند `data` همیشه تعریف شده است (نه `undefined`): `loading` و `error` به `Suspense/ErrorHandler` سپرده می‌شوند. برای تغییر داده:

src/features/products/AddProduct.tsx

```
import { useMutation, useQueryClient } from '@tanstack/react-query';

export function AddProduct() {
  const qc = useQueryClient();
  const mutation = useMutation({
    mutationFn: api.createProduct,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['products'] }),
  });

  return (
    <form action={({fd}) => mutation.mutate({ title: fd.get('title') as string })}>
      <input name="title" required />
      <button disabled={mutation.isPending}>افزودن</button>
    </form>
  );
}
```


جلوگیری از آبشار (Waterfall)

آبشار زمانی است که درخواست دوم فقط بعد از اتمام اولی شروع می‌شود، چون کامپوننت فرزند تا رندر والد mount نمی‌شود. راه حل‌ها:

راهکار	توضیح
Fetch در والد، پاس به فرزند	Promise ها را همزمان بسازید و با <code>use</code> در فرزندان بخوانید
Prefetch در روتر	<code>loader</code> در React Router / TanStack Router قبل از رندر شروع می‌کند
<code>queryClient.prefetchQuery</code>	هنگام hover روی لینک، داده صفحه بعد را بگیرید
Server Components	(فصل ۲۸) واکنشی روی سرور بدون آبشار کلاینت

مقایسه رویکردها

TanStack Query	Suspense + <code>use</code>	<code>useState</code> + <code>useEffect</code>	
✓	✗ (خودتان)	✗	کش
هر دو حالت	Suspense/Boundary	دستی	Loading/Error
ندارد	ندارد	باید خودتان حل کنید	Race condition
SPA واقعی	با فریم‌ورک/RSC	آموزش، پروژه خیلی کوچک	مناسب

➤ ویتترین محصولات

با API عمومی <https://dummyjson.com/products> یک صفحه بسازید: لیست محصولات با `useSuspenseQuery`، فیلتر دسته‌بندی (هر دسته یک `queryKey`)، Skeleton هنگام بارگذاری، `ErrorBoundary` با دکمه تلاش مجدد، و `prefetch` جزئیات محصول هنگام hover روی کارت.

“ سؤالات مصاحبه (با پاسخ)

Junior – Suspense چه کاری انجام می‌دهد؟ نمایش fallback تا زمانی که کامپوننت‌های داخلش (که داده یا کد lazy منتظرند) آماده شوند. خودش داده واکنشی نمی‌کند.

Mid – چرا `use(fetch(...))` مستقیم داخل کامپوننت اشتباه است؟ هر رندر Promise جدیدی می‌سازد و React نمی‌تواند تشخیص دهد همان درخواست قبلی است؛ نتیجه حلقه suspend بی‌پایان است. Promise باید خارج از رندر ساخته یا کش شود.

Senior – TanStack Query چگونه با Suspense و transitions تعامل دارد و کجا باید مراقب بود؟ `useSuspenseQuery` هنگام نبود داده suspend می‌کند و React 19 با transitions می‌تواند به‌جای نمایش fallback، UI قدیمی را نگه دارد (وقتی تغییر کلید داخل `startTransition` باشد). نکته مهم: بدون prefetch، هر `useSuspenseQuery` در سطوح مختلف درخت می‌تواند آبشار بسازد؛ برای صفحات پیچیده queryها را در loader روتر یا با `useSuspenseQueries` موازی کنید.

13 useReducer و Context

وقتی داده باید در نقاط دور درخت در دسترس باشد (تم، کاربر لاگین شده، زبان)، Context راه حل است. وقتی منطق تغییر state پیچیده می شود، useReducer آن را متمرکز و قابل تست می کند. ترکیب این دو، الگوی مدیریت state داخلی React بدون کتابخانه است.

| مشکل prop drilling

وقتی `user` باید از `App` به `Avatar` در عمق پنج سطح برسد و سه کامپوننت میانی فقط آن را «رد می کنند»، به آن prop drilling می گویند. اول راه حل های ساده تر را امتحان کنید (ترکیب با `children`، فصل ۵)؛ اگر کافی نبود، Context.

| ساخت Context – سینتکس React 19

```
src/contexts/ThemeContext.tsx

import { createContext, use, useState, type ReactNode } from 'react';

type Theme = 'light' | 'dark';
type ThemeContextValue = { theme: Theme; toggle: () => void };

const ThemeContext = createContext<ThemeContextValue | null>(null);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>('light');
  const toggle = () => setTheme((t) => (t === 'light' ? 'dark' : 'light'));

  // React 19: Provider به عنوان Context خود
  return <ThemeContext value={{ theme, toggle }}>{children}</ThemeContext>;
}

// هوک سفارشی: مصرف امن با پیام خطای واضح
export function useTheme() {
  const ctx = use(ThemeContext);
  if (!ctx) throw new Error('useTheme باید داخل <ThemeProvider> استفاده شود');
  return ctx;
}
```

src/components/ThemeToggle.tsx

```
import { useTheme } from '@contexts/ThemeContext';

export function ThemeToggle() {
  const { theme, toggle } = useTheme();
  return <button onClick={toggle}>{theme === 'light' ? '☺' : '☀'}</button>;
}
```

i سه تغییر React 19 در Context

۱) `<Context value>` به جای `<Context.Provider value>` (فرم قدیمی هنوز کار می‌کند اما deprecated است). ۲) `use(Context)` به جای `useContext` که داخل شرط هم مجاز است. ۳) `Context.Consumer` عملاً منسوخ شده است.

کارایی Context: مشکل و راه حل

هر بار `value` تغییر کند، همه مصرف‌کننده‌ها رندر می‌شوند. دو اشتباه رایج:

✓ راه حل

```
// خودکار حل می‌شود با React Compiler
// بدون کامپایلر:
const value = useMemo(
  () => ({ theme, toggle }),
  [theme]
);
<ThemeContext value={value}>
```

✗ شیء جدید در هر رندر

```
<ThemeContext value={{ theme, toggle }}>
// والد رندر شود → شیء جدید
// همه مصرف‌کننده‌ها رندر می‌شوند
```

راهکار مهم‌تر: Context را بشکنید. داده‌هایی که با فرکانس متفاوت تغییر می‌کنند در Context های جدا باشند. مثلاً `UserContext` (به‌ندرت) و `NotificationsContext` (مکرراً). همچنین `state` و `dispatch` را جدا کنید تا کامپوننت‌هایی که فقط `dispatch` می‌خواهند، با تغییر `state` رندر نشوند.

چه چیزی در Context بگذاریم؟

نامناسب ❌	مناسب ✅
داده سرور (→ TanStack Query)	تم، زبان، جهت (RTL)
state فرم (محلی نگه دارید)	کاربر احراز هویت شده
داده‌ای که مرتب تغییر می‌کند (موقعیت موس)	تنظیمات سراسری، feature flags
هر چیزی که فقط ۲ سطح پایین لازم است	dispatch یک reducer سراسری

useReducer – منطق متمرکز تغییر state

وقتی چند `useState` با هم تغییر می‌کنند یا تغییرات قوانین دارند، reducer خواناتر و قابل‌تست‌تر است:

src/features/cart/cartReducer.ts (1/2)

```
export type CartItem = { id: string; title: string; price: number; qty: number };
export type CartState = { items: CartItem[]; coupon: string | null };

export type CartAction =
  | { type: 'added'; item: Omit<CartItem, 'qty'> }
  | { type: 'removed'; id: string }
  | { type: 'qtyChanged'; id: string; qty: number }
  | { type: 'couponApplied'; code: string }
  | { type: 'cleared' };

export const initialCart: CartState = { items: [], coupon: null };
```

نوع `CartAction` یک **discriminated union** است؛ به همین دلیل TypeScript داخل هر `case` دقیقاً می‌داند کدام فیلدها در دسترس‌اند. خودِ reducer یک تابع خالص است: state جدید برمی‌گرداند و هرگز ورودی را تغییر نمی‌دهد:

```
src/features/cart/cartReducer.ts (2/2)

export function cartReducer(state: CartState, action: CartAction): CartState {
  switch (action.type) {
    case 'added': {
      const exists = state.items.find((i) => i.id === action.item.id);
      const items = exists
        ? state.items.map((i) =>
            i.id === action.item.id ? { ...i, qty: i.qty + 1 } : i,
          )
        : [...state.items, { ...action.item, qty: 1 }];
      return { ...state, items };
    }
    case 'removed':
      return { ...state, items: state.items.filter((i) => i.id !== action.id) };
    case 'qtyChanged':
      return {
        ...state,
        items: state.items
          .map((i) => (i.id === action.id ? { ...i, qty: action.qty } : i))
          .filter((i) => i.qty > 0),
      };
    case 'couponApplied':
      return { ...state, coupon: action.code };
    case 'cleared':
      return initialCart;
  }
}
```

نکات: reducer یک تابع خالص است (state جدید برمی‌گرداند، mutate نمی‌کند)، نام action‌ها «چه اتفاقی افتاد» را می‌گویند (added) نه «چه کار کن» (setItems)، و TypeScript با union نوع action، هر case را دقیقاً تایپ می‌کند.

```
src/features/cart/CartContext.tsx

import { createContext, use, useReducer, type ReactNode, type Dispatch } from 'react';
import { cartReducer, initialCart } from './cartReducer';
import type { CartState, CartAction } from './cartReducer';

const CartStateContext = createContext<CartState | null>(null);
const CartDispatchContext = createContext<Dispatch<CartAction> | null>(null);

export function CartProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(cartReducer, initialCart);
  return (
    <CartStateContext value={state}>
      <CartDispatchContext value={dispatch}>{children}</CartDispatchContext>
    </CartStateContext>
  );
}

export const useCart = () => use(CartStateContext)!;
export const useCartDispatch = () => use(CartDispatchContext)!;
```

dispatch هویت پایداری دارد؛ کامپوننتی که فقط useCartDispatch را مصرف می‌کند (مثل دکمه «افزودن به سبد») با تغییر سبد رندر نمی‌شود.

```
src/features/products/AddToCartButton.tsx

export function AddToCartButton({ product }: { product: Product }) {
  const dispatch = useCartDispatch();
  const add = () => dispatch({ type: 'added', item: product });
  return <button onClick={add}>افزودن</button>;
}
```

تست reducer – بدون React

```
src/features/cart/cartReducer.test.ts

import { cartReducer, initialCart } from './cartReducer';

test('افزودن دوباره همان محصول qty را زیاد می‌کند', () => {
  const item = { id: '1', title: 'کتاب', price: 100 };
  let s = cartReducer(initialCart, { type: 'added', item });
  s = cartReducer(s, { type: 'added', item });
  expect(s.items).toEqual([ ...item, qty: 2 ]);
});
```

چون reducer تابع خالص است، تستش نه به DOM نیاز دارد نه به رندر.

useReducer یا useState؟

نشانه	انتخاب
یک یا دو مقدار مستقل	useState
چند مقدار که با هم تغییر می‌کنند	useReducer
منطق انتقال پیچیده (اعتبارسنجی، شرط)	useReducer
می‌خواهید منطق را جدا تست کنید	useReducer
state باید بین صفحات/تب‌ها زنده بماند	کتابخانه (فصل ۲۰)

“سؤالات مصاحبه (با پاسخ)”

Junior – Context چه زمانی رندر مجدد ایجاد می‌کند؟ هر بار `value` پاس داده شده به Provider با مقایسه `Object.is` تغییر کند، همه کامپوننت‌هایی که آن Context را مصرف می‌کنند رندر می‌شوند – صرف‌نظر از این‌که از کدام بخش `value` استفاده می‌کنند.

Mid – چرا state و dispatch را در دو Context جدا می‌گذاریم؟ چون `dispatch` هویت ثابتی دارد؛ کامپوننت‌هایی که فقط action ارسال می‌کنند نباید با هر تغییر state رندر شوند. جداسازی، رندهای غیرضروری را حذف می‌کند.

Senior – Context جایگزین Redux/Zustand است؟ نه دقیقاً. Context یک مکانیزم انتقال (dependency injection) است، نه مدیریت state؛ selector ندارد و هر تغییر، همه مصرف‌کننده‌ها را رندر می‌کند. برای state سراسری پرتغییر با نیاز به انتخاب جزئی، ابزارهای مبتنی بر `useSyncExternalStore` (Zustand، Jotai، Redux Toolkit) مناسب‌ترند. برای داده کم‌تغییر (تم، کاربر)، Context کاملاً کافی است.

14 هوک‌های سفارشی

هوک سفارشی مهم‌ترین ابزار انتزاع در React مدرن است: منطق stateful را از کامپوننت جدا می‌کنید، نام‌گذاری معنادار می‌دهید و در چند جا استفاده می‌کنید. این فصل اصول طراحی و ده هوک کاربردی آماده تحویل می‌دهد.

| هوک سفارشی چیست؟

یک تابع جاوااسکریپت معمولی که نامش با `use` شروع می‌شود و داخلش از هوک‌های دیگر استفاده می‌کند. هیچ جادویی نیست؛ پیشوند `use` قراردادی است که به ESLint و React Compiler می‌گوید قوانین هوک را روی آن اعمال کنند.

```
src/hooks/useLocalStorage.ts

import { useState, useEffect } from 'react';

export function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    try {
      const raw = localStorage.getItem(key);
      return raw ? (JSON.parse(raw) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}

// useState استفاده - دقیقاً مثل
const [theme, setTheme] = useLocalStorage<'light' | 'dark'>('theme', 'light');
```

i هر فراخوانی، state مستقل

دو کامپوننت که `useLocalStorage('theme', ...)` را صدا می‌زنند **state** مشترک ندارند؛ هرکدام نمونه خودشان را دارند. هوک‌ها **منطق** را به اشتراک می‌گذارند، نه **state** را. برای **state** مشترک از Context یا store استفاده کنید.

اصول طراحی هوک خوب

اصل	توضیح
نام از روی هدف	❌ <code>useEffectWrapper</code> ، ✅ <code>useOnlineStatus</code>
API شبیه هوک‌های داخلی	tuple برای state-like (<code>[value, setValue]</code>)، object برای چندمقداره
ورودی reactive بپذیرید	اگر <code>url</code> تغییر کرد، هوک باید واکنش نشان دهد
<code>as const</code> برای tuple	تا TypeScript آن را آرایه ثابت با تایپ دقیق بداند
مسئولیت واحد	<code>useFetch</code> که همزمان <code>cache</code> و <code>auth</code> هم انجام دهد، بد است
مستقل از UI	هوک نباید JSX برگرداند (آن کامپوننت است)

ده هوک پرکاربرد

`useToggle` ♦

src/hooks/useToggle.ts

```
import { useCallback, useState } from 'react';

export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle, setOn] as const;
}
```

```
src/hooks/useMediaQuery.ts

import { useSyncExternalStore } from 'react';

export function useMediaQuery(query: string) {
  const subscribe = (cb: () => void) => {
    const mql = window.matchMedia(query);
    mql.addEventListener('change', cb);
    return () => mql.removeEventListener('change', cb);
  };
  const getSnapshot = () => window.matchMedia(query).matches;
  const getServerSnapshot = () => false; // برای SSR
  return useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot);
}

const isMobile = useMediaQuery('(max-width: 768px)');
```

راه درست اتصال به منابع بیرونی (مرورگر، store) است: بدون tearing و سازگار با `useSyncExternalStore` Concurrent Rendering.

```
src/hooks/useOnlineStatus.ts

import { useSyncExternalStore } from 'react';

const subscribe = (cb: () => void) => {
  window.addEventListener('online', cb);
  window.addEventListener('offline', cb);
  return () => {
    window.removeEventListener('online', cb);
    window.removeEventListener('offline', cb);
  };
};

export const useOnlineStatus = () =>
  useSyncExternalStore(subscribe, () => navigator.onLine, () => true);
```

src/hooks/usePrevious.ts

```
import { useEffect, useRef } from 'react';

export function usePrevious<T>(value: T) {
  const ref = useRef<T | undefined>(undefined);
  useEffect(() => {
    ref.current = value;
  }, [value]);
  return ref.current;
}
```

src/hooks/useClickOutside.ts

```
import { useEffect, useEffectEvent, type RefObject } from 'react';

export function useClickOutside(
  ref: RefObject<HTMLInputElement | null>,
  onOutside: () => void,
) {
  const handleOutside = useEffectEvent(onOutside); // همیشه آخرین callback

  useEffect(() => {
    const listener = (e: PointerEvent) => {
      if (ref.current && !ref.current.contains(e.target as Node)) handleOutside();
    };
    document.addEventListener('pointerdown', listener);
    return () => document.removeEventListener('pointerdown', listener);
  }, [ref]);
}
```

با `useEffectEvent` دیگر لازم نیست مصرف‌کننده callback را با `useCallback` بپیچد.

```

src/hooks/useIntersectionObserver.ts

import { useEffect, useState, type RefObject } from 'react';

export function useIntersectionObserver(
  ref: RefObject<Element | null>,
  options?: IntersectionObserverInit,
) {
  const [isIntersecting, setIntersecting] = useState(false);

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    const observer = new IntersectionObserver(
      ([entry]) => setIntersecting(entry.isIntersecting),
      options,
    );
    observer.observe(el);
    return () => observer.disconnect();
  }, [ref, options?.rootMargin, options?.threshold]);

  return isIntersecting;
}

```

```

src/hooks/useCopyToClipboard.ts

import { useState } from 'react';

export function useCopyToClipboard(resetAfter = 2000) {
  const [copied, setCopied] = useState(false);

  async function copy(text: string) {
    await navigator.clipboard.writeText(text);
    setCopied(true);
    setTimeout(() => setCopied(false), resetAfter);
  }

  return { copied, copy };
}

```

```
// React 19: نیازی به هوک نیست!
function ProductPage({ product }) {
  return (
    <>
      <title>{product.title} | فروشگاه</title>
      <meta name="description" content={product.summary} />
      <article>...</article>
    </>
  );
}
```

React 19 تگ‌های `<title>`، `<meta>` و `<link>` را از هر جای درخت به `<head>` منتقل می‌کند (فصل ۱۷).

useDebounceCallback ♦

```
src/hooks/useDebounceCallback.ts

import { useEffect, useEffectEvent, useRef } from 'react';

export function useDebounceCallback<A extends unknown[]>(
  fn: (...args: A) => void,
  delay = 300,
) {
  const timer = useRef<number | null>(null);
  const stable = useEffectEvent(fn);

  useEffect(() => () => { if (timer.current) clearTimeout(timer.current); }, []);

  return (...args: A) => {
    if (timer.current) clearTimeout(timer.current);
    timer.current = window.setTimeout(() => stable(...args), delay);
  };
}
```

```
src/hooks/useAsync.ts

import { useState, useTransition } from 'react';

export function useAsync<T>(fn: () => Promise<T>) {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<Error | null>(null);
  const [isPending, startTransition] = useTransition();

  function run() {
    startTransition(async () => {
      try {
        setError(null);
        setData(await fn());
      } catch (e) {
        setError(e as Error);
      }
    });
  }

  return { data, error, isPending, run };
}
```

useDebugValue — برچسب در DevTools

```
export function useOnlineStatus() {
  const online = useSyncExternalStore(...);
  useDebugValue(online ? 'Online 🟢' : 'Offline 🔴');
  return online;
}
```

کتابخانه‌های هوک آماده

ویژگی	کتابخانه
تایپ‌شده، سبک، متداول‌ترین هوک‌ها	usehooks-ts
مدرن، مستندات تعاملی عالی	@uidotdev/usehooks
بسیار جامع (۱۰۰+ هوک) اما قدیمی‌تر	react-use
از Alibaba؛ هوک‌های پیشرفته درخواست و کش	ahooks

✦ قبل از نوشتن، جستجو کنید – ولی بفهمید

هوک‌های بالا را اغلب نباید از صفر بنویسید، اما باید بتوانید بنویسید. در مصاحبه، «یک `useDebounce` بنویس» سؤال بسیار رایجی است.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا نام هوک سفارشی باید با `use` شروع شود؟ تا ESLint و React Compiler بدانند قوانین هوک داخل آن اعمال می‌شود (ترتیب فراخوانی ثابت) و شما هم بلافاصله بفهمید این تابع state یا Effect دارد و نمی‌توان آن را در شرط صدا زد.

Mid – تفاوت هوک سفارشی با تابع کمکی معمولی چیست؟ تابع کمکی خالص است و به چرخه رندر وابسته نیست (می‌توانید هر جا صدایش بزنید). هوک از هوک‌های React استفاده می‌کند، بنابراین به «نمونه کامپوننت» متصل است و قوانین هوک بر آن حاکم است.

Senior – چرا `useSyncExternalStore` را به `useState` + `useEffect` برای منابع بیرونی ترجیح می‌دهیم؟ در React، Concurrent Rendering ممکن است رندر را متوقف کند و بین دو بخش درخت، مقدار store بیرونی تغییر کند؛ نتیجه «tearing» است (دو کامپوننت دو مقدار متفاوت نشان می‌دهند). `useSyncExternalStore` با خواندن همگام snapshot و اجبار به رندر همگام در صورت تغییر، سازگاری را تضمین می‌کند و همچنین `getServerSnapshot` را برای hydration بدون mismatch فراهم می‌کند.

15 TypeScript در React

TypeScript دیگر «انتخاب» نیست؛ استاندارد پروژه‌های جدی React است. این فصل تمام الگوهای تایپ‌دهی که در کار روزمره لازم دارید را در یک جا جمع کرده و تغییرات تایپی 19 React را پوشش می‌دهد.

چرا TypeScript؟

خطاها را قبل از اجرا می‌گیرد، تکمیل خودکار دقیق می‌دهد، ری‌فکتور را امن می‌کند و **مستندات زنده** است: تایپ props یعنی هر کس بداند کامپوننت چه می‌خواهد. هزینه اولیه یادگیری در هفته اول جبران می‌شود.

تایپ‌دهی Props

```
src/components/Badge.tsx

import type { ReactNode, ComponentProps } from 'react';

// هم مجاز است؛ در تیم یکی را انتخاب کنید (interface برای type ۱)
type BadgeProps = {
  children: ReactNode; // هر چیز قابل‌رندر
  variant?: 'info' | 'success' | 'danger'; // union به جای string
  count?: number;
  onDismiss?: () => void;
  icon?: ReactNode;
};

export function Badge(props: BadgeProps) {
  const { children, variant = 'info', count, onDismiss, icon } = props;
  return (
    <span className={`badge badge-${variant}`}>
      {icon}
      {children}
      {count !== undefined && <b>{count}</b>}
      {onDismiss && <button onClick={onDismiss} aria-label="بستن"></button>}
    </span>
  );
}

// ۲) عنصر بومی props به ارث بردن
type InputProps = ComponentProps<'input'> & { label: string; error?: string };
```

کاربرد	تایپ
هر چیزی که رندر می‌شود (JSX، string، null) – برای <code>children</code>	<code>ReactNode</code>
فقط JSX (نه string/null) – وقتی باید حتماً عنصر باشد	<code>ReactElement</code>
همه props بومی یک تگ	<code>ComponentProps<'button'></code>
props یک کامپوننت دیگر	<code>ComponentProps<typeof Button></code>
<code>P & { children?: ReactNode }</code>	<code>PropsWithChildren<P></code>
شیء style	<code>CSSProperties</code>

✗ `React.FC` را فراموش کنید

در پروژه‌های قدیمی رایج بود اما اکنون توصیه نمی‌شود: `const Comp: React.FC<Props> = ...`
`children` را به‌طور ضمنی اضافه نمی‌کند (از React 18)، جنریک را سخت می‌کند و مزیت خاصی ندارد. تابع معمولی با props تایپ‌شده بنویسید.

تایپ‌دهی State

```
const [count, setCount] = useState(0); // استنتاج: number
const [user, setUser] = useState<User | null>(null); // صریح وقتی مقدار اولیه null
const [items, setItems] = useState<Item[]>([]); // آرایه خالی نیاز به تایپ دارد
const [status, setStatus] = useState<'idle' | 'loading' | 'done'>('idle');
```

تایپ‌دهی رویدادها

```
import type { ChangeEvent, FormEvent, KeyboardEvent, MouseEvent } from 'react';

const onChange = (e: ChangeEvent<HTMLInputElement>) => setValue(e.target.value);
const onSelect = (e: ChangeEvent<HTMLSelectElement>) => ...;
const onSubmit = (e: FormEvent<HTMLFormElement>) => { e.preventDefault(); };
const onKeyDown = (e: KeyboardEvent<HTMLInputElement>) => e.key === 'Enter' && ...;
const onClick = (e: MouseEvent<HTMLButtonElement>) => ...;

// خودش تایپ را استنتاج می‌کند TypeScript، بنویسید inline handler را ترفند: اگر
<input onChange={e => setValue(e.target.value)} /> // e تایپ دارد
```

تایپ‌دهی Ref

```
const inputRef = useRef<HTMLInputElement>(null); // DOM: شروع با null
const timerRef = useRef<number | null>(null); // مقدار قابل‌تغییر
const countRef = useRef(0); // استنتاج: RefObject<number>

// React 19: ref به‌عنوان prop
type InputProps = ComponentProps<'input'> & { ref?: React.Ref<HTMLInputElement> };
```

i تغییر `useRef` React 19: آرگومان اجباری

در React 19، `useRef()` بدون آرگومان خطای تایپ می‌دهد؛ باید `useRef<T>(null)` یا `useRef<T>()` بنویسید. همچنین `RefObject.current` دیگر `readonly` نیست و `MutableRefObject` منسوخ شده است.

تایپ‌دهی Context و Reducer

```
// Context با null (فصل ۱۳) و هوک نگهبان
const AuthContext = createContext<AuthValue | null>(null);

// Reducer: union برای action ⇒ TypeScript هر case را دقیق می‌داند
type Action =
  | { type: 'added'; item: Item }
  | { type: 'removed'; id: string };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case 'added': return ...; // action.item در دسترس
    case 'removed': return ...; // action.id در دسترس
  }
}
```

```
src/components/Select.tsx

type SelectProps<T> = {
  options: T[];
  value: T | null;
  onChange: (value: T) => void;
  getLabel: (option: T) => string;
  getKey: (option: T) => string;
};

export function Select<T>(props: SelectProps<T>) {
  const { options, value, onChange, getLabel, getKey } = props;
  return (
    <select
      value={value ? getKey(value) : ''}
      onChange={(e) => {
        const found = options.find((o) => getKey(o) === e.target.value);
        if (found) onChange(found);
      }}
    >
      {options.map((o) => (
        <option key={getKey(o)} value={getKey(o)}>{getLabel(o)}</option>
      ))}
    </select>
  );
}

// به صورت خودکار استنتاج می‌شود T = City استفاده
<Select
  options={cities}
  value={city}
  onChange={setCity}
  getLabel={(c) => c.name}
  getKey={(c) => c.id}
/>
```

Discriminated Union برای props متقابلاً انحصاری

```
type ButtonProps =
  | { as: 'button'; onClick: () => void; href?: never }
  | { as: 'link'; href: string; onClick?: never };

// <Button as="link" onClick={...} /> ← ❌ خطای کامپایل
```

تایپ‌دهی Actions و useActionState

```
type FormState = {
  error?: string;
  fieldErrors?: Partial<Record<'email' | 'password', string>>;
};

async function loginAction(prev: FormState, formData: FormData): Promise<FormState> {
  /* ... */
}

const initialState: FormState = {};
const [state, action, isPending] = useActionState(loginAction, initialState);
```

as const و satisfies

```
const routes = {
  home: '/',
  profile: '/profile',
} as const satisfies Record<string, `/${string}`>;
// ☒ حفظ می‌شود و هم‌زمان با الگو بررسی می‌شود literal مقدار
type Route = (typeof routes)[keyof typeof routes]; // '/' | '/profile'
```

پیکربندی توصیه‌شده tsconfig

در 6 TypeScript گزینه‌های `baseUrl`، `moduleResolution: "node10"` و `target: "ES5"` منسوخ شده‌اند (و در نسخه ۷ حذف می‌شوند)؛ مسیرهای `paths` را نسبی به `tsconfig` بنویسید (`./src/*`):

```
tsconfig.app.json

{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noUnusedLocals": true,
    "verbatimModuleSyntax": true,
    "skipLibCheck": true,
    "paths": { "@/*": [".src/*"] }
  },
  "include": ["src"]
}
```

`verbatimModuleSyntax` شما را مجبور می‌کند تایپ‌ها را با `import type` وارد کنید — که برای tree-shaking و سازگاری با ابزارهای Rust-محور (Rollup، SWC) ضروری است.

✦ فایل‌های تایپ سراسری

برای `import.meta.env` و فایل‌های استاتیک، Vite `vite-env.d.ts` را می‌سازد. متغیرهای محیطی خودتان را در آن تعریف کنید تا تکمیل خودکار داشته باشید:

```
interface ImportMetaEnv { readonly VITE_API_URL: string }
```

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت `ReactNode` و `ReactElement` چیست؟ `ReactElement` فقط خروجی JSX است؛ `ReactNode` مجموعه بزرگتری است که `boolean`، `null`، `number`، `string`، آرایه و `ReactElement` را شامل می‌شود. برای `children` تقریباً همیشه `ReactNode`.

Mid – چطور `props` یک دکمه سفارشی را طوری تایپ می‌کنید که همه ویژگی‌های `<button>` را بپذیرد؟ با `{ ... } & ComponentProps<'button'>` و `spread` کردن باقی‌مانده روی `<button>`. اگر می‌خواهید `prop` خاصی را حذف کنید: `Omit<ComponentProps<'button'>, 'type'>`.

Senior – Discriminated Union در طراحی API کامپوننت چه مزیتی دارد و محدودیتش چیست؟ ترکیب‌های نامعتبر `props` را در زمان کامپایل غیرممکن می‌کند (مثلاً `href` بدون `as="link"`). محدودیت: `destructuring` در ابتدای تابع باعث ازدست‌رفتن `narrowing` می‌شود؛ باید ابتدا روی `props.as` شرط بگذارید و بعد فیلدها را بخوانید، یا از تابع کمکی `type-guard` استفاده کنید.

16 روتینگ با React Router v8

هر اپلیکیشن واقعی چند صفحه دارد. React Router v8 با API مبتنی بر داده (loader/action) واکنشی داده را به لایه مسیر منتقل می‌کند و آبشار درخواست‌ها را از بین می‌برد. این فصل حالت «Data» آن را که برای SPA مناسب است، عمیق آموزش می‌دهد.

نصب و پیکربندی

Terminal

```
npm install react-router
```

i نسخه ۸

از v7 به بعد همه‌چیز در یک پکیج `react-router` است و `react-router-dom` حذف شده؛ فقط `RouterProvider` (وابسته به DOM) از مسیر `react-router/dom` وارد می‌شود. v8 (تابستان ۲۰۲۶) همان API حالت Data را دارد – تغییر اصلی، فعال‌بودن پیش‌فرض `middleware` و حداقل‌های `Node 22` و `React 19.2` است.

React Router v8 سه حالت دارد: **Declarative** (ساده، فقط `<Routes>`)، **Data** (loader/action) – توصیه ما برای **Framework** (SPA و SSR) و فایل‌محور، جانشین Remix). این فصل روی حالت Data تمرکز دارد. پی‌گیری حالت Data دو فایل دارد – تعریف مسیرها و اتصال روتر به ریشه:

src/router.tsx

```
import { createBrowserRouter } from 'react-router';
import { RootLayout } from '../layouts/RootLayout';
import { HomePage } from '../pages/HomePage';
import { ProductPage, productLoader } from '../pages/ProductPage';
import { ErrorPage } from '../pages/ErrorPage';

export const router = createBrowserRouter([
  {
    path: '/',
    element: <RootLayout />,
    errorElement: <ErrorPage />,
    children: [
      { index: true, element: <HomePage /> },
      { path: 'products/:id', element: <ProductPage />, loader: productLoader },
      { path: 'about', lazy: () => import('../pages/AboutPage') },
      { path: '*', element: <NotFound /> },
    ],
  },
]);
```

src/main.tsx

```
import { RouterProvider } from 'react-router-dom';
import { router } from '../router';

createRoot(root).render(<RouterProvider router={router} />);
```

src/layouts/RootLayout.tsx

```
import { Outlet, NavLink } from 'react-router';

export function RootLayout() {
  return (
    <>
      <nav>
        <NavLink to="/" end>خانه</NavLink>
        <NavLink to="/about" className={({ isActive }) => (isActive ? 'active' : '')}>
          درباره
        </NavLink>
      </nav>
      <main>
        <Outlet /> { /* صفحه فرزند اینجا رندر می‌شود */ }
      </main>
    </>
  );
}
```

`<NavLink>` نسخه‌ای از `<Link>` است که وضعیت `active` را می‌داند. `end` یعنی فقط برای تطابق دقیق `active` باشد (وگرنه `/` با همه مسیرها تطابق دارد).

Loader – داده قبل از رندر

src/pages/ProductPage.tsx

```
import { useLoaderData, type LoaderFunctionArgs } from 'react-router';

export async function productLoader({ params }: LoaderFunctionArgs) {
  const res = await fetch(`/api/products/${params.id}`);
  if (!res.ok) throw new Response('Not Found', { status: 404 });
  return res.json() as Promise<Product>;
}

export function ProductPage() {
  const product = useLoaderData() as Product;
  return <h1>{product.title}</h1>;
}
```

مزیت: روتر قبل از رندر و به صورت موازی برای همه مسیرهای تودرتو loader را اجرا می‌کند؛ آنبشار حذف می‌شود. `throw new Response(...)` مستقیماً به `errorElement` می‌رود.

✦ ترکیب با TanStack Query

loader می‌تواند فقط `queryClient.ensureQueryData(...)` را صدا بزند تا کش گرم شود، و کامپوننت با `useSuspenseQuery` بخواند. این‌طور هم ناوبری آنی دارید و هم کش، `refetch` و `mutation` کتابخانه را.

| نمایش تدریجی با `defer-style: Await`

برای صفحاتی که بخشی از داده کند است، Promise را از loader برگردانید (بدون `await`) و با `<Suspense>` + `<Await>` نمایش دهید:

```
src/pages/DashboardPage.tsx

import { Suspense } from 'react';
import { Await, useLoaderData } from 'react-router';

export function dashboardLoader() {
  return {
    user: fetchUser(),           // Promise - بدون await
    stats: fetchStats(),         // Promise کند
  };
}

export function DashboardPage() {
  const { user, stats } = useLoaderData() as ReturnType<typeof dashboardLoader>;
  return (
    <>
      <Suspense fallback=<{<p>...</p>}>
        <Await resolve={user}>{(u) => <h1>سلام {u.name}</h1>}</Await>
      </Suspense>
      <Suspense fallback=<{<StatsSkeleton />}>
        <Await resolve={stats}>{(s) => <StatsPanel stats={s} />}</Await>
      </Suspense>
    </>
  );
}
```

Action – تغییر داده در سطح مسیر

src/pages/NewPostPage.tsx

```
import { Form, redirect, useNavigation, type ActionFunctionArgs } from 'react-router';

export async function newPostAction({ request }: ActionFunctionArgs) {
  const formData = await request.formData();
  const title = formData.get('title') as string;
  if (title.length < 3) return { error: 'عنوان کوتاه است' };
  const post = await api.createPost({ title });
  return redirect(`/posts/${post.id}`);
}

export function NewPostPage() {
  const navigation = useNavigation();
  const busy = navigation.state === 'submitting';
  return (
    <Form method="post">
      <input name="title" />
      <button disabled={busy}>{busy ? '...' : 'انتشار'}</button>
    </Form>
  );
}
```

<Form> روتر مثل <form action> React 19 کار می‌کند، اما پس از موفقیت همه loaderهای فعال را دوباره اجرا (revalidate) می‌کند تا UI با سرور همگام بماند.

ناوبری برنامه‌ریزی‌شده و پارامترها

```
import { useNavigate, useParams, useSearchParams, useLocation } from 'react-router';

const navigate = useNavigate();
navigate('/login'); // push
navigate(-1); // back
navigate('/done', { replace: true }); // بدون تاریخچه

const { id } = useParams<{ id: string }>();
const [searchParams, setSearchParams] = useSearchParams();
const page = Number(searchParams.get('page') ?? 1);
setSearchParams({ page: String(page + 1) });

const location = useLocation(); // pathname, search, state
```

مسیرهای محافظت‌شده

```
src/router.tsx

async function requireAuth() {
  const user = await auth.getUser();
  if (!user) throw redirect('/login?from=' + encodeURIComponent(location.pathname));
  return user;
}

{
  path: 'dashboard',
  loader: requireAuth, // کاربر لاگین نکرده هرگز صفحه را نمی‌بیند
  element: <DashboardLayout />,
  children: [...]
}
```

بررسی در loader بهتر از بررسی در کامپوننت است: قبل از رندر انجام می‌شود و فلش UI محافظت‌شده رخ نمی‌دهد.

Code Splitting با lazy

```
// Component و Loader هایی به نام export: فایل صفحه
export async function loader() { ... }
export function Component() { ... }

// روتر
{ path: 'reports', lazy: () => import('./pages/ReportsPage') }
```

هر صفحه یک chunk جدا می‌شود و فقط هنگام نیاز دانلود می‌شود — بدون `React.lazy` و `Suspense` دستی.

اسکرول و انتقال صفحه

```
import { ScrollRestoration } from 'react-router';
// در RootLayout: <ScrollRestoration /> موقعیت اسکرول را هنگام back/forward حفظ می‌کند

// در فصل ۳۲ Hero image جزئیات و) View Transitions API انیمیشن انتقال با
<Link to="/gallery" viewTransition>گالری</Link>
```

گزینه جایگزین: TanStack Router

اگر type-safety کامل (پارامترها و search params تایپ‌شده در زمان کامپایل) اولویت است، **TanStack Router** انتخاب بسیار قوی و در حال رشدی است. API آن مفاهیم مشابه (loader، layout، lazy) دارد؛ مهارت شما قابل انتقال است.

➤ مینی‌بلاگ

سه صفحه بسازید: لیست پست‌ها (/)، جزئیات پست (/posts/:id) و ایجاد پست (/posts/new). از loader برای داده، action برای ایجاد، errorElement برای ۴۰۴، و lazy برای صفحه ایجاد استفاده کنید. API: <https://jsonplaceholder.typicode.com/posts>.

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت <a href> و <Link to> چیست؟ <a> صفحه را کامل بارگذاری مجدد می‌کند و state از بین می‌رود؛ <Link> با History API آدرس را عوض می‌کند و فقط کامپوننت‌های لازم رندر می‌شوند.

Mid – loader چه مزیتی نسبت به fetch در useEffect دارد؟ موازی‌سازی خودکار برای مسیرهای تودرتو، شروع واکنشی قبل از رندر (بدون آبشار)، مدیریت خطای یکپارچه با errorElement، revalidation خودکار پس از action، و امکان prefetch هنگام hover.

Senior – حالت Data و Framework در React Router v8 چه تفاوتی دارند و کی به Framework مهاجرت کنیم؟ حالت Data روی کلاینت اجرا می‌شود (SPA) و همان مفاهیم loader/action را دارد. حالت Framework (میراث Remix) همین API را روی سرور اجرا می‌کند: Streaming، SSR، مسیرهای فایل‌محور، Server Actions و type-safety تولیدشده. مهاجرت وقتی توجیه دارد که TTFB، SEO، یا کاهش JS کلاینت اولویت شود؛ چون API مشترک است، هزینه مهاجرت پایین است.

17 استایل‌دهی، Tailwind v4 و Metadata

رابط کاربری زیبا و قابل‌نگهداری به یک استراتژی استایل روشن نیاز دارد. این فصل گزینه‌های مدرن را مقایسه می‌کند، Tailwind v4 را با پشتیبانی RTL راه‌اندازی می‌کند و قابلیت جدید React 19 برای مدیریت `<head>` را معرفی می‌کند.

گزینه‌های استایل‌دهی در ۲۰۲۶

روش	مزیت	عیب	مناسب
CSS Modules	بومی Vite، scoped runtime، بدون	نام‌گذاری دستی، بدون design token	پروژه‌های میانی
Tailwind v4	سریع، سازگار، design system داخلی	HTML شلوغ، منحنی یادگیری	اکثر پروژه‌ها
CSS-in-JS runtime (styled-components)	dynamic styles راحت	سر بار runtime، ناسازگار با RSC	پروژه‌های قدیمی
Zero-runtime CSS-in-JS (Panda, Vanilla Extract)	تایپ‌شده + بدون runtime	پیکربندی بیشتر	Design System بزرگ
shadcn/ui	کامپوننت‌های قابل مالکیت روی Tailwind + Radix	نه یک کتابخانه؛ کد را نگه می‌دارید	اپ‌های واقعی

CSS Modules

src/components/Card.module.css

```
.card { border-radius: 12px; padding: 16px; box-shadow: 0 1px 4px rgb(0 0 0 / 8%); }
.title { font-weight: 700; }
```

```
import styles from './Card.module.css';
<div className={styles.card}><h3 className={styles.title}>...</h3></div>
```

Vite نام کلاس‌ها را یکتا می‌کند (`Card_card_x7f2a`)؛ تداخل بین کامپوننت‌ها غیرممکن است.

نسخه ۴ (۲۰۲۵) پیکربندی را به خود CSS منتقل کرد و نیازی به `tailwind.config.js` نیست:

Terminal

```
npm install tailwindcss @tailwindcss/vite
```

vite.config.ts

```
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({ plugins: [react(), tailwindcss()] });
```

src/index.css

```
@import "tailwindcss";

@theme {
  --font-sans: "Vazirmatn", system-ui, sans-serif;
  --color-brand-500: oklch(0.65 0.18 250);
  --color-brand-600: oklch(0.58 0.18 250);
}
```

```
<button
  className="rounded-lg bg-brand-500 px-4 py-2 text-white
    hover:bg-brand-600 disabled:opacity-50"
>
  ذخیره
</button>
```


Tailwind در RTL

از **logical properties** استفاده کنید تا با `dir="rtl"` خودکار درست شوند:

معنی	بنویسید	به جای
margin-inline-start/end	<code>me-4</code> / <code>ms-4</code>	<code>mr-4</code> / <code>ml-4</code>
padding-inline	<code>pe-4</code> / <code>ps-4</code>	<code>pr-4</code> / <code>pl-4</code>
inset-inline	<code>end-0</code> / <code>start-0</code>	<code>right-0</code> / <code>left-0</code>
تراز منطقی	<code>text-start</code>	<code>text-left</code>
گوشه منطقی	<code>rounded-s-lg</code>	<code>rounded-l-lg</code>

برای موارد خاص از variant های `rtl:` و `ltr:` استفاده کنید: `rtl:rotate-180`.

ترکیب کلاس‌ها: `tailwind-merge` + `clsx`

src/lib/cn.ts

```
import { clsx, type ClassValue } from 'clsx';
import { twMerge } from 'tailwind-merge';

export const cn = (...inputs: ClassValue[]) => twMerge(clsx(inputs));
```

```
<div className={cn('p-4 bg-white', isActive && 'bg-brand-500', className)} />
// twMerge تضاد bg-white و bg-brand-500 (آخری برنده است) را حل می‌کند
```

src/components/ui/Button.tsx

```
import { cva, type VariantProps } from 'class-variance-authority';
import type { ComponentProps } from 'react';
import { cn } from '@lib/cn';

const button = cva('inline-flex items-center rounded-lg font-medium transition', {
  variants: {
    variant: {
      primary: 'bg-brand-500 text-white hover:bg-brand-600',
      ghost: 'bg-transparent hover:bg-slate-100',
      danger: 'bg-red-600 text-white hover:bg-red-700',
    },
    size: { sm: 'h-8 px-3 text-sm', md: 'h-10 px-4', lg: 'h-12 px-6 text-lg' },
  },
  defaultVariants: { variant: 'primary', size: 'md' },
});

type ButtonProps = ComponentProps<'button'> & VariantProps<typeof button>;

export function Button({ variant, size, className, ...props }: ButtonProps) {
  return <button className={cn(button({ variant, size }), className)} {...props} />;
}
```

این دقیقاً الگویی است که **shadcn/ui** استفاده می‌کند. با `npm install shadcn@latest add button dialog` کامپوننت‌های آماده و دسترس‌پذیر (بر پایه Radix) به پروژه شما کپی می‌شوند؛ مالک کد هستید و هر چه بخواهید تغییر می‌دهید.

Metadata داخلی در React 19

قبلاً برای تغییر `<title>` به `react-helmet` نیاز داشتید. React 19 تگ‌های `<title>`، `<meta>` و `<link>` را از هر جای درخت به `<head>` منتقل می‌کند:

```
src/pages/ProductPage.tsx

export function ProductPage({ product }: { product: Product }) {
  return (
    <>
      <title>`${product.title} | فروشگاه`</title>
      <meta name="description" content={product.summary} />
      <meta property="og:image" content={product.image} />
      <link rel="canonical" href={`https://shop.example/p/${product.slug}`} />

      <article>...</article>
    </>
  );
}
```

نکات Metadata ⚠

- ♦ `<title>` باید یک رشته واحد باشد: ☒ `` ${a} | ${b} `` — ☐ `{a} | {b}` (آرایه می‌شود).
- ♦ در SPA خالص این تگ‌ها بعد از اجرای JS اعمال می‌شوند؛ خزنده‌ها ممکن است آن‌ها را نبینند. برای SEO واقعی به SSR/فریم‌ورک نیاز است (فصل ۲۸).
- ♦ Stylesheet هم پشتیبانی می‌شود: `<link rel="stylesheet" href="..." precedence="default" />` — React ترتیب و بارگذاری را مدیریت می‌کند.

فونت فارسی و بهینه‌سازی

```
src/index.css

@font-face {
  font-family: "Vazirmatn";
  src: url("/fonts/Vazirmatn[wght].woff2") format("woff2-variations");
  font-weight: 100 900;
  font-display: swap;
}
```

از فونت متغیر (variable) استفاده کنید تا یک فایل همه وزن‌ها را پوشش دهد؛ `font-display: swap` از نامرئی‌ماندن متن جلوگیری می‌کند. فونت را در `public/fonts/` بگذارید و در `index.html` با `<link rel="preload" as="font" crossorigin>` پیش‌بارگذاری کنید.

src/index.css

```
@custom-variant dark (&:where(.dark, .dark *));
```

```
// در ThemeProvider (۱۳ فصل):
useEffect(() => {
  document.documentElement.classList.toggle('dark', theme === 'dark');
}, [theme]);

<div className="bg-white text-slate-900 dark:bg-slate-900 dark:text-slate-100">
```

♦ جلوگیری از فلش تم

تم ذخیره‌شده را با یک اسکریپت inline کوچک در `<head>` (قبل از بارگذاری React) روی `<html>` اعمال کنید تا کاربر تم روشن را برای یک لحظه نبیند.

“ سؤالات مصاحبه (با پاسخ)

Junior – CSS Modules چطور از تداخل کلاس‌ها جلوگیری می‌کند؟ در زمان build هر نام کلاس را با یک hash یکتا ترکیب می‌کند و شیء نگاشتی می‌سازد که از آن استفاده می‌کنید. دو فایل با `.title` هرگز برخورد نمی‌کنند.

Mid – چرا twMerge لازم است؟ چون در CSS، ترتیب تعریف کلاس در stylesheet برنده است نه ترتیب در `className`. اگر `p-4` و بعد `p-2` بنویسید، نتیجه غیرقطعی است. `twMerge` کلاس‌های متضاد Tailwind را می‌شناسد و فقط آخری را نگه می‌دارد.

Senior – چرا CSS-in-JS runtime با Server Components سازگار نیست و راه‌حل چیست؟ این کتابخانه‌ها استایل را در زمان رندر با Context و `useInsertionEffect` تزریق می‌کنند؛ Server Components نه Context کلاینت دارند نه Effect. راه‌حل: ابزارهای (Tailwind، Panda، Vanilla Extract، StyleX) zero-runtime که CSS را در زمان build استخراج می‌کنند.

18 پروژه عملی: مدیریت وظایف کامل

وقت آن است که همه چیز را کنار هم بگذاریم. در این فصل یک اپلیکیشن مدیریت وظایف (Task Manager) با فیلتر، ویرایش درجا، به روزرسانی خوش بینانه، ذخیره در `localStorage` و تم تاریک می سازیم – با معماری ای که در پروژه واقعی هم قابل استفاده است.

آنچه می سازیم

- ◆ لیست وظایف با افزودن، تیک زدن، ویرایش درجا و حذف
- ◆ فیلتر (همه / فعال / انجام شده) از طریق `search params` (قابل bookmark)
- ◆ **Optimistic UI** برای تیک زدن و حذف با API شبیه سازی شده
- ◆ ذخیره پایدار در `localStorage`
- ◆ تم تاریک/روشن با `Context`
- ◆ ساختار **feature-based** و `TypeScript` کامل

ساختار پروژه



گام ۱: تایپ‌ها و API شبیه‌سازی‌شده

src/features/tasks/types.ts

```
export type Task = { id: string; title: string; done: boolean; createdAt: number };
export type Filter = 'all' | 'active' | 'done';
```

src/features/tasks/api.ts

```
import type { Task } from './types';

const KEY = 'tasks:v1';
const delay = (ms: number) => new Promise((r) => setTimeout(r, ms));
const read = (): Task[] => JSON.parse(localStorage.getItem(KEY) ?? '[]');
const write = (tasks: Task[]) => localStorage.setItem(KEY, JSON.stringify(tasks));

export const tasksApi = {
  async list() {
    await delay(300);
    return read();
  },
  async add(title: string) {
    await delay(500);
    const task: Task = {
      id: crypto.randomUUID(), title, done: false, createdAt: Date.now(),
    };
    write([task, ...read()]);
    return task;
  },
  async toggle(id: string) {
    await delay(400);
    if (Math.random() < 0.15) throw new Error('خطای شبکه (شبیه‌سازی)');
    write(read().map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  },
  async rename(id: string, title: string) {
    await delay(300);
    write(read().map((t) => (t.id === id ? { ...t, title } : t)));
  },
  async remove(id: string) {
    await delay(400);
    write(read().filter((t) => t.id !== id));
  },
};
```

گام ۲: صفحه اصلی با loader و search params

```
src/features/tasks/TasksPage.tsx

import { useLoaderData, useRevalidator, useSearchParams } from 'react-router';
import { tasksApi } from './api';
import type { Filter, Task } from './types';
import { AddTaskForm } from './AddTaskForm';
import { FilterTabs } from './FilterTabs';
import { TaskList } from './TaskList';

export const tasksLoader = () => tasksApi.list();

export function TasksPage() {
  const tasks = useLoaderData() as Task[];
  const { revalidate } = useRevalidator();
  const [params] = useSearchParams();
  const filter = (params.get('filter') ?? 'all') as Filter;

  const visible = tasks.filter((t) =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done,
  );
  const remaining = tasks.filter((t) => !t.done).length;

  return (
    <section className="mx-auto max-w-xl space-y-4 p-4">
      <header className="flex items-center justify-between">
        <h1 className="text-2xl font-bold">وظایف</h1>
        <span className="text-sm text-slate-500">باقی‌مانده مورد {remaining}</span>
      </header>

      <AddTaskForm onAdded={revalidate} />
      <FilterTabs value={filter} />
      <TaskList tasks={visible} onChange={revalidate} />
    </section>
  );
}
```

`useRevalidator` loader را دوباره اجرا می‌کند؛ ساده‌ترین راه همگام‌سازی پس از تغییر بدون کتابخانه اضافی.

src/features/tasks/AddTaskForm.tsx

```
import { useActionState } from 'react';
import { tasksApi } from './api';
import { Button } from '@components/ui/Button';

type State = { error?: string };

export function AddTaskForm({ onAdded }: { onAdded: () => void }) {
  const [state, action, isPending] = useActionState(
    async (_prev: State, formData: FormData): Promise<State> => {
      const title = String(formData.get('title') ?? '').trim();
      if (title.length < 2) return { error: 'عنوان حداقل ۲ کاراکتر باشد' };
      await tasksApi.add(title);
      onAdded();
      return {};
    },
    {},
  );

  return (
    <form action={action} className="flex gap-2">
      <input
        name="title"
        placeholder="...کار جدید"
        autoComplete="off"
        className="flex-1 rounded-lg border px-3 py-2"
        aria-invalid={!!state.error}
      />
      <Button disabled={isPending}>{isPending ? '...' : 'افزودن'}</Button>
      {state.error && <p role="alert" className="text-red-600">{state.error}</p>}
    </form>
  );
}
```


src/features/tasks/FilterTabs.tsx

```
import { NavLink } from 'react-router';
import type { Filter } from './types';

const tabs: { value: Filter; label: string }[] = [
  { value: 'all', label: 'همه' },
  { value: 'active', label: 'فعال' },
  { value: 'done', label: 'انجام‌شده' },
];

export function FilterTabs({ value }: { value: Filter }) {
  return (
    <nav className="flex gap-1 rounded-lg bg-slate-100 p-1 dark:bg-slate-800">
      {tabs.map((t) => (
        <NavLink
          key={t.value}
          to={t.value === 'all' ? '.' : `?filter=${t.value}`}
          className={`flex-1 rounded-md py-1 text-center text-sm ${
            value === t.value ? 'bg-white shadow dark:bg-slate-700' : ''
          }`}
        >
          {t.label}
        </NavLink>
      ))}
    </nav>
  );
}
```

src/features/tasks/TaskList.tsx (1/2)

```
import { useOptimistic, useTransition } from 'react';
import { tasksApi } from './api';
import type { Task } from './types';
import { TaskItem } from './TaskItem';

type Optimistic =
  | { kind: 'toggle'; id: string }
  | { kind: 'remove'; id: string };
type Props = { tasks: Task[]; onChange: () => void };

export function TaskList({ tasks, onChange }: Props) {
  const [, startTransition] = useTransition();
  const [optimisticTasks, apply] = useOptimistic(tasks, (state, op: Optimistic) =>
    op.kind === 'remove'
      ? state.filter((t) => t.id !== op.id)
      : state.map((t) => (t.id === op.id ? { ...t, done: !t.done } : t)),
  );

  function run(op: Optimistic, request: () => Promise<void>) {
    startTransition(async () => {
      apply(op);
      try {
        await request();
      } catch (e) {
        alert((e as Error).message); // toast: در پروژه واقعی
      }
      onChange(); // موفق یا ناموفق، با سرور همگام شو
    });
  }
}
```

تابع `run` قلب این کامپوننت است: ابتدا تغییر خوش‌بینانه را اعمال می‌کند، بعد درخواست واقعی را می‌فرستد و در هر حال با `onChanged` داده را از سرور تازه می‌کند. بخش رندر فقط این تابع را به آیتم‌ها وصل می‌کند:

```
src/features/tasks/TaskList.tsx (2/2)

if (optimisticTasks.length === 0)
  return <p className="py-8 text-center text-slate-400">👉 نیست نمایش برای چیزی</p>;

return (
  <ul className="divide-y rounded-lg border">
    {optimisticTasks.map((task) => (
      <TaskItem
        key={task.id}
        task={task}
        onToggle={() =>
          run({ kind: 'toggle', id: task.id }, () => tasksApi.toggle(task.id))
        }
        onRemove={() =>
          run({ kind: 'remove', id: task.id }, () => tasksApi.remove(task.id))
        }
        onRename={(title) => tasksApi.rename(task.id, title).then(onChanged)}
      />
    ))}
  </ul>
);
}
```

توجه کنید که با ۱۵٪ خطای تصادفی در `toggle`، تیک به‌صورت خودکار برمی‌گردد — بدون هیچ کد `rollback`.

src/features/tasks/TaskItem.tsx (1/2)

```
import { useState, useRef, useEffect } from 'react';
import type { Task } from '../types';

type Props = {
  task: Task;
  onToggle: () => void;
  onRemove: () => void;
  onRename: (title: string) => void;
};

export function TaskItem({ task, onToggle, onRemove, onRename }: Props) {
  const [editing, setEditing] = useState(false);
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    if (editing) inputRef.current?.select();
  }, [editing]);

  function commit(value: string) {
    const title = value.trim();
    if (title && title !== task.title) onRename(title);
    setEditing(false);
  }
}
```

و ادامه همان فایل — JSX آیتم. کلید طراحی: در حالت ویرایش، ورودی **uncontrolled** است (`defaultValue`) و فقط هنگام `blur` یا `Enter` مقدار نهایی خوانده می‌شود؛ بنابراین هر کلید، رندر مجدد لیست را نمی‌سازد:

```
src/features/tasks/TaskItem.tsx (2/2)

return (
  <li className="flex items-center gap-3 px-3 py-2">
    <input
      type="checkbox"
      checked={task.done}
      onChange={onToggle}
      aria-label="انجام شد"
    />

    {editing ? (
      <input
        ref={inputRef}
        defaultValue={task.title}
        className="flex-1 rounded border px-2 py-1"
        onBlur={(e) => commit(e.target.value)}
        onKeyDown={(e) => {
          if (e.key === 'Enter') commit(e.currentTarget.value);
          if (e.key === 'Escape') setEditing(false);
        }}
      />
    ) : (
      <span
        onDoubleClick={() => setEditing(true)}
        className={`flex-1 ${task.done ? 'text-slate-400 line-through' : ''}`}
      >
        {task.title}
      </span>
    )}

    <button
      onClick={onRemove}
      aria-label="حذف"
      className="text-slate-400 hover:text-red-600"
    >
      ×
    </button>
  </li>
);
}
```

src/app/router.tsx

```
import { createBrowserRouter } from 'react-router';
import { RootLayout } from './RootLayout';
import { TasksPage, tasksLoader } from '@/features/tasks/TasksPage';

export const router = createBrowserRouter([
  {
    path: '/',
    element: <RootLayout />,
    children: [{ index: true, element: <TasksPage />, loader: tasksLoader }],
  },
]);
```

src/main.tsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { RouterProvider } from 'react-router-dom';
import { ThemeProvider } from '@/contexts/ThemeContext';
import { router } from '@app/router';
import './index.css';

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <ThemeProvider>
      <RouterProvider router={router} />
    </ThemeProvider>
  </StrictMode>,
);
```

چه چیزی یاد گرفتیم؟

مفهوم	کجا استفاده شد
Actions و <code>useActionState</code>	فرم افزودن با اعتبارسنجی
<code>useTransition</code> + <code>useOptimistic</code>	تیک و حذف آتی با rollback خودکار
revalidate و Loader	واکشی اولیه و همگام‌سازی
Search params	فیلتر قابل bookmark
<code>useEffect</code> + <code>useRef</code>	فوکوس و انتخاب متن هنگام ویرایش
Context	تم
feature-based ساختار	پوشه <code>features/tasks</code> مستقل و قابل حمل

تمرین‌های تکمیلی

۱) جابه‌جایی ترتیب با drag & drop (کتابخانه `@dnd-kit/core`) و ۲. Optimistic UI جایگزینی API شبیه‌سازی‌شده با `json-server` واقعی. ۳) افزودن TanStack Query به‌جای `useRevalidator` و مقایسه تجربه. ۴) نوشتن تست برای `AddTaskForm` با Testing Library (فصل ۲۷).

19 کارایی، React Compiler و Concurrent Rendering

کارایی در React یعنی «رندر کمتر، رندر ارزان‌تر و رندر با اولویت درست». این فصل از ابزارهای کلاسیک memoization شروع می‌کند، نشان می‌دهد React Compiler چگونه آن‌ها را خودکار می‌کند و سپس وارد قابلیت‌های Concurrent (transition و deferred value) می‌شود که تجربه کاربری را واقعاً متحول می‌کنند.

اول اندازه بگیرید، بعد بهینه کنید

اکثر اپلیکیشن‌های React بدون هیچ بهینه‌سازی به‌اندازه کافی سریع‌اند. بهینه‌سازی زود هنگام کد را پیچیده می‌کند بدون این‌که تفاوت محسوسی ایجاد کند. ابزارها:

ابزار	چه چیزی نشان می‌دهد
React DevTools → Profiler	کدام کامپوننت، چند بار و چرا رندر شد (گزینه «Record why each component rendered»)
Chrome Performance + React Tracks (۱۹.۲)	رندهای Scheduler و Components در timeline کروم: اولویت هر کار، مدت رندر و commit
API <Profiler>	اندازه‌گیری برنامه‌نویسی‌شده: <Profiler id="Feed" onRender={log}>
Lighthouse / Web Vitals	LCP، INP، CLS – معیارهای واقعی کاربر

چرا کامپوننت رندر می‌شود؟

فقط سه دلیل: (۱) state خودش تغییر کرده، (۲) والدش رندر شده، (۳) Context مصرفی‌اش تغییر کرده. **تغییر props به‌تنهایی دلیل نیست** – props تغییر می‌کند چون والد رندر شده. رندر مجدد فرزندان به‌طور پیش‌فرض اتفاق می‌افتد، حتی اگر props یکسان باشد.

ابزار کلاسیک: memo ، useMemo ، useCallback

```
src/features/products/ProductGrid.tsx

import { memo, useMemo, useCallback, useState } from 'react';

// memo: تغییر نکرده، رندر نکن (با مقایسه سطحی)
const ProductCard = memo(function ProductCard({ product, onSelect }: CardProps) {
  return <div onClick={() => onSelect(product.id)}>{product.title}</div>;
});

export function ProductGrid({ products }: { products: Product[] }) {
  const [query, setQuery] = useState('');
  const [selected, setSelected] = useState<string | null>(null);

  // useMemo: محاسبه گران فقط وقتی ورودی‌هایش تغییر کرد
  const filtered = useMemo(
    () => products.filter((p) => p.title.includes(query)),
    [products, query],
  );

  // useCallback: فرزند بی‌اثر نشود memo هویت تابع ثابت بماند تا
  const handleSelect = useCallback((id: string) => setSelected(id), []);

  return (
    <>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      {filtered.map((p) => (
        <ProductCard key={p.id} product={p} onSelect={handleSelect} />
      ))}
    </>
  );
}
```

سه اشتباه رایج memoization

۱) روی کامپوننتی که همیشه prop جدید می‌گیرد (مثل `style={{}}` یا `onClick={() => ...}`) بدون `memo` (۱) `useCallback` – بی‌اثر است و فقط هزینه مقایسه اضافه می‌کند. ۲) `useMemo` برای محاسبات ارزان (چند عملیات ساده) – سر بارش بیشتر از فایده است. ۳) `useCallback` بدون این‌که گیرنده `memo` شده باشد – هیچ چیزی را ذخیره نمی‌کند.

React Compiler: خدا حافظی با memoization دستی

React Compiler (نسخه ۱.۰، اکتبر ۲۰۲۵) یک ابزار زمان build است که کد شما را تحلیل می‌کند و به‌طور خودکار memoization دقیق‌تر از آنچه انسان می‌نویسد اضافه می‌کند. کد بالا بدون `useCallback` / `useMemo` / `memo` همان کارایی را خواهد داشت:

آنچه کامپایلر تولید می‌کند (ساده‌شده)

```
function ProductGrid(t0) {
  const $ = _c(6); // cache slots
  // ...
  let filtered;
  if ($[0] !== products
    || $[1] !== query) {
    filtered = products.filter(...);
    $[0] = products; $[1] = query;
    $[2] = filtered;
  } else filtered = $[2];
  // هم‌کش می‌شوند JSX و handleSelect
}
```

آنچه می‌نویسید

```
function ProductGrid({ products }) {
  const [query, setQuery] = useState('');
  const filtered = products.filter(
    p => p.title.includes(query)
  );
  const handleSelect = (id) => ...;
  return filtered.map(p =>
    <ProductCard
      product={p}
      onSelect={handleSelect}
    />
  );
}
```

کامپایلر در سطح عبارت (نه فقط کامپوننت) کش می‌کند: حتی JSX فرزند اگر ورودی‌هایش تغییر نکرده باشد، دوباره ساخته نمی‌شود. نصب در فصل ۲ توضیح داده شد.

۱ قوانین بازی با Compiler

- ♦ کامپایلر فقط کدی را بهینه می‌کند که **Rules of React** را رعایت کند؛ کد متخلف را رد می‌کند (نه این‌که بشکند). ESLint می‌گوید کدام.
- ♦ `useMemo` / `memo` موجود را لازم نیست حذف کنید؛ کامپایلر با آن‌ها سازگار است. اما در کد جدید ننویسید.
- ♦ در DevTools کامپوننت‌های بهینه‌شده نشان «Memo» دارند.
- ♦ برای پذیرش تدریجی از گزینه `compilationMode: 'annotation'` و دایرکتیو `"use memo"` استفاده کنید.

Concurrent Rendering: اولویت‌بندی به‌روزرسانی‌ها

از React 18، رندر قابل‌قطع است. React می‌تواند رندر کم‌اهمیت را متوقف کند، به ورودی کاربر پاسخ دهد و بعد ادامه دهد. برای استفاده از این قابلیت باید بگویید کدام به‌روزرسانی‌ها «فوری» نیستند.

```

src/features/search/SearchPage.tsx

import { useState, useTransition } from 'react';

export function SearchPage({ allItems }: { allItems: Item[] }) {
  const [query, setQuery] = useState(''); // باید آتی باشد input: فوری
  const [results, setResults] = useState(allItems);
  const [isPending, startTransition] = useTransition();

  function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
    const q = e.target.value;
    setQuery(q); // اولویت بالا
    startTransition(() => {
      setResults(heavyFilter(allItems, q)); // اولویت پایین؛ قابل قطع
    });
  }

  return (
    <>
      <input value={query} onChange={handleChange} />
      <div style={{ opacity: isPending ? 0.6 : 1 }}>
        <HeavyList items={results} />
      </div>
    </>
  );
}

```

اگر کاربر سریع تایپ کند، React رندر لیست قدیمی را **رها می‌کند** و فقط آخرین را کامل می‌کند. تایپ در input هرگز گیر نمی‌کند.

useDeferredValue ♦

وقتی مقدار را کنترل نمی‌کنید (از props می‌آید) یا نمی‌خواهید دو state جدا نگه دارید:

```

function SearchResults({ query }: { query: string }) {
  const deferredQuery = useDeferredValue(query);
  const isStale = query !== deferredQuery;

  const results = useMemo(() => heavyFilter(items, deferredQuery), [deferredQuery]);
  return <HeavyList items={results} style={{ opacity: isStale ? 0.6 : 1 }} />;
}

```

React اول با مقدار قدیمی رندر می‌کند (سریع)، بعد در پس‌زمینه با مقدار جدید. React 19 آرگومان دوم `initialValue` را برای رندر اولیه اضافه کرد.

<code>useDeferredValue</code>		<code>useTransition</code>
یک مقدار	خود <code>setState</code>	کنترل روی
وقتی مقدار از <code>props</code> /کتابخانه می‌آید	وقتی <code>state</code> را خودتان <code>set</code> می‌کنید	کاربرد
با مقایسه <code>value !== deferred</code>	مستقیم ✓	<code>isPending</code>
همین‌طور	<code>fallback</code> را نشان نمی‌دهد؛ <code>UI</code> قدیمی می‌ماند	با <code>Suspense</code>

✦ Suspense و Transition

اگر `setState` داخل `startTransition` باعث `suspend` شود (مثلاً تغییر تب که داده جدید می‌خواهد)، React به جای نشان دادن `fallback`، **UI فعلی را نگه می‌دارد** تا داده جدید برسد. این دقیقاً رفتاری است که کاربران انتظار دارند: هیچ فلش اسکلتی هنگام تغییر تب.

| <Activity> – پنهان کردن بدون (React 19.2) unmount

```
src/app/Tabs.tsx

import { Activity } from 'react';

export function Tabs({ active }: { active: 'feed' | 'profile' }) {
  return (
    <>
      <Activity mode={active === 'feed' ? 'visible' : 'hidden'}>
        <Feed />
      </Activity>
      <Activity mode={active === 'profile' ? 'visible' : 'hidden'}>
        <Profile />
      </Activity>
    </>
  );
}
```

در حالت `hidden`: کامپوننت از DOM حذف نمی‌شود اما `display: none` می‌گیرد، `Effect`هایش `cleanup` می‌شوند، و `state` (اسکرول، مقدار `input`) حفظ می‌شود. به روزرسانی‌های آن با کمترین اولویت انجام می‌شود. کاربردها: تب‌هایی که کاربر بین‌شان رفت و برگشت می‌کند، پیش‌رندر صفحه بعدی، حفظ `state` فرم هنگام ناوبری.

لیست‌های بزرگ: Virtualization

برای لیست‌های چندهزارتایی، فقط آیتم‌های داخل viewport را رندر کنید:

```
src/components/Virtuallist.tsx

import { useVirtualizer } from '@tanstack/react-virtual';
import { useRef } from 'react';

export function Virtuallist({ items }: { items: string[] }) {
  const parentRef = useRef<HTMLDivElement>(null);
  const virtualizer = useVirtualizer({
    count: items.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 48,
  });

  return (
    <div ref={parentRef} style={{ height: 600, overflow: 'auto' }}>
      <div style={{ height: virtualizer.getTotalSize(), position: 'relative' }}>
        {virtualizer.getVirtualItems().map((v) => (
          <div
            key={v.key}
            style={{
              position: 'absolute', top: v.start, height: v.size, width: '100%',
            }}
            >
            {items[v.index]}
          </div>
        ))}
      </div>
    </div>
  );
}
```

چک‌لیست کارایی

- ☐ React Compiler فعال است و ESLint خطای Rules of React ندارد
- ☐ state تا حد امکان **پایین** (نزدیک مصرف‌کننده) نگه داشته شده
- ☐ Context‌های پرتغییر از کم‌تغییر جدا شده‌اند
- ☐ به‌روزرسانی‌های سنگین داخل `startTransition` هستند
- ☐ لیست‌های بزرگ virtualize شده‌اند
- ☐ صفحات با `lazy` code-split شده‌اند (فصل ۱۶)

- تصاویر lazy، با اندازه صریح و فرمت مدرن (WebP/AVIF)
- در Profiler هیچ کامپوننتی بی دلیل بیش از چند بار رندر نمی شود

“ سؤالات مصاحبه (با پاسخ)

Junior – memo چه کاری می کند؟ کامپوننت را طوری می پیچد که اگر props (با مقایسه سطحی) تغییر نکرده باشد، رندر مجدد ناشی از والد را رد کند. تغییر state داخلی یا Context را همچنان رندر می کند.

Mid – تفاوت useTransition و useDeferredValue چیست و کی هر کدام؟ هر دو به روزرسانی را «غیرفوری» می کنند. useTransition وقتی خودتان setState را صدا می زنید و به isPending نیاز دارید؛ useDeferredValue وقتی مقدار از بیرون می آید یا می خواهید یک نسخه «عقب افتاده» از مقدار داشته باشید.

Senior – React Compiler چگونه تصمیم می گیرد چه چیزی را کش کند و چرا به Rules of React وابسته است؟ کامپایلر کد را به یک نمایش میانی (HIR) تبدیل می کند، وابستگی هر عبارت به props/state/متغیرها را تحلیل می کند و برای هر «واحد reactive» یک اسلات کش می سازد که فقط با تغییر ورودی های آن بازمحاسبه می شود. این تحلیل فرض می کند مقادیر **ناتغییرپذیر**ند و رندر **خالص** است؛ اگر کدی props را mutate کند یا نتیجه به چیزی خارج از ورودی های قابل ردیابی وابسته باشد، کش نادرست می شود – بنابراین کامپایلر چنین توابعی را تشخیص داده و از بهینه سازی صرف نظر می کند.

20 مدیریت state در مقیاس: Zustand، Redux Toolkit و Jotai

«کدام کتابخانه state؟» یکی از پرتکرارترین سؤالات React است و پاسخ درست با «چه نوع state ای؟» شروع می‌شود. این فصل ابتدا state را دسته‌بندی می‌کند، سپس سه ابزار محبوب را با کد واقعی مقایسه می‌کند تا انتخاب آگاهانه داشته باشید.

اول: state خود را دسته‌بندی کنید

نوع state	مثال	ابزار مناسب
محلی (Local)	باز/بسته بودن منو، مقدار input	useState ، useReducer
مشترک کم‌تغییر	تم، زبان، کاربر لاگین‌شده	Context
سراسری پرتغییر	سبد خرید، پخش‌کننده موسیقی، ویرایشگر	Zustand / Redux Toolkit / Jotai
سرور (Server cache)	لیست محصولات، پروفایل	TanStack Query / SWR
URL	فیلتر، صفحه، تب فعال	Search params روتر
فرم	مقادیر و خطاهای فیلدها	React Hook Form / useActionState

× بزرگ‌ترین اشتباه: داده سرور در store سراسری

قراردادن پاسخ API در Redux/Zustand یعنی خودتان باید کش، loading، refetch، invalidation و race condition را مدیریت کنید. این دقیقاً کاری است که TanStack Query انجام می‌دهد. با جداکردن server state، اغلب می‌بینید state سراسری باقی‌مانده آن‌قدر کوچک است که Context کافی است.

Zustand – ساده، سریع، بدون boilerplate

Terminal

```
npm install zustand
```

```

import { create } from 'zustand';
import { persist } from 'zustand/middleware';

type CartItem = { id: string; title: string; price: number; qty: number };

type CartStore = {
  items: CartItem[];
  add: (item: Omit<CartItem, 'qty'>) => void;
  remove: (id: string) => void;
  clear: () => void;
  total: () => number;
};

export const useCartStore = create<CartStore>()(
  persist(
    (set, get) => ({
      items: [],
      add: (item) =>
        set((s) => {
          const exists = s.items.find((i) => i.id === item.id);
          return {
            items: exists
              ? s.items.map((i) => (i.id === item.id ? { ...i, qty: i.qty + 1 } : i))
              : [...s.items, { ...item, qty: 1 }],
          };
        }),
      remove: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
      clear: () => set({ items: [] }),
      total: () => get().items.reduce((sum, i) => sum + i.price * i.qty, 0),
    }),
    { name: 'cart' }, // localStorage key
  ),
);

```


src/features/cart/CartBadge.tsx

```
// selector: این کامپوننت فقط با تغییر تعداد رندر می‌شود، نه با هر تغییر
export function CartBadge() {
  const count = useCartStore((s) => s.items.length);
  return <span className="badge">{count}</span>;
}

export function AddButton({ product }: { product: Product }) {
  const add = useCartStore((s) => s.add); // تابع پایدار؛ هرگز رندر اضافه نمی‌دهد
  return <button onClick={() => add(product)}>افزودن</button>;
}
```

مزایا: بدون selector، Provider داخلی، ۱ کیلوبایت، middleware برای persist/devtools/immer، و قابل استفاده خارج از کامپوننت (`useCartStore.getState()`).

✦ قاعده selector

همیشه با selector کوچک‌ترین بخش لازم را انتخاب کنید. `useCartStore()` بدون selector یعنی با هر تغییر store رندر می‌شود. برای انتخاب چند فیلد از `useShallow` استفاده کنید: `useCartStore(useShallow((s) => ({ a: s.a, b: s.b })))`.

Redux Toolkit – استاندارد سازمانی

Redux کلاسیک به دلیل boilerplate بدنام شد، اما **Redux Toolkit (RTK)** آن را مدرن کرد و همچنین در تیم‌های بزرگ به دلیل ساختار سخت‌گیرانه، DevTools بی‌نظیر و RTK Query محبوب است:

```
src/store/cartSlice.ts

import { createSlice, type PayloadAction } from '@reduxjs/toolkit';

type CartState = { items: CartItem[] };
const initialState: CartState = { items: [] };

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    // Immer بنویسید، خروجی «mutation» داخلی
    added(state, action: PayloadAction<Omit<CartItem, 'qty'>>) {
      const existing = state.items.find((i) => i.id === action.payload.id);
      if (existing) existing.qty += 1;
      else state.items.push({ ...action.payload, qty: 1 });
    },
    removed(state, action: PayloadAction<string>) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
  },
  selectors: {
    selectCount: (s) => s.items.length,
    selectTotal: (s) => s.items.reduce((sum, i) => sum + i.price * i.qty, 0),
  },
});

export const { added, removed } = cartSlice.actions;
export const { selectCount, selectTotal } = cartSlice.selectors;
```

src/store/index.ts

```
import { configureStore } from '@reduxjs/toolkit';
import { cartSlice } from '../cartSlice';

export const store = configureStore({ reducer: { cart: cartSlice.reducer } });
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

// هوک‌های تایپ‌شده
import { useDispatch, useSelector } from 'react-redux';
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

```
const count = useAppSelector(selectCount);
const dispatch = useAppDispatch();
dispatch(added(product));
```

state – Jotai (پایین به بالا)

به جای یک store بزرگ، «اتم‌های» کوچک و ترکیب‌پذیر:

src/atoms/cart.ts

```
import { atom } from 'jotai';
import { atomWithStorage } from 'jotai/utils';

export const cartItemsAtom = atomWithStorage<CartItem[]>('cart', []);

// به روز می‌شود cartItemsAtom اتم مشتق‌شده: خودکار با تغییر
export const cartTotalAtom = atom((get) =>
  get(cartItemsAtom).reduce((sum, i) => sum + i.price * i.qty, 0),
);

// اتم فقط-نوشتنی (action)
export const addToCartAtom = atom(null, (get, set, item: Omit<CartItem, 'qty'>) => {
  const items = get(cartItemsAtom);
  const exists = items.find((i) => i.id === item.id);
  set(
    cartItemsAtom,
    exists
      ? items.map((i) => (i.id === item.id ? { ...i, qty: i.qty + 1 } : i))
      : [...items, { ...item, qty: 1 }],
  );
});
```

```
import { useAtomValue, useSetAtom } from 'jotai';
const total = useAtomValue(cartTotalAtom);
const addToCart = useSetAtom(addToCartAtom);
```

Jotai برای state های پراکنده و وابسته به هم (ویرایشگرها، فرم های پیچیده، داشبوردهای تعاملی) عالی است و با Suspense (اتم async) یکپارچه می شود.

مقایسه نهایی

Context	Jotai	Redux Toolkit	Zustand	
حجم	۳KB~	۱۱KB~	۱KB~	◦
Boilerplate	کم	متوسط	خیلی کم	کم
Selector / رندر جزئی	✓ (per-atom)	✓	✓	✗
DevTools	✓	✓✓ بهترین	✓ (middleware)	React DevTools
Server state داخلی	✗	RTK Query	✗ TanStack (Query)	✗
منحنی یادگیری	کم متوسط	متوسط	کم	صفر
مناسب	state اتمی پیچیده	تیم بزرگ، قوانین سخت	اکثر SPA ها	داده کم تغییر

i همه بر پایه `useSyncExternalStore`

Jotai و Zustand، همگی برای اتصال به React از `useSyncExternalStore` استفاده می کنند؛ یعنی با Concurrent Rendering سازگارند و tearing ندارند. اگر روزی store خودتان را می سازید، همین هوک را استفاده کنید (فصل ۱۴).

State architecture

URL (search params) → فیلتر، صفحه، تب
TanStack Query → همه داده‌های سرور
Zustand (۱ تا ۳ store) → پرتغییر UI سبد، پخش‌کننده، تنظیمات
Context → auth session، تم، زبان
useState / useReducer → همه چیز دیگر (اکثریت!)

“سؤالات مصاحبه (با پاسخ)”

Junior – چرا Context برای state پرتغییر مناسب نیست؟ چون selector ندارد؛ هر تغییر value همه مصرف‌کننده‌ها را رندر می‌کند حتی اگر فقط به بخش کوچکی نیاز داشته باشند.

Mid – Zustand بدون Provider چطور کار می‌کند و چه عیبی دارد؟ store یک singleton در سطح ماژول است و هوک با `useSyncExternalStore` به آن subscribe می‌شود. عیب: در SSR، store بین درخواست‌ها مشترک می‌شود و باید per-request با Context ساخته شود (الگوی رسمی Zustand برای Next.js).

Senior – Redux Toolkit چه زمانی هنوز انتخاب درست است؟ وقتی تیم بزرگ به قرارداد سخت‌گیرانه (action/reducer/selector)، ردیابی کامل تغییرات (time-travel debugging)، middleware پیچیده (saga/listener) برای جریان‌های کاری چندمرحله‌ای یا RTK Query به‌عنوان راه‌حل یکپارچه server state نیاز دارد. برای اپ کوچک/متوسط، سربار ذهنی آن توجیه ندارد.

21 معماری پروژه و Clean Code در React

اپلیکیشنی که در هفته اول خوب کار می‌کند، در ماه ششم ممکن است غیرقابل نگهداری شود – نه به‌خاطر React، بلکه به‌خاطر نبود معماری. این فصل ساختار پوشه‌ها، لایه‌بندی و اصولی را آموزش می‌دهد که پروژه را در مقیاس زنده نگه می‌دارد.

مشکل ساختار «بر اساس نوع»

```
src/
├── components/      # فایل ۱۲۰!
├── hooks/           # فایل ۴۰
├── utils/
├── contexts/
└── pages/
```

با رشد پروژه، یافتن «همه‌چیز مربوط به سبد خرید» یعنی جستجو در پنج پوشه. تغییر یک فیچر، فایل‌های پراکنده را لمس می‌کند و حذف فیچر تقریباً غیرممکن است.

ساختار feature-based (توصیه‌شده)

```
src/
├── app/              # راه‌اندازی: router.tsx, providers.tsx, layouts/
├── features/         # هر فیچر: خودکفا و قابل حذف
│   ├── auth/
│   │   ├── api/      # fetch و query options توابع
│   │   ├── components/ # LoginForm, UserMenu
│   │   ├── hooks/     # useSession
│   │   ├── stores/    # (در صورت نیاز)
│   │   ├── types.ts
│   │   └── index.ts   # عمومی فیچر API
│   ├── cart/
│   └── products/
├── components/      # عمومی و بدون منطق دامنه UI
│   ├── ui/          # Button, Input, Dialog (shadcn)
│   └── layout/        # Container, PageHeader
├── hooks/            # هوک‌های عمومی (useMediaQuery, useDebounce)
├── lib/              # ابزارهای زیرساختی: apiClient, cn, formatters
├── config/           # ثابت‌ها، env
└── types/            # تایپ‌های سراسری
```

قاعده: کد داخل هر feature فقط از خودش، از `lib`، `hooks`، `components/ui` و `import types` می‌کند – نه از فیچر دیگر به صورت مستقیم. ارتباط بین فیچرها از طریق `index.ts` (API عمومی) یا در لایه `app` انجام می‌شود.

```
src/features/cart/index.ts

// فقط چیزهایی که بیرون فیچر لازم است
export { CartBadge } from './components/CartBadge';
export { useCartStore } from './stores/cartStore';
export type { CartItem } from './types';
```

✦ اعمال مرزها با ESLint

با `eslint-plugin-boundaries` یا قانون `no-restricted-imports` جلوی `import` مستقیم `features/` را بگیرید. معماری که ابزار آن را اعمال نکند، در اولین `deadline` فراموش می‌شود.

لایه‌بندی درون یک فیچر



```
src/features/products/api/products.api.ts

import { apiClient } from '@lib/apiClient';
import { queryOptions } from '@tanstack/react-query';
import { productSchema, type Product } from '../types';

async function fetchProducts(category: string): Promise<Product[]> {
  const data = await apiClient.get(`/products?category=${category}`);
  return productSchema.array().parse(data); // اعتبارسنجی مرز سیستم
}

// query options قابل استفاده در کامپوننت و loader
export const productsQuery = (category: string) =>
  queryOptions({
    queryKey: ['products', category],
    queryFn: () => fetchProducts(category),
  });
```

src/features/products/components/ProductList.tsx

```
export function ProductList({ category }: { category: string }) {
  const { data } = useSuspenseQuery(productsQuery(category));
  return <ProductGrid products={data} />; // ProductGrid: کاملاً نمایشی
}
```

کلاينت API متمرکز

src/lib/apiClient.ts

```
import { API_URL } from '@config/env';

class ApiError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

async function request<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${API_URL}${path}`, {
    ...init,
    headers: { 'Content-Type': 'application/json', ...init?.headers },
    credentials: 'include',
  });
  if (!res.ok) throw new ApiError(res.status, await res.text());
  return res.status === 204 ? (undefined as T) : res.json();
}

export const apiClient = {
  get: <T>(p: string) => request<T>(p),
  post: <T>(p: string, body: unknown) =>
    request<T>(p, { method: 'POST', body: JSON.stringify(body) }),
  put: <T>(p: string, body: unknown) =>
    request<T>(p, { method: 'PUT', body: JSON.stringify(body) }),
  delete: <T>(p: string) => request<T>(p, { method: 'DELETE' }),
};
```

یک نقطه برای auth header، مدیریت خطا، لاگ و retry. هرگز fetch خام در کامپوننت ننویسید.

اصل	در عمل
کامپوننت کوچک	بیش از ۱۵۰ خط یا بیش از ۳ مسئولیت → بشکنید
جداسازی منطق از نما	منطق در هوک سفارشی، نما در کامپوننت
نام‌گذاری گویا	UserAvatarWithStatus بهتر از Avatar2
کم Props	بیش از ۷ prop نشانه کامپوننت چندمنظوره است → ترکیب
Early return	حالت‌های خالی/خطا/loading اول، مسیر اصلی آخر
بدون magic number/string	const MAX_UPLOAD_MB = 5 در config/
Colocation	تست، استایل و تایپ کنار فایل استفاده‌کننده
حذف کد مرده	کامنت‌شده = حذف‌شده؛ git تاریخچه دارد

♦ مثال ری‌فکتور: از کامپوننت شلوغ به لایه‌بندی

❌ قبل

```
function UserPage({ id }) {
  const [user, setUser] = useState();
  const [loading, setLoading] =
    useState(true);
  const [err, setErr] = useState();
  useEffect(() => {
    fetch('/api/users/' + id)
      .then(r => r.json())
      .then(setUser)
      .catch(setErr)
      .finally(() => setLoading(false));
  }, [id]);
  if (loading) return <Spinner/>;
  if (err) return <p>خطا</p>;
  return <div>{/* خط ۸۰ JSX */</div>;
}
```

✅ بعد

```
// api/users.api.ts
export const userQuery = (id) =>
  queryOptions({
    queryKey: ['user', id],
    queryFn: () => fetchUser(id),
  });

// components/UserPage.tsx
function UserPage({ id }) {
  const { data: user } =
    useSuspenseQuery(userQuery(id));
  return <UserProfile user={user} />;
}

// ErrorBoundary + Suspense در layout
```

قراردادهای نام‌گذاری فایل

نوع	قرارداد	مثال
کامپوننت	PascalCase	ProductCard.tsx
هوک	use با camelCase	useCart.ts
ابزار / سرویس	dot-suffix یا camelCase	products.api.ts ، formatPrice.ts
تایپ	types.ts در هر فیچر	—
تست	کنار فایل با .test	ProductCard.test.tsx
ثابت‌ها	UPPER_SNAKE در فایل	MAX_ITEMS

ابزارهای کیفیت کد

```
package.json (scripts)

{
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint . --max-warnings 0",
    "format": "prettier --write .",
    "typecheck": "tsc --noEmit",
    "test": "vitest",
    "prepare": "husky"
  }
}
```

با **husky + lint-staged** قبل از هر lint و commit و format روی فایل‌های تغییریافته اجرا شود؛ کد بد اصلاً وارد مخزن نمی‌شود.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا ساختار feature-based بهتر از type-based است؟ چون کدی که با هم تغییر می‌کند کنار هم است (cohesion بالا)، حذف یا استخراج فیچر ساده است، و onboarding سریع‌تر می‌شود چون هر فیچر یک نقشه کوچک دارد.

Mid – «Presentational vs Container» هنوز معتبر است؟ ایده اصلی (جدایی منطق از نما) معتبر است اما پیاده‌سازی تغییر کرده: به جای کامپوننت Container، **هوک سفارشی** منطق را نگه می‌دارد و کامپوننت نما آن را مصرف می‌کند. این الگو تست‌پذیری بهتری دارد.

Senior – چطور معماری را در برابر فرسایش (architecture erosion) محافظت می‌کنید؟ با تبدیل قواعد به ابزار: ESLint برای مرزهای TypeScript strict، import، برای قراردادها، `index.ts` به عنوان API عمومی، تست‌های معماری (مثل dependency-cruiser)، بازبینی کد با چک‌لیست، و ADR (Architecture Decision Records) برای ثبت «چرا»ها تا تصمیم‌ها با رفتن افراد گم نشوند.

22 الگوهای پیشرفته کامپوننت

کتابخانه‌های UI حرفه‌ای (Radix، Headless UI، shadcn) روی چند الگوی مشخص ساخته شده‌اند. با یادگیری این الگوها می‌توانید کامپوننت‌هایی بسازید که هم انعطاف‌پذیرند و هم API ساده‌ای دارند.

Compound Components – کامپوننت‌های مرکب

به‌جای یک کامپوننت با ۲۰ prop، خانواده‌ای از کامپوننت‌ها که state را از طریق Context به اشتراک می‌گذارند. مثل `<select>` و `<option>` در HTML:

src/components/ui/Tabs.tsx (1/2)

```
import { createContext, use, useId, useState, type ReactNode } from 'react';

type TabsCtx = { active: string; setActive: (v: string) => void; baseId: string };
const TabsContext = createContext<TabsCtx | null>(null);
const useTabs = () => {
  const ctx = use(TabsContext);
  if (!ctx) throw new Error('<Tabs> باید داخل Tabs.* باشد');
  return ctx;
};

type TabsProps = { defaultValue: string; children: ReactNode };

export function Tabs({ defaultValue, children }: TabsProps) {
  const [active, setActive] = useState(defaultValue);
  const baseId = useId();
  return <TabsContext value={{ active, setActive, baseId }}>{children}</TabsContext>;
}
```

اجزای فرزند از طریق `useTabs()` به state مشترک می‌رسند و با `useId` شناسه‌های ARIA سازگار می‌سازند؛ در پایان به صورت property روی `Tabs` سوار می‌شوند تا API `<Tabs.Trigger>` شکل بگیرد:

```
src/components/ui/Tabs.tsx (2/2)

function List({ children }: { children: ReactNode }) {
  return <div role="tablist" className="flex gap-1 border-b">{children}</div>;
}

function Trigger({ value, children }: { value: string; children: ReactNode }) {
  const { active, setActive, baseId } = useTabs();
  const selected = active === value;
  return (
    <button
      role="tab"
      id={` ${baseId}-tab-${value}`}
      aria-selected={selected}
      aria-controls={` ${baseId}-panel-${value}`}
      tabIndex={selected ? 0 : -1}
      onClick={() => setActive(value)}
      className={selected ? 'border-b-2 border-brand-500 font-bold' : ''}
    >
      {children}
    </button>
  );
}

function Panel({ value, children }: { value: string; children: ReactNode }) {
  const { active, baseId } = useTabs();
  if (active !== value) return null;
  return (
    <div
      role="tabpanel"
      id={` ${baseId}-panel-${value}`}
      aria-labelledby={` ${baseId}-tab-${value}`}
    >
      {children}
    </div>
  );
}

Tabs.List = List;
Tabs.Trigger = Trigger;
Tabs.Panel = Panel;
```

```

<Tabs defaultValue="specs">
  <Tabs.List>
    <Tabs.Trigger value="specs">مشخصات</Tabs.Trigger>
    <Tabs.Trigger value="reviews">نظرات</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Panel value="specs"><SpecsTable /></Tabs.Panel>
  <Tabs.Panel value="reviews"><Reviews /></Tabs.Panel>
</Tabs>

```

مصرف‌کننده آزادانه ساختار را می‌چیند (مثلاً یک آیکون کنار یک Trigger) بدون این‌که Tabs prop جدیدی بخواهد. `useId` شناسه‌های یکتا برای ARIA می‌سازد — حتی در SSR بدون mismatch.

Controlled / Uncontrolled – پشتیبانی هم‌زمان

کامپوننت‌های حرفه‌ای هر دو حالت را می‌پذیرند: اگر `value` داده شد، کنترل‌شده؛ در غیر این صورت state داخلی:

```

src/hooks/useControllableState.ts

import { useState } from 'react';

export function useControllableState<T>({
  value,
  defaultValue,
  onChange,
}: {
  value?: T;
  defaultValue: T;
  onChange?: (v: T) => void;
}) {
  const [internal, setInternal] = useState(defaultValue);
  const isControlled = value !== undefined;
  const current = isControlled ? value : internal;

  function set(next: T) {
    if (!isControlled) setInternal(next);
    onChange?.(next);
  }

  return [current, set] as const;
}

```

```
export function Tabs(props: TabsProps) {
  const { value, defaultValue = '', onChange, children } = props;
  const [active, setActive] = useControllableState({
    value, defaultValue, onChange: onChange,
  });
  // ...
}

<Tabs defaultValue="a" /> // uncontrolled
<Tabs value={tab} onChange={setTab} /> // controlled (مثلاً همگام با URL)
```

⚠ تغییر حالت در طول عمر ممنوع

کامپوننت نباید بین controlled و uncontrolled جابه‌جا شود (value از undefined به مقدار یا برعکس). React برای `<input>` هم همین هشدار را می‌دهد. اگر می‌خواهید «خالی» را نشان دهید، `null` یا `'` بدهید نه `undefined`.

Portal – رندر خارج از سلسله‌مراتب DOM

Toast و Modal، Tooltip باید بالای همه چیز باشند؛ اما اگر داخل عنصری با `overflow: hidden` یا `transform` باشند، بریده می‌شوند. `createPortal` آن‌ها را در `document.body` رندر می‌کند در حالی که در درخت React همان‌جا (با Context و رویدادها) باقی می‌مانند.

✦ <dialog> بومی را جدی بگیرید

عنصر `<dialog>` با `showModal()` به‌طور بومی focus trap، بستن با Escape، `inert` کردن پس‌زمینه و لایه top-layer را دارد. در ۲۰۲۶ همه مرورگرهای اصلی پشتیبانی می‌کنند؛ صدها خط کد دسترسی‌پذیری صرفه‌جویی می‌شود. حتی Portal هم برای top-layer لازم نیست، اما برای سازگاری با `overflow` والدین همچنان مفید است.

ترکیب هر دو – Portal برای جای‌گذاری و `<dialog>` برای رفتار – یک Modal کامل در کمتر از ۳۰ خط می‌دهد:

```
src/components/ui/Modal.tsx

import { createPortal } from 'react-dom';
import { useEffect, useRef, type ReactNode } from 'react';

type ModalProps = { open: boolean; onClose: () => void; children: ReactNode };

export function Modal({ open, onClose, children }: ModalProps) {
  const dialogRef = useRef<HTMLDialogElement>(null);

  useEffect(() => {
    const el = dialogRef.current;
    if (!el) return;
    if (open && !el.open) el.showModal(); // بومی: focus trap + Escape + backdrop
    if (!open && el.open) el.close();
  }, [open]);

  if (!open) return null;

  return createPortal(
    <dialog
      ref={dialogRef}
      onClose={onClose}
      className="rounded-xl p-6 backdrop:bg-black/50"
    >
      {children}
    </dialog>,
    document.body,
  );
}
```


as Polymorphic Component — prop

```
src/components/ui/Text.tsx

import type { ComponentPropsWithoutRef, ElementType } from 'react';

type TextProps<T extends ElementType> = {
  as?: T;
  size?: 'sm' | 'md' | 'lg';
} & Omit<ComponentPropsWithoutRef<T>, 'as' | 'size'>;

export function Text<T extends ElementType = 'p'>({
  as, size = 'md', className, ...props }: TextProps<T>,
) {
  const Component = as ?? 'p';
  return <Component className={`text-${size} ${className ?? ''}`} {...props} />;
}

<Text as="h1" size="lg">عنوان</Text>           // props مجاز h1
<Text as="a" href="/x">لینک</Text>             // href فقط وقتی as="a" ✓
```

TypeScript بر اساس `as`، `props` مجاز را استنتاج می‌کند. الگوی `asChild` (Radix) جایگزین مدرن‌تر است: به جای `as`، فرزند را با `Slot` کلون می‌کند و `props` را روی آن merge می‌کند.

Headless Component – منطق بدون UI

منطق کامل (state، کیبورد، ARIA) را ارائه کنید و رندر را کاملاً به مصرف‌کننده بسپارید:

```
src/hooks/useDisclosure.ts

import { useCallback, useId, useState } from 'react';

export function useDisclosure(initial = false) {
  const [isOpen, setOpen] = useState(initial);
  const id = useId();
  const open = useCallback(() => setOpen(true), []);
  const close = useCallback(() => setOpen(false), []);
  const toggle = useCallback(() => setOpen((v) => !v), []);

  return {
    isOpen, open, close, toggle,
    // «prop getters»: ویژگی‌های آماده برای پخش روی عناصر
    getTriggerProps: () => ({
      'aria-expanded': isOpen, 'aria-controls': id, onClick: toggle,
    }),
    getPanelProps: () => ({ id, hidden: !isOpen }),
  };
}
```

```
const d = useDisclosure();
<button {...d.getTriggerProps()}>جزئیات</button>
<section {...d.getPanelProps()}>...</section>
```

این دقیقاً فلسفه **Radix Primitives**، **React Aria** و **Headless UI** است: شما ۱۰۰٪ کنترل استایل دارید و آن‌ها دسترسی‌پذیری را تضمین می‌کنند.

```
src/components/ui/Slot.tsx

import { cloneElement, isValidElement } from 'react';
import type { ReactNode, HTMLAttributes } from 'react';

type SlotProps = HTMLAttributes<HTMLElement> & { children: ReactNode };

export function Slot({ children, ...props }: SlotProps) {
  if (!isValidElement<HTMLAttributes<HTMLElement>>(children)) return null;
  return cloneElement(children, {
    ...props,
    ...children.props,
    className: [props.className, children.props.className].filter(Boolean).join(' '),
  });
}

// Button با asChild
function Button({ asChild, ...props }: ButtonProps & { asChild?: boolean }) {
  const Comp = asChild ? Slot : 'button';
  return <Comp className={buttonStyles} {...props} />;
}

// استایل دکمه، رفتار لینک:
<Button asChild><Link to="/shop">فروشگاه</Link></Button>
```

جدول انتخاب الگو |

نیاز	الگو
چند بخش مرتبط با state مشترک (تب، آکاردئون، منو)	Compound Components
پشتیبانی همزمان از state داخلی و خارجی	Controlled/Uncontrolled
رندر بالای همه چیز (modal, toast)	<dialog> + Portal
یک ظاهر روی عناصر مختلف (دکمه‌ای که لینک است)	Polymorphic / asChild
اشتراک منطق پیچیده بدون تحمیل ال	prop getters با Headless hook
رندر سفارشی هر آیتم لیست	(renderItem) Render prop

“ سؤالات مصاحبه (با پاسخ)

Junior – Portal چه مشکلی را حل می‌کند؟ اجازه می‌دهد کامپوننت در جای دیگری از DOM (مثل `body`) رندر شود تا `overflow`، `z-index` و `transform` والدین آن را نبرند؛ در حالی که در درخت React همان‌جا می‌ماند و Context و رویدادها کار می‌کنند.

Mid – Compound Components چه مزیتی نسبت به **props** پیکربندی دارند؟ انعطاف ساختاری (مصرف‌کننده ترتیب، `wrapper` و محتوای هر بخش را کنترل می‌کند) بدون انفجار تعداد `props`، و API که شبیه HTML خوانده می‌شود. عیب: کمی پیچیدگی پیاده‌سازی و وابستگی به Context.

Senior – رویدادها در Portal چطور حباب می‌کنند و چه تله‌ای دارد؟ رویدادهای React از درخت **React** (نه DOM) بالا می‌روند؛ کلیک داخل `modal` که در `body` است، به `onClick` والدِ React ای آن می‌رسد. تله: اگر والد یک handler «کلیک بیرون» دارد که `contains` را روی DOM بررسی می‌کند، کلیک داخل `modal` را «بیرون» تشخیص می‌دهد. راه‌حل: بررسی با `e.composedPath()` یا استفاده از `onPointerDownOutside` کتابخانه‌های `headless`.

23 فرم‌ها و اعتبارسنجی پیشرفته

فصل ۹ مبانی Actions را آموخت. حالا سراغ فرم‌های واقعی می‌رویم: اعتبارسنجی با schema تایپ‌شده، نمایش خطای هر فیلد، فرم‌های بزرگ با React Hook Form و فرم‌های چندمرحله‌ای – با همان الگوهایی که در بک‌اند هم قابل‌استفاده‌اند.

۱ | Zod – یک schema، تایپ + اعتبارسنجی

Terminal

```
npm install zod
```

src/features/auth/schemas.ts

```
import { z } from 'zod';

export const signupSchema = z
  .object({
    name: z.string().trim().min(2, 'نام حداقل ۲ حرف'),
    email: z.email('ایمیل معتبر نیست'),
    password: z.string().min(8, 'حداقل ۸ کاراکتر').regex(/[0-9]/, 'حداقل یک عدد'),
    confirm: z.string(),
    phone: z.string().regex(/^09\d{9}$/, 'شماره موبایل معتبر نیست').optional(),
    terms: z.literal(true, { error: 'پذیرش قوانین الزامی است' }),
  })
  .refine((d) => d.password === d.confirm, {
    error: 'رمزها یکسان نیستند',
    path: ['confirm'],
  });

// استخراج می‌شود schema تایپ از
export type SignupInput = z.infer<typeof signupSchema>;
```

همین فایل می‌تواند در بک‌اند Node/Next هم import شود: یک منبع حقیقت برای اعتبارسنجی کلاینت و سرور.

src/lib/forms.ts

```
import { z } from 'zod';

export type FormState<T> = {
  values?: Partial<Record<keyof T, string>>;
  fieldErrors?: Partial<Record<keyof T, string[]>>;
  formError?: string;
  success?: boolean;
};

export function parseForm<S extends z.ZodType>(schema: S, formData: FormData) {
  const raw = Object.fromEntries(formData) as Record<string, string>;
  // چک‌باکس: 'on' → true
  const normalized = Object.fromEntries(
    Object.entries(raw).map(([k, v]) => [k, v === 'on' ? true : v]),
  );
  const result = schema.safeParse(normalized);
  return { raw, result };
}
```

src/features/auth/SignupForm.tsx (1/2)

```
import { useActionState } from 'react';
import { z } from 'zod';
import { signupSchema, type SignupInput } from './schemas';
import { parseForm, type FormState } from '@lib/forms';
import { Field } from '@components/ui/Field';

type SignupState = FormState<SignupInput>;

async function signupAction(_: SignupState, fd: FormData): Promise<SignupState> {
  const { raw, result } = parseForm(signupSchema, fd);
  if (!result.success) {
    // Zod 4: z.flattenError به جای result.error.flatten()
    return { values: raw, fieldErrors: z.flattenError(result.error).fieldErrors };
  }
  try {
    await api.signup(result.data);
    return { success: true };
  } catch (e) {
    return { values: raw, formError: (e as Error).message };
  }
}
```

Action همه‌ی منطق (اعتبارسنجی، درخواست، شکل خطا) را در خود دارد؛ کامپوننت فقط وضعیت بازگشتی را به فیلدها وصل می‌کند و در صورت خطا مقادیر قبلی را با `defaultValue` نگه می‌دارد:

```
src/features/auth/SignupForm.tsx (2/2)

export function SignupForm() {
  const [state, action, pending] = useActionState(signupAction, {});
  if (state.success) return <SuccessMessage />;

  return (
    <form action={action} noValidate className="space-y-3">
      <Field name="name" label="نام"
        defaultValue={state.values?.name} errors={state.fieldErrors?.name} />
      <Field name="email" label="ایمیل" type="email"
        defaultValue={state.values?.email} errors={state.fieldErrors?.email} />
      <Field name="password" label="رمز عبور" type="password"
        errors={state.fieldErrors?.password} />
      <Field name="confirm" label="تکرار رمز" type="password"
        errors={state.fieldErrors?.confirm} />
      <label className="flex items-center gap-2">
        <input type="checkbox" name="terms" /> می‌پذیرم را قوانین
      </label>
      {state.fieldErrors?.terms && (
        <p className="text-red-600">{state.fieldErrors.terms[0]}</p>
      )}
      {state.formError && (
        <p role="alert" className="text-red-600">{state.formError}</p>
      )}
      <button disabled={pending}>{pending ? '...' : 'ثبت‌نام'}</button>
    </form>
  );
}
```

```
import { useId, type ComponentProps } from 'react';

type FieldProps = ComponentProps<'input'> & { label: string; errors?: string[] };

export function Field({ label, errors, ...input }: FieldProps) {
  const id = useId();
  const errorId = `${id}-error`;
  const invalid = !!errors?.length;
  return (
    <div>
      <label htmlFor={id} className="block text-sm font-medium">{label}</label>
      <input
        id={id}
        aria-invalid={invalid}
        aria-describedby={invalid ? errorId : undefined}
        className={`rounded-lg border px-3 py-2 ${invalid ? 'border-red-500' : ''}`}
        {...input}
      />
      {invalid && (
        <p id={errorId} className="mt-1 text-sm text-red-600">{errors![0]}</p>
      )}
    </div>
  );
}
```

`noValidate` اعتبارسنجی بومی مرورگر را خاموش می‌کند تا پیام‌های خودتان نمایش داده شوند؛ اما ویژگی‌های `type="email" / required` را برای دسترسی‌پذیری نگه دارید.

◆ Progressive Enhancement در عمل

همین فرم در Next.js/React Router با Server Function بدون تغییر کار می‌کند و حتی بدون JS هم submit می‌شود. الگوی `values + fieldErrors` استاندارد این اکوسیستم است.

React Hook Form – برای فرم‌های بزرگ و تعاملی

وقتی فرم ۲۰ فیلد، اعتبارسنجی لحظه‌ای، فیلدهای وابسته و آرایه‌های پویا دارد، **React Hook Form** کارایی بالایی دارد (فیلدها `uncontrolled` و تایپ‌کردن رندر مجدد کل فرم را نمی‌سازد):

```
npm install react-hook-form @hookform/resolvers
```



```
import { useForm, useFieldArray } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const itemSchema = z.object({
  sku: z.string().min(1),
  qty: z.coerce.number().int().min(1),
});
const orderSchema = z.object({
  customer: z.string().min(2),
  items: z.array(itemSchema).min(1, 'حداقل یک قلم'),
});
type OrderInput = z.infer<typeof orderSchema>;

export function OrderForm() {
  const { register, control, handleSubmit, formState } = useForm<OrderInput>({
    resolver: zodResolver(orderSchema),
    defaultValues: { customer: '', items: [{ sku: '', qty: 1 }] },
    mode: 'onBlur',
  });
  const { errors, isSubmitting } = formState;
  const { fields, append, remove } = useFieldArray({ control, name: 'items' });

  const onSubmit = async (data: OrderInput) => { await api.createOrder(data); };
}
```

با `useFieldArray` آرایه‌ی اقلام سفارش مدیریت می‌شود؛ هر ردیف با `f.id` (نه `index`) کلید می‌خورد تا حذف و افزودن، `state` فیلدهای دیگر را به هم نریزد:

```
src/features/orders/OrderForm.tsx (2/2)

return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-3">
    <input {...register('customer')} placeholder="نام مشتری" />
    {errors.customer && <p>{errors.customer.message}</p>}

    {fields.map((f, i) => (
      <div key={f.id} className="flex gap-2">
        <input {...register(`items.${i}.sku`)} placeholder="کد کالا" />
        <input type="number" {...register(`items.${i}.qty`)} />
        <button type="button" onClick={() => remove(i)}>حذف</button>
      </div>
    ))}
    {errors.items?.root && <p>{errors.items.root.message}</p>}

    <button type="button" onClick={() => append({ sku: '', qty: 1 })}>
      + جدید قلم
    </button>
    <button disabled={isSubmitting}>سفارش ثبت</button>
  </form>
);
}
```

useActionState یا React Hook Form؟

معیار	Actions + Zod	React Hook Form
فرم ساده تا متوسط	✅ ایده‌آل	بیش از حد
اعتبارسنجی لحظه‌ای (onChange)	دستی	✅ داخلی
آرایه‌های پویا و فیلدهای وابسته	سخت	✅ watch, useFieldArray
Progressive Enhancement	✅	❌
بدون وابستگی	✅	+ کتابخانه
هم‌راستا با Server Functions	✅	نیاز به پل

قاعده سرانگشتی: **پیش‌فرض Actions**؛ اگر فرم واقعاً پیچیده شد، RHF.

فرم چندمرحله‌ای (Wizard)

فرم چندمرحله‌ای همان فرم است، فقط state آن «کدام مرحله + داده‌ی جمع‌شده تا این‌جا» است. این state را با `useReducer` (فصل ۱۳) مدل کنید تا قوانین عبور بین مراحل یک‌جا و قابل‌تست باشند:

```
src/features/onboarding/Wizard.tsx (1/2)

import { useReducer } from 'react';

type Data = { name?: string; email?: string; plan?: 'free' | 'pro' };
type State = { step: 0 | 1 | 2; data: Data };
type Action = { type: 'next'; patch: Data } | { type: 'back' };

function reducer(s: State, a: Action): State {
  switch (a.type) {
    case 'next':
      return {
        step: Math.min(s.step + 1, 2) as State['step'],
        data: { ...s.data, ...a.patch },
      };
    case 'back':
      return { ...s, step: Math.max(s.step - 1, 0) as State['step'] };
  }
}
```

reducer تنها جایی است که قانون «مرحله بعد/قبل» و ادغام داده‌ها نوشته می‌شود؛ کامپوننت Wizard فقط مرحله‌ی فعال را رندر می‌کند و هر گام، داده‌ی خود را با `onNext` بالا می‌فرستد:

```
src/features/onboarding/Wizard.tsx (2/2)

const LABELS = ['پروفایل', 'پلن', 'بازبینی'];

export function Wizard() {
  const [state, dispatch] = useReducer(reducer, { step: 0, data: {} });
  const next = (patch: Data) => dispatch({ type: 'next', patch });
  const back = () => dispatch({ type: 'back' });
  const steps = [
    <StepProfile key="p" data={state.data} onNext={next} />,
    <StepPlan key="l" data={state.data} onNext={next} onBack={back} />,
    <StepReview key="r" data={state.data} onBack={back} />,
  ];
  return (
    <>
      <ol className="flex gap-2">{LABELS.map((label, i) => (
        <li key={label} aria-current={i === state.step ? 'step' : undefined}>
          {label}
        </li>
      ))}</ol>
      {steps[state.step]}
    </>
  );
}
```

هر مرحله یک فرم مستقل با schema خودش (`z.object({...}).pick(...)`) است و فقط داده معتبر را به بالا می‌فرستد. برای حفظ پیشرفت هنگام رفرش، `data` را در `sessionStorage` یا URL نگه دارید.

نکات UX فرم

- ♦ خطا را بعد از `blur` یا `submit` نشان دهید، نه هنگام اولین کاراکتر.
- ♦ فوکوس روی اولین فیلد خطا دار پس از `submit` ناموفق.
- ♦ `autoComplete` درست: `email` ، `new-password` ، `current-password` ، `tel` ، `one-time-code`.
- ♦ `inputMode="numeric"` برای اعداد در موبایل به جای `type="number"` (که اسکرول را می‌شکند).
- ♦ اعداد فارسی را در سرور/قبل از اعتبارسنجی به انگلیسی تبدیل کنید: `s.replace(/[۰-۹]/g, d => '۰۱۲۳۴۵۶۷۸۹'.indexOf(d))`.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا اعتبارسنجی فقط سمت کلاینت کافی نیست؟ کلاینت قابل دور زدن است (DevTools، curl). کلاینت برای UX است؛ سرور برای امنیت. Zod اجازه می‌دهد یک schema را در هر دو جا استفاده کنید.

Mid – تفاوت `z.coerce.number()` و `z.number()` در فرم‌ها چیست؟ `FormData` همه‌چیز را رشته می‌دهد؛ `z.number()` روی "5" خطا می‌دهد. `z.coerce.number()` ابتدا `Number()` را اعمال می‌کند. برای چک‌باکس هم باید 'on' را به boolean تبدیل کنید.

Senior – چرا React Hook Form از uncontrolled inputs استفاده می‌کند و این با Concurrent React چه نسبتی دارد؟ با ثبت `ref` و خواندن مقادیر مستقیماً از DOM، تایپ در یک فیلد باعث رندر مجدد کل فرم نمی‌شود – برای فرم‌های ۵۰ فیلدی تفاوت محسوس است. چون RHF state خود را خارج از React نگه می‌دارد، از `useSyncExternalStore` -مانند برای `formState / watch` استفاده می‌کند تا با Concurrent Rendering سازگار بماند. هزینه: با `<form action>` و Progressive Enhancement مستقیماً یکپارچه نیست و نیاز به adapter دارد.

24 مدیریت خطا – مرجع کامل

خطا اجتناب‌ناپذیر است؛ سؤال این است که کاربر صفحه سفید می‌بیند یا پیام مفید با دکمه «تلاش مجدد». این فصل یک استراتژی چندلایه برای مدیریت خطاهای رندر، `async` و رویداد ارائه می‌دهد و API‌های جدید React 19 برای گزارش خطا را معرفی می‌کند.

انواع خطا در اپلیکیشن React

نوع	مثال	چه کسی می‌گیرد؟
خطای رندر	<code>user.name</code> وقتی <code>user</code> <code>null</code> است	Error Boundary
خطای <code>async</code> در <code>Action/Suspense</code>	<code>reject</code> شدن Promise در <code>use()</code> یا <code><form action></code>	Error Boundary
خطای <code>event handler</code>	<code>throw</code> داخل <code>onClick</code>	✗ Error Boundary نمی‌گیرد؛ <code>try/catch</code> خودتان
خطای Effect	<code>throw</code> داخل <code>useEffect</code>	Error Boundary
خطای شبکه	500 از API	کتابخانه داده / Boundary
Promise رهاشده	<code>fetch().then()</code> بدون handler در <code>catch</code>	<code>window.onunhandledrejection</code>

Error Boundary

Error Boundary کامپوننتی است که خطای زیردرخت خود را می‌گیرد و UI جایگزین نشان می‌دهد. React هنوز نسخه هوکی ندارد؛ از `react-error-boundary` استفاده کنید:

```
Terminal
npm install react-error-boundary
```

src/components/ErrorFallback.tsx

```
import type { FallbackProps } from 'react-error-boundary';

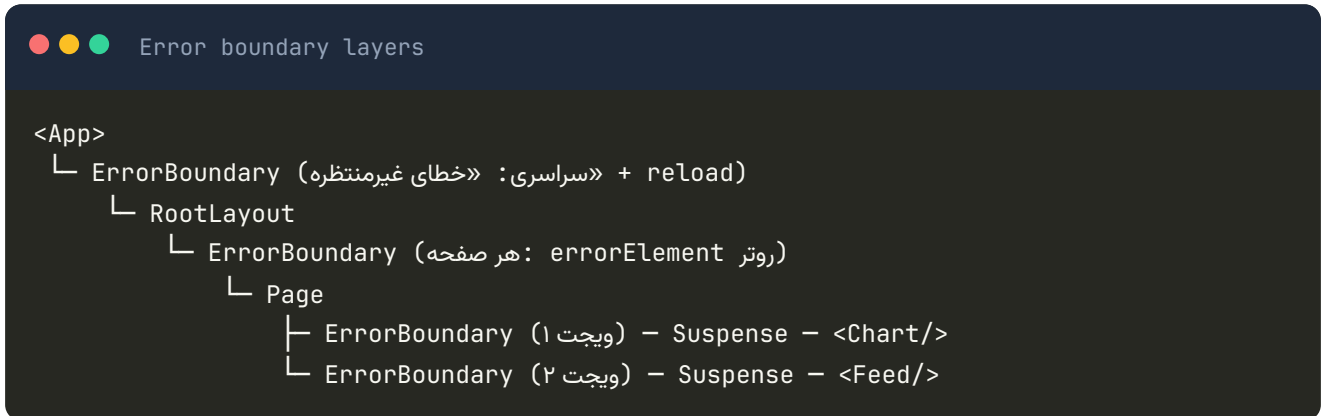
export function ErrorFallback({ error, resetErrorBoundary }: FallbackProps) {
  const message = error instanceof Error ? error.message : 'خطای ناشناخته';
  return (
    <div
      role="alert"
      className="rounded-xl border border-red-200 bg-red-50 p-6 text-center"
    >
      <h2 className="text-lg font-bold text-red-700">آمده پیش مشکلی</h2>
      <p className="mt-2 text-sm text-red-600">{message}</p>
      <button
        onClick={resetErrorBoundary}
        className="mt-4 rounded-lg bg-red-600 px-4 py-2 text-white"
      >
        مجدد تلاش
      </button>
    </div>
  );
}
```

src/features/dashboard/Dashboard.tsx

```
import { ErrorBoundary } from 'react-error-boundary';
import { QueryErrorResetBoundary } from '@tanstack/react-query';

export function Dashboard() {
  return (
    <QueryErrorResetBoundary>
      {({ reset }) => (
        <ErrorBoundary
          FallbackComponent={ErrorFallback}
          onReset={reset} // کندمی پاک را TanStack Query خطا دار کش
          onError={(err, info) => logError(err, info.componentStack)}
          resetKeys={[userId]} // شود ریست خودکار کاربر تغییر با
        >
          <Suspense fallback={<Skeleton />}>
            <Stats />
          </Suspense>
        </ErrorBoundary>
      )}
    </QueryErrorResetBoundary>
  );
}
```

استراتژی چندلایه (Granular Boundaries)



قاعده: هر واحد مستقل UI که می‌تواند جداگانه شکست بخورد، مرز خودش را داشته باشد. شکست نمودار فروش نباید فید اخبار را از بین ببرد. مرز سراسری فقط «آخرین خط دفاع» است.

خطا در event handler

```
async function handleDelete() {
  try {
    await api.deleteItem(id);
    toast.success('حذف شد');
  } catch (e) {
    const forbidden = e instanceof ApiError && e.status === 403;
    toast.error(forbidden ? 'دسترسی ندارید' : 'حذف ناموفق بود');
    logError(e);
  }
}
```

Error Boundary خطای handler را نمی‌گیرد چون خارج از رندر است. اما اگر همان کار را داخل `startTransition(async () => ...)` یا `<form action>` انجام دهید، خطا به Boundary می‌رسد – یکی از مزایای Actions.

♦ پرتاب عمدی به Boundary از handler

گاهی می‌خواهید خطای handler هم به Boundary برود (مثلاً session منقضی شده). با `useErrorBoundary` از `const { showBoundary } = useErrorBoundary(); ... catch (e) : react-error-boundary` `{ showBoundary(e); }`

گزینه‌های خطا در `createRoot` (React 19)

React 19 سه hook در سطح root اضافه کرد که برای لاگ کردن متمرکز ایده‌آل‌اند:

```
src/main.tsx

createRoot(document.getElementById('root')!, {
  // می‌رفت console.error قبلاً به گرفت Boundary خطایی که
  onError(error, errorInfo) {
    monitoring.capture(error, { stack: errorInfo.componentStack, handled: true });
  },
  // کرد unmount درخت را React نگرفت و Boundary خطایی که هیچ
  onUncaughtError(error, errorInfo) {
    monitoring.capture(error, { stack: errorInfo.componentStack, handled: false });
  },
  // (مثل hydration mismatch) از آن بازایی کرد React خطایی که
  onRecoverableError(error, errorInfo) {
    monitoring.capture(error, { level: 'warning' });
  },
}).render(<App />);
```

React 19 همچنین گزارش خطا را ساده کرد: به جای دو بار لاگ (یکی `throw` اصلی و یکی `rethrow`)، یک پیام واحد با تمام اطلاعات چاپ می‌شود.

Owner Stack برای دیباگ (React 19.1)

```
import { captureOwnerStack } from 'react';

// نشان می‌دهد کدام کامپوننت این کامپوننت را «رندر کرده: development فقط در
// اشتباه بسیار مفید props برای ردگیری - (DOM نه فقط والد)
console.log(captureOwnerStack());
```

خطاهای شبکه: الگوی نتیجه به جای throw

برای خطاهای «قابل انتظار» (اعتبارسنجی، ۴۰۴، دسترسی)، به جای throw یک شیء نتیجه برگردانید و در UI به درستی نمایش دهید. throw را برای خطاهای «غیرمنتظره» نگه دارید که Boundary باید بگیرد:

```
src/lib/result.ts

export type Result<T, E = string> = { ok: true; data: T } | { ok: false; error: E };

export async function safe<T>(p: Promise<T>): Promise<Result<T>> {
  try {
    return { ok: true, data: await p };
  } catch (e) {
    return { ok: false, error: e instanceof Error ? e.message : 'خطای ناشناخته' };
  }
}
```

کلاس‌های خطای معنادار

```
src/lib/errors.ts

export class AppError extends Error {
  constructor(
    message: string,
    public readonly code: string,
    public readonly status = 500,
  ) {
    super(message);
    this.name = 'AppError';
  }
}

export class NotFoundError extends AppError {
  constructor(what = 'منبع') { super(`یافت نشد`, 'NOT_FOUND', 404); }
}

export class UnauthorizedError extends AppError {
  constructor() { super('ابتدا وارد شوید', 'UNAUTHORIZED', 401); }
}
```

در Fallback بر اساس `error.code` تصمیم بگیرید: ۴۰۱ → ریدایرکت به لاگین، ۴۰۴ → صفحه «یافت نشد»، بقیه → پیام عمومی + تلاش مجدد.

ابزار	ویژگی
Sentry	@sentry/react با ErrorBoundary داخلی، replay جلسه، source map
Bugsnag / Rollbar	مشابه؛ گاهی ساده‌تر
OpenTelemetry	استاندارد باز؛ برای تیم‌هایی که observability یکپارچه می‌خواهند
	endpoint + window.onerror خودتان؛ برای پروژه‌های کوچک

هرچه باشد، **source map** را آپلود کنید وگرنه stack trace کد minify شده بی‌فایده است.

✖ اشتباه رایج: Boundary سراسری تنها

یک Boundary فقط دور `<App>` یعنی هر خطای کوچک، **کل** اپ را با صفحه خطا جایگزین می‌کند و همه state کاربر از بین می‌رود. Boundary ها را نزدیک محل احتمالی خطا بگذارید.

“ سؤالات مصاحبه (با پاسخ)

Junior – Error Boundary چه خطاهایی را نمی‌گیرد؟ خطاهای event handler، کد async خارج از Actions / Suspense (مثل `setTimeout`)، خطاهای خود Boundary، و خطاهای SSR (در فریم‌ورک با مکانیزم خودش مدیریت می‌شود).

Mid – چطور خطای یک query در TanStack Query را با Error Boundary و دکمه تلاش مجدد مدیریت می‌کنید؟ `useSuspenseQuery` (یا `throwOnError`) خطا را `throw` می‌کند تا Boundary بگیرد؛ `QueryErrorResetBoundary` تابع `reset` می‌دهد که در `onReset` صدا می‌زنیم تا کش خطا دار پاک و query دوباره اجرا شود.

Senior – تفاوت `onCaughtError`، `onUncaughtError` و `onRecoverableError` چیست و در پایش چه کاربردی دارند؟ Caught: خطایی که Boundary گرفت – severity متوسط، UI بازیابی شده. Uncaught: هیچ Boundary نبود، React root را `unmount` کرد – severity بحرانی، باید alert بزند. Recoverable: React hydration mismatch که با رندر کلاینت جبران شد) – هشدار برای کیفیت، نه incident. جداسازی این سه در dashboard پایش، نویز را کم و اولویت‌بندی را دقیق می‌کند.

25 دسترسی پذیری، RTL و بین‌المللی‌سازی

دسترسی‌پذیری (a11y) «ویژگی اضافه» نیست؛ بخشی از کیفیت است و در بسیاری از کشورها الزام قانونی. برای توسعه‌دهنده فارسی‌زبان، RTL و اعداد فارسی هم چالش‌های خاص خودشان را دارند. این فصل هر دو را پوشش می‌دهد.

اصول پایه دسترسی‌پذیری

اصل	در عمل
HTML معنایی اول	<code><button></code> ، <code><nav></code> ، <code><main></code> ، <code><h1></code> - <code><h6></code> قبل از هر ARIA
قابل استفاده با کیبورد	همه چیز با Tab/Enter/Space/Arrow قابل دسترسی باشد
فوکوس قابل مشاهده	هرگز <code>outline: none</code> بدون جایگزین
متن جایگزین	<code>alt</code> معنادار برای تصاویر؛ <code>alt=""</code> برای تزئینی
کنتراست	حداقل ۴.۵:۱ برای متن معمولی (WCAG AA)
برچسب فرم	هر <code>input</code> یک <code><label></code> متصل با <code>htmlFor</code>
اعلام تغییرات	<code>aria-live</code> برای پیام‌های پویا (toast، خطا)

i قانون اول ARIA

«از ARIA استفاده نکنید» – اگر عنصر بومی HTML همان معنا را دارد. `<button>` بهتر از `<div role="button">` است. ARIA فقط برای الگوهایی که HTML ندارد (tabs، combobox، tree). `tabIndex={0} onKeyDown=...`

```
function PasswordField() {
  const id = useId();
  const hintId = `${id}-hint`;
  return (
    <>
      <label htmlFor={id}>عبور رمز</label>
      <input id={id} type="password" aria-describedby={hintId} />
      <p id={hintId}>عدد شامل کاراکتر ۸ حداقل</p>
    </>
  );
}
```

useId شناسه‌ای یکتا و پایدار بین سرور و کلاینت می‌سازد؛ برخلاف `Math.random()` که در SSR باعث mismatch می‌شود. React 19.2 پیشنهاد آن را به `_r_` تغییر داد تا در `view-transition-name` معتبر باشد.

مدیریت فوکوس

```
src/features/todos/ToDoList.tsx

export function ToDoList() {
  const headingRef = useRef<HTMLHeadingElement>(null);
  const [items, setItems] = useState(initial);

  function remove(id: string) {
    setItems((s) => s.filter((i) => i.id !== id));
    // برود body به پس از حذف، فوکوس نباید «گم» شود
    headingRef.current?.focus();
  }

  return (
    <section>
      <h2 ref={headingRef} tabIndex=-1>وظایف</h2>
      {items.map((i) => (
        <ToDoItem key={i.id} item={i} onRemove={() => remove(i.id)} />
      ))}
    </section>
  );
}
```

قواعد فوکوس:

- ◆ هنگام باز شدن modal → فوکوس به داخل آن؛ هنگام بستن → به عنصری که آن را باز کرد (`<dialog>` بومی این را انجام می‌دهد).

- ♦ پس از نوبری SPA → فوکوس به `<h1>` صفحه جدید یا اعلام با `aria-live`.
- ♦ پس از حذف آیتم → فوکوس به آیتم بعدی یا عنوان لیست.
- ♦ عنصر را «برنامه‌ریزی‌شده قابل فوکوس» می‌کند بدون این‌که در ترتیب Tab قرار گیرد. `tabIndex={-1}`

Live Region برای پیام‌های پویا

```
src/components/ui/Toaster.tsx

export function Toaster({ toasts }: { toasts: Toast[] }) {
  return (
    <div aria-live="polite" className="fixed bottom-4 end-4 space-y-2">
      {toasts.map((t) => (
        <div
          key={t.id}
          role="status"
          className="rounded-lg bg-slate-900 px-4 py-2 text-white"
        >
          {t.message}
        </div>
      ))}
    </div>
  );
}
```

`polite` منتظر می‌ماند تا screen reader جمله فعلی را تمام کند؛ `assertive` فوراً قطع می‌کند (فقط برای خطاهای بحرانی). live region باید از ابتدا در DOM باشد تا تغییراتش اعلام شود.

ناوبری با کیبورد در لیست (Roving tabindex)

```
src/components/ui/Menu.tsx

export function Menu({ items }: { items: string[] }) {
  const [active, setActive] = useState(0);
  const refs = useRef<(HTMLButtonElement | null)[]>([]);

  function onKeyDown(e: React.KeyboardEvent) {
    const next =
      e.key === 'ArrowDown' ? (active + 1) % items.length
      : e.key === 'ArrowUp' ? (active - 1 + items.length) % items.length
      : e.key === 'Home' ? 0
      : e.key === 'End' ? items.length - 1
      : null;
    if (next !== null) {
      e.preventDefault();
      setActive(next);
      refs.current[next]?.focus();
    }
  }

  return (
    <div role="menu" onKeyDown={onKeyDown}>
      {items.map((label, i) => (
        <button
          key={label}
          role="menuitem"
          ref={(el) => { refs.current[i] = el; }}
          tabIndex={i === active ? 0 : -1}
        >
          {label}
        </button>
      ))}
    </div>
  );
}
```

فقط یک آیتم در ترتیب Tab است؛ Arrow بین آیتم‌ها حرکت می‌کند. این الگو در کتابخانه‌های Radix، headless، React Aria (آماده است – ترجیحاً از آن‌ها استفاده کنید).

index.html

```
<html lang="fa" dir="rtl">
```

موضوع	راهکار
فاصله و موقعیت	ویژگی‌های منطقی: <code>margin-inline-start</code> ، <code>inset-inline-end</code> (Tailwind) <code>ms-</code> ، <code>me-</code> (<code>start-</code> ، <code>end-</code>)
آیکون‌های جهت‌دار	فلش «بعدی» باید در RTL برعکس شود: <code>rtl:rotate-180</code> یا <code>scale-x-[-1]</code>
متن مختلط	<code>unicode-bidi: isolate</code> و <code><bdi></code> برای نام کاربری/کد داخل متن فارسی
اعداد و کد	<code>dir="ltr"</code> روی عنصر شماره تلفن، کد، URL
ورودی‌ها	<code>dir="auto"</code> روی <code><input></code> تا با محتوا تنظیم شود
انیمیشن اسلاید	بر اساس <code>document.dir</code> جهت را انتخاب کنید

```
<p>
  شد ثبت <span dir="ltr">{phone}</span> شماره با <bdi>{username}</bdi> کاربر
</p>
```


Intl API – اعداد، تاریخ و جمع‌بندی بدون کتابخانه

src/lib/format.ts

```
const rial = new Intl.NumberFormat('fa-IR', {
  style: 'currency', currency: 'IRR', maximumFractionDigits: 0,
});
export const formatPrice = (n: number) => rial.format(n);
// ۱,۲۵۰,۰۰۰ ریال

export const formatNumber = (n: number) => new Intl.NumberFormat('fa-IR').format(n);
// ۱۲,۳۴۵

export const formatDate = (d: Date) =>
  new Intl.DateTimeFormat('fa-IR', { dateStyle: 'long' }).format(d);
// fa-IR شهریور ۱۴۰۵ ← تقویم جلالی به صورت خودکار برای ۱۴

export const relativeTime = (minutes: number) =>
  new Intl.RelativeTimeFormat('fa', { numeric: 'auto' }).format(-minutes, 'minute');
// دقیقه پیش ۵

export const listFormat = (items: string[]) =>
  new Intl.ListFormat('fa', { type: 'conjunction' }).format(items);
// علی، رضا و سارا
```

برای تبدیل تاریخ‌های جلالی/میلادی و محاسبات تقویمی، `date-fns-jalali` یا `Temporal` (در حال ورود به مرورگرها) گزینه‌های خوبی‌اند.

بین‌المللی‌سازی (i18n) چندزبانه

Terminal

```
npm install i18next react-i18next i18next-browser-languagedetector
```

src/i18n.ts

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import LanguageDetector from 'i18next-browser-languagedetector';
import fa from './locales/fa.json';
import en from './locales/en.json';

i18n.use(LanguageDetector).use(initReactI18next).init({
  resources: { fa: { translation: fa }, en: { translation: en } },
  fallbackLng: 'fa',
  interpolation: { escapeValue: false },
});

i18n.on('languageChanged', (lng) => {
  document.documentElement.lang = lng;
  document.documentElement.dir = i18n.dir(lng);
});
```

```
const { t } = useTranslation();
<h1>{t('cart.title', { count: items.length })}</h1>
// fa.json → cart: { title_one: "{{count}} کالا", title_other: "{{count}} کالا" }
```

ابزارهای بررسی

ابزار	کاربرد
axe DevTools (افزونه)	اسکن خودکار صفحه؛ ۳۰-۴۰٪ مشکلات را پیدا می‌کند
eslint-plugin-jsx-a11y	خطاهای رایج در زمان نوشتن (تصویر بدون alt، روی div، onClick)
@axe-core/react	هشدار در کنسول هنگام توسعه
Lighthouse → Accessibility	امتیاز کلی
تست دستی با کیبورد	Tab بزنید؛ اگر جایی گیر کردید، کاربر هم گیر می‌کند
NVDA / VoiceOver	تجربه واقعی screen reader

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا `<div onClick>` به جای `<button>` مشکل ساز است؟ `div` قابل فوکوس نیست، با `Enter/Space` فعال نمی شود، و `screen reader` آن را «دکمه» اعلام نمی کند. برای جبران باید `role`، `tabIndex` و `onKeyDown` اضافه کنید که همه را `<button>` رایگان می دهد.

Mid – `aria-live="polite"` و `assertive` چه تفاوتی دارند و کی هر کدام؟ `polite` بعد از اتمام گفتار فعلی اعلام می کند (پیام های موفقیت، به روز رسانی ها). `assertive` فوراً قطع می کند (خطای بحرانی که مانع ادامه است). استفاده بیش از حد از `assertive` تجربه را آزاردهنده می کند.

Senior – چالش های RTL در اپلیکیشن React چندزبانه چیست و چطور به صورت سیستمی حل می کنید؟ چالش ها: استایل های فیزیکی (`left/right`) پراکنده، آیکون های جهت دار، انیمیشن ها، کتابخانه های third-party بدون پشتیبانی RTL، و متن مختلط. راه حل سیستمی: ESLint/Stylelint برای ممنوع کردن ویژگی های فیزیکی، design token برای جهت، wrapper برای آیکون های جهت دار با flip خودکار، تست بصری (Storybook + Chromatic) در هر دو جهت، و `dir` مشتق از زبان در یک نقطه (root).

26 امنیت اپلیکیشن React

React به طور پیش فرض در برابر XSS محافظت می کند، اما امنیت اپلیکیشن بسیار فراتر از آن است: کجا توکن را نگه داریم؟ چطور از CSRF جلوگیری کنیم؟ چه وابستگی هایی خطرناک اند؟ این فصل چک لیست عملی امنیت Front-End را ارائه می دهد.

| XSS – چیزی که React برایتان انجام می دهد و چیزی که نمی دهد

React همه مقادیر را در JSX **escape** می کند: `<p>{userInput}</p>` هرگز HTML اجرا نمی کند. اما چند در پشتی وجود دارد:

خطر	مثال	راهکار
<code>dangerouslySetInnerHTML</code>	رندر HTML از CMS	پاک سازی با DOMPurify قبل از رندر
href با <code>javascript:</code>	<code></code>	فقط <code>http(s):</code> مجاز؛ React 19 هشدار می دهد و در آینده بلاک می کند
ویژگی های event از داده	<code><div {...userProps}></code>	هرگز شیء کنترل نشده را spread نکنید
SVG/Markdown از کاربر	<code><svg onLoad></code>	DOMPurify با پروفایل SVG یا رندر امن Markdown
<code>new Function / eval</code>	تفسیر کد کاربر	ممنوع

src/components/RichText.tsx

```
import DOMPurify from 'dompurify';
import { useMemo } from 'react';

export function RichText({ html }: { html: string }) {
  const clean = useMemo(
    () => DOMPurify.sanitize(html, { USE_PROFILES: { html: true } }),
    [html],
  );
  return <div className="prose" dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

src/lib/safeUrl.ts

```
const SAFE_PROTOCOLS = ['http:', 'https:', 'mailto:', 'tel:'];

export function safeHref(url: string, fallback = '#') {
  try {
    const u = new URL(url, window.location.origin);
    return SAFE_PROTOCOLS.includes(u.protocol) ? u.href : fallback;
  } catch {
    return fallback;
  }
}
```

احراز هویت: توکن را کجا نگه داریم؟

کوکی httpOnly ✓

- ♦ JavaScript نمی‌تواند بخواند → XSS نمی‌تواند سرقت کند
- ♦ CSRF → SameSite=Lax/Strict + Secure محدود
- ♦ مرورگر خودکار ارسال می‌کند (credentials: 'include')

localStorage ✗

- ♦ هر اسکریپت (از جمله XSS و وابستگی آلوده) می‌تواند بخواند
- ♦ توکن سرقت‌شده تا انقضا معتبر است
- ♦ «راحت است» تنها مزیتش است

الگوی توصیه‌شده: **session cookie** (یا refresh token در کوکی + httpOnly + access token کوتاه‌عمر در حافظه):

src/features/auth/session.ts

```
let accessToken: string | null = null; // فقط در حافظه؛ با رفرش صفحه پاک می‌شود

export async function refreshAccessToken() {
  const res = await fetch('/api/auth/refresh', {
    method: 'POST', credentials: 'include', // کوکی HttpOnly همراه درخواست می‌رود
  });
  if (!res.ok) throw new UnauthorizedError();
  accessToken = (await res.json()).accessToken;
  return accessToken;
}

export const getAccessToken = () => accessToken;
```

```

async function request<T>({
  path: string, init?: RequestInit, retry = true,
}): Promise<T> {
  const res = await fetch(url(path), {
    ...init,
    credentials: 'include',
    headers: { ...init?.headers, Authorization: `Bearer ${getAccessToken()} ?? ''` },
  });
  if (res.status === 401 && retry) {
    await refreshAccessToken(); // یک بار تلاش مجدد
    return request<T>(path, init, false);
  }
  // ...
}

```

⚠ «محافظة مسیر» در کلاینت امنیت نیست

مخفی کردن دکمه «حذف» یا ریدایرکت کاربر غیرادمین در React فقط UX است. هر کسی می‌تواند با DevTools آن را دور بزند. هر بررسی مجوز باید در سرور تکرار شود. کلاینت را طوری بنویسید که انگار کاربر مهاجم است.

CSRF

اگر از کوکی برای session استفاده می‌کنید، مرورگر آن را با هر درخواست به دامنه شما می‌فرستد – حتی از سایت مهاجم. دفاع‌ها:

- ◆ `SameSite=Lax` (پیش‌فرض مرورگرهای مدرن) درخواست‌های POST cross-site را بدون کوکی می‌فرستد.
- ◆ **توکن CSRF** (double-submit cookie یا synchronizer) برای API‌های حساس؛ در `X-CSRF-Token` header ارسال کنید.
- ◆ بررسی `Referer / Origin` header در سرور.
- ◆ اگر API فقط با `Authorization: Bearer` (بدون کوکی) کار می‌کند، CSRF عملاً منتفی است.

Content Security Policy (CSP)

CSP لایه دفاعی قدرتمندی است که حتی اگر XSS رخ دهد، اجرای اسکریپت inline و بارگذاری از دامنه‌های ناشناس را بلاک می‌کند:

HTTP header

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-{random}';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https://cdn.example.com;
  connect-src 'self' https://api.example.com;
  frame-ancestors 'none';
  base-uri 'self';
```

Vite در build خروجی بدون inline script تولید می‌کند (به‌جز اگر خودتان اضافه کنید)، پس `script-src 'self'` کافی است. اسکریپت تم inline (فصل ۱۷) به `nonce` نیاز دارد.

امنیت وابستگی‌ها (Supply Chain)

Terminal

```
npm audit           # آسیب‌پذیری‌های شناخته‌شده
npx npm-check-updates # نسخه‌های قدیمی
npm ci              # در CI lockfile نصب دقیق از
```

- ♦ **lockfile** را **commit** کنید و در CI از `npm ci` استفاده کنید.
- ♦ **Dependabot / Renovate** برای به‌روزرسانی خودکار با PR.
- ♦ پکیج‌های کم‌ستاره با دسترسی `postinstall` را بررسی کنید.
- ♦ **پین‌کردن نسخه** ابزارهای build (`--save-exact`) برای `babel-plugin-react-compiler` توصیه رسمی (است).

✖ آسیب‌پذیری‌های (۲۰۲۵-۲۰۲۶) React Server Components

در دسامبر ۲۰۲۵ آسیب‌پذیری بحرانی **CVE-2025-55182** (معروف به React2Shell) در پکیج‌های `react-server-dom-*` کشف شد که اجرای کد از راه دور روی سرور را ممکن می‌کرد، و در ادامه چند مورد DoS و افشای سورس تا ژانویه ۲۰۲۶. اگر از (Next.js App Router، React Router Framework، ...) RSC استفاده می‌کنید، حتماً روی **React 19.0.4 / 19.1.5 / 19.2.4** یا بالاتر و آخرین نسخه فریم‌ورک باشید. SPAهای خالص (بدون RSC) تحت تأثیر نبودند. درس: به‌روزرسانی امنیتی را در همان هفته اعمال کنید.

اسرار و متغیرهای محیطی

- ♦ هر چیزی با پیشوند `VITE_` در باندل عمومی است – هرگز کلید خصوصی.
- ♦ کلیدهای «عمومی» (مثل Google Maps با محدودیت دامنه) قابل قبول اند اما محدودیت `referrer`/دامنه را در پنل سرویس فعال کنید.
- ♦ عملیات حساس (پرداخت، ارسال ایمیل، AI API) همیشه از طریق بک‌اند یا `Server Function`.

چک‌لیست امنیت Front-End

- هیچ `dangerouslySetInnerHTML` بدون `DOMPurify`
- URLهای کاربر با `safeHref` فیلتر می‌شوند
- توکن در `localStorage` نیست؛ `session` با کوکی `httpOnly/Secure/SameSite`
- هر بررسی مجوز در سرور تکرار شده
- CSP فعال و بدون `unsafe-inline` برای `script`
- `npm audit` در CI بدون خطای `high/critical`
- React و فریم‌ورک روی نسخه‌های وصله‌شده
- هیچ `secret` در `VITE_*`
- فرم‌ها `rate-limit` سمت سرور دارند (کلاينت فقط `debounce`)
- هدرهای امنیتی: `Permissions-Policy` , `Referrer-Policy` , `X-Content-Type-Options`

“سؤالات مصاحبه (با پاسخ)”

React – Junior چطور از XSS جلوگیری می‌کند؟ با `escape` خودکار همه مقادیر داخل `JSX`؛ رشته `<script>` به‌صورت متن نمایش داده می‌شود نه اجرا. استثناها `dangerouslySetInnerHTML` و `href` های `javascript:` هستند.

Mid – چرا ذخیره JWT در localStorage توصیه نمی‌شود و جایگزین چیست؟ هر XSS یا وابستگی آلوده می‌تواند آن را بخواند و به سرور مهاجم بفرستد. جایگزین: کوکی `httpOnly` (غیرقابل خواندن با JS) با `Secure` و `SameSite`، به‌همراه دفاع `CSRF`؛ یا `access token` کوتاه‌عمر فقط در حافظه + `refresh token` در کوکی `httpOnly`.

CSP – Senior با `nonce` در یک SPA چطور پیاده می‌شود و چه چالش‌هایی دارد؟ سرور برای هر پاسخ HTML یک `nonce` تصادفی می‌سازد، در `CSP header` و روی هر `<script nonce>` قرار می‌دهد. چالش‌ها: با فایل `index.html` استاتیک (CDN) `nonce` ثابت می‌شود که بی‌معناست – باید HTML را در `edge`/سرور رندر کرد یا از `hash` به‌جای `nonce` استفاده کرد؛ کتابخانه‌هایی که `inline style/script` تزریق می‌کنند (بعضی `CSS-in-JS`، ابزارهای `analytics`) می‌شکنند و باید `nonce` به آن‌ها پاس داده شود؛ و `'strict-dynamic'` برای اجازه به اسکریپت‌های بارگذاری‌شده توسط اسکریپت معتبر لازم می‌شود.

27 تست‌نویسی: Vitest، Testing Library و Playwright

تست‌ها به شما اجازه می‌دهند با اطمینان ری‌فکتور کنید و باگ‌ها را قبل از کاربر بگیرید. فلسفه مدرن تست React ساده است: «تست‌هایتان باید شبیه استفاده کاربر از نرم‌افزار باشد». این فصل ابزارها و الگوهای آن را آموزش می‌دهد.

هرم تست در Front-End



راه‌اندازی Vitest + Testing Library

Terminal

```
npm install -D vitest jsdom @testing-library/react \
  @testing-library/user-event @testing-library/jest-dom
```

vite.config.ts

```
/// <reference types="vitest/config" />
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test/setup.ts',
    css: false,
  },
});
```

src/test/setup.ts

```
import '@testing-library/jest-dom/vitest';
import { cleanup } from '@testing-library/react';
import { afterEach } from 'vitest';

afterEach(() => cleanup());
```

تست کامپوننت: رفتار، نه پیاده‌سازی

```
src/features/counter/Counter.test.tsx

import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

describe('Counter', () => {
  it('با کلیک افزایش می‌یابد', async () => {
    const user = userEvent.setup();
    render(<Counter />);

    const button = screen.getByRole('button', { name: /افزایش/ });
    await user.click(button);
    await user.click(button);

    expect(screen.getByText('۲ تعداد')).toBeInTheDocument();
  });

  it('در صفر دکمه کاهش غیرفعال است', () => {
    render(<Counter />);
    expect(screen.getByRole('button', { name: /کاهش/ })).toBeDisabled();
  });
});
```

اولویت انتخاب‌گرها (query priority)

اولویت	Query	چرا
۱	<code>getByRole('button', { name })</code>	همان‌طور که screen reader می‌بیند؛ دسترسی‌پذیری را هم تست می‌کند
۲	<code>getByLabelText</code>	فرم‌ها
۳	<code>getByPlaceholderText</code> ، <code>getByText</code>	متن قابل‌مشاهده
۴	<code>getByDisplayValue</code>	مقدار فعلی input
آخر	<code>getByTestId</code>	فقط وقتی هیچ راه معنایی نیست

✖ تست پیاده‌سازی نکنید

`expect(setState).toHaveBeenCalled()`، بررسی نام کلاس CSS، یا تست state داخلی یعنی با هر ری‌فکتور تست می‌شکند بدون این‌که باگی وجود داشته باشد. فقط چیزی را تست کنید که کاربر می‌بیند یا انجام می‌دهد.

تست فرم با Actions

```
src/features/auth/LoginForm.test.tsx

import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { LoginForm } from './LoginForm';

it('خطای ایمیل نامعتبر را نشان می‌دهد', async () => {
  const user = userEvent.setup();
  render(<LoginForm />);

  await user.type(screen.getByLabelText('ایمیل'), 'not-an-email');
  await user.type(screen.getByLabelText('رمز عبور'), '12345678');
  await user.click(screen.getByRole('button', { name: 'ورود' }));

  expect(await screen.findByRole('alert')).toHaveTextContent('ایمیل معتبر نیست');
});

it('هنگام ارسال دکمه غیرفعال می‌شود', async () => {
  const user = userEvent.setup();
  render(<LoginForm />);
  await user.type(screen.getByLabelText('ایمیل'), 'a@b.com');
  await user.type(screen.getByLabelText('رمز عبور'), '12345678');
  await user.click(screen.getByRole('button', { name: 'ورود' }));

  expect(screen.getByRole('button', { name: /در حال/ })).toBeDisabled();
  await waitFor(() => {
    expect(screen.queryByRole('button', { name: /در حال/ })).not.toBeInTheDocument(),
  });
});
```

`findBy*` منتظر ظاهر شدن عنصر می‌ماند (تا ۱ ثانیه) – برای `Suspense` و `Actions` ضروری است.

Mock شبکه با MSW

به جای mock کردن `fetch` یا `axios`، شبکه را در سطح **درخواست** شبیه‌سازی کنید؛ کد اپ بدون تغییر اجرا می‌شود:

Terminal

```
npm install -D msw
```

src/test/handlers.ts

```
import { http, HttpResponse } from 'msw';

export const handlers = [
  http.get('/api/products', () =>
    HttpResponse.json([
      { id: '1', title: 'کتاب React', price: 250000 }
    ]),
  ),
  http.post('/api/login', async ({ request }) => {
    const body = (await request.json()) as { email: string };
    if (!body.email.includes('@')) {
      return HttpResponse.json(
        { error: 'ایمیل معتبر نیست' }, { status: 400 }
      );
    }
    return HttpResponse.json({ token: 'fake' });
  }),
];
```

src/test/setup.ts (افزوده)

```
import { setupServer } from 'msw/node';
import { handlers } from '../handlers';

export const server = setupServer(...handlers);
beforeAll(() => server.listen({ onUnhandledRequest: 'error' }));
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

```
// override در یک تست خاص
server.use(
  http.get('/api/products', () => HttpResponse.error())
);
render(<ProductList />, { wrapper: Providers });
expect(await screen.findByRole('alert')).toHaveTextContent('مشکلی پیش آمد');
```

MSW در مرورگر هم کار می‌کند (Service Worker) — برای توسعه بدون بک‌اند و Storybook.

src/test/utils.tsx

```
import { render, type RenderOptions } from '@testing-library/react';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { MemoryRouter } from 'react-router';
import type { ReactNode } from 'react';

type Options = RenderOptions & { route?: string };

export function renderWithProviders(
  ui: ReactNode,
  { route = '/', ...options }: Options = {},
) {
  const qc = new QueryClient({ defaultOptions: { queries: { retry: false } } });
  return render(
    <QueryClientProvider client={qc}>
      <MemoryRouter initialEntries={[route]}>{ui}</MemoryRouter>
    </QueryClientProvider>,
    options,
  );
}
```

src/hooks/useDebounceValue.test.ts

```
import { renderHook, act } from '@testing-library/react';
import { useDebounceValue } from './useDebounceValue';

it('مقدار را با تأخیر به روز می‌کند', () => {
  vi.useFakeTimers();
  const { result, rerender } = renderHook(({ v }) => useDebounceValue(v, 300), {
    initialProps: { v: 'a' },
  });

  rerender({ v: 'ab' });
  expect(result.current).toBe('a');

  act(() => vi.advanceTimersByTime(300));
  expect(result.current).toBe('ab');
  vi.useRealTimers();
});
```

تست E2E با Playwright

Terminal

```
npm init playwright@latest
```

e2e/checkout.spec.ts

```
import { test, expect } from '@playwright/test';

test('کاربر می‌تواند محصول را به سبد اضافه کند و پرداخت کند', async ({ page }) => {
  await page.goto('/');
  await page.getByRole('link', { name: 'کتاب React' }).click();
  await page.getByRole('button', { name: 'افزودن به سبد' }).click();
  await expect(page.getByTestId('cart-badge')).toHaveText('1');

  await page.getByRole('link', { name: 'سبد خرید' }).click();
  await page.getByRole('button', { name: 'پرداخت' }).click();
  await expect(page).toHaveURL(/\/checkout\/success/);
  await expect(page.getByRole('heading', { name: 'سفارش ثبت شد' })).toBeVisible();
});
```

Playwright روی Chromium، Firefox و WebKit اجرا می‌شود، trace و ویدئو ضبط می‌کند و در CI پایدار است. فقط جریان‌های حیاتی (ثبت‌نام، خرید، جستجو) را E2E کنید؛ بقیه را با تست کامپوننت.

چه چیزی را تست کنیم؟

اولویت بالا ☒	اولویت پایین ↓
منطق کسب‌وکار (reducer، محاسبات قیمت)	کامپوننت‌های صرفاً نمایشی
فرم‌ها و اعتبارسنجی	استایل و layout
جریان‌های حیاتی کاربر (E2E)	کد third-party
مدیریت خطا و حالت‌های لبه	getter/setter ساده
هوک‌های سفارشی با منطق	wrapper نازک

✦ Coverage عدد جادویی نیست

۱۰۰٪ coverage یعنی هر خط اجرا شده، نه این‌که درست کار می‌کند. هدف واقع‌بینانه: ۷۰-۸۰٪ با تمرکز روی منطق مهم. تست‌های خوب با **اعتماد ری‌فکتور** را ممکن می‌کنند؛ این معیار موفقیت است.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا Testing Library به جای دسترسی به state کامپوننت، DOM را بررسی می‌کند؟ چون کاربر state را نمی‌بیند؛ DOM را می‌بیند. تست بر اساس خروجی قابل مشاهده، با ری‌فکتور داخلی نمی‌شکند و دسترسی‌پذیری را هم ضمناً بررسی می‌کند.

Mid – MSW چه مزیتی نسبت به `vi.mock('./api')` دارد؟ کد واقعی `fetch/apiClient` اجرا می‌شود (شامل `header`، `serialization`، مدیریت خطا)، `mock`ها بین تست، `Storybook` و توسعه مشترک‌اند، و تست به ساختار داخلی ماثول API وابسته نیست.

Senior – استراتژی تست برای اپلیکیشنی با `Suspense`، `transitions` و `Optimistic UI` چیست؟ تست‌های کامپوننت باید `async-aware` باشند: `findBy*` و `waitFor` برای `Suspense`، `act` برای `transitions`، و بررسی هر دو حالت خوش‌بینانه و نهایی. `MSW` با `delay()` برای شبیه‌سازی تأخیر و خطا. برای رفتارهای وابسته به زمان از `fake timers`. `E2E` با `Playwright` برای اطمینان از این‌که ترکیب واقعی (شبکه واقعی، مرورگر واقعی) کار می‌کند – چون `jsdom` برخی APIها (`layout`، `IntersectionObserver`) را ندارد.

28 Server Functions و Server Components

React Server Components (RSC) بزرگ‌ترین تغییر معماری React از ابتدای آن است: کامپوننت‌هایی که فقط روی سرور اجرا می‌شوند و هیچ JavaScript به مرورگر نمی‌فرستند. این فصل مدل ذهنی درست را می‌سازد تا وقتی به Next.js یا React Router Framework رفتید، دقیقاً بدانید چه اتفاقی می‌افتد.

! مدل ذهنی: React برای دو کامپیوتر

تا اینجا کتاب، همه کامپوننت‌ها روی یک کامپیوتر (مرورگر) اجرا می‌شدند. RSC یک محیط دوم اضافه می‌کند: **سرور** (در زمان build یا هر درخواست). حالا هر کامپوننت به یکی از دو دنیا تعلق دارد:

Client Component	Server Component	
سرور (SSR) و مرورگر	فقط سرور (request یا build)	کجا اجرا می‌شود
✓ در باندل	✗ صفر	JS به کلاینت
✗	✓	دسترسی مستقیم به DB / FS / secrets
✓	✗	event handler , useEffect , useState
✗ (از use استفاده کنید)	✓	async/await در بدنه کامپوننت
'use client' بالای فایل	پیش‌فرض (در فریم‌ورک RSC)	نحوه علامت‌گذاری

```
app/products/page.tsx (Server Component)

import { db } from '@lib/db';
import { AddToCartButton } from './AddToCartButton';

export default async function ProductsPage() {
  const products = await db.product.findMany(); // API مستقیم از دیتابیس! بدون
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          {p.title} - {p.price}
          <AddToCartButton productId={p.id} />      {/* جزیره تعاملی */}
        </li>
      ))}
    </ul>
  );
}
```


app/products/AddToCartButton.tsx (Client Component)

```
'use client';
import { useState } from 'react';

export function AddToCartButton({ productId }: { productId: string }) {
  const [added, setAdded] = useState(false);
  const handleClick = () => {
    addToCart(productId);
    setAdded(true);
  };
  return <button onClick={handleClick}>{added ? '✓' : 'افزودن'}</button>;
}
```

i RSC به فریم‌ورک نیاز دارد

React خودش فقط پروتکل و runtime را ارائه می‌دهد؛ باندلر و سرور را فریم‌ورک تأمین می‌کند. در ۲۰۲۶: **Next.js** (App Router)، **React Router v8 (Framework mode)**، **TanStack Start**، **Waku** و **Parcel/Vite** با پلاگین **RSC** (تجربی). **SPA** خالص **Vite** بدون پلاگین، **RSC** ندارد.

مرز 'use client' چگونه کار می‌کند؟

'use client' یک مرز تعریف می‌کند، نه «این کامپوننت کلاینتی است». هر چیزی که از یک فایل `import 'use client'` شود، وارد باندل کلاینت می‌شود – از جمله همه وابستگی‌هایش.

Component tree

```
<Page>                                Server
├── <Header>                            Server
├── <Sidebar>                          'use client' └─ باندل کلاینت
│   ├── <NavItem>                      └─ (کلاینت، حتی بدون دایرکتیو)
└── <Article>                          Server
    └── <LikeButton>                  'use client'
```

- ♦ Server Component می‌تواند Client Component را رندر کند ✓
- ♦ Client Component نمی‌تواند Server Component را import کند ✗
- ♦ اما Client Component می‌تواند Server Component را به‌عنوان `children` بگیرد ✓ (الگوی «سوراخ»)

```
// ✓ الگوی children: Server Component داخل Client Component
<ClientLayout>                { /* 'use client' */ }
  <ServerContent />           { /* Server به‌عنوان children پاس داده می‌شود */ }
</ClientLayout>
```

⚠ props باید سریال‌پذیر باشند

از Server به Client فقط مقادیر قابل‌سریال‌سازی پاس می‌شوند: primitive، آرایه، شیء ساده، Date، Map/Set، Promise (با `use` در کلاینت خوانده می‌شود) و **Server Function**. تابع معمولی، کلاس یا JSX با `state.X` اگر می‌خواهید تابع پاس دهید، باید Server Function باشد.

'use server' – Server Functions

تابعی که روی سرور اجرا می‌شود اما از کلاینت (مثلاً `<form action>`) قابل‌فراخوانی است. فریم‌ورک آن را به یک endpoint امن تبدیل می‌کند:

```
app/actions/comments.ts

'use server';
import { db } from '@lib/db';
import { getSession } from '@lib/auth';
import { z } from 'zod';

const schema = z.object({ postId: z.string(), text: z.string().min(3) });

export async function addComment(prev: { error?: string }, formData: FormData) {
  const session = await getSession();
  if (!session) return { error: 'ابتدا وارد شوید' }; // ✓ بررسی مجوز در سرور

  const parsed = schema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: 'ورودی نامعتبر' };

  await db.comment.create({ data: { ...parsed.data, userId: session.userId } });
  revalidatePath(`/posts/${parsed.data.postId}`); // (Next.js) کش را باطل کن
  return {};
}
```

```
'use client';
import { useActionState } from 'react';
import { addComment } from '@app/actions/comments';

export function CommentForm({ postId }: { postId: string }) {
  const [state, action, pending] = useActionState(addComment, {});
  return (
    <form action={action}>
      <input type="hidden" name="postId" value={postId} />
      <textarea name="text" required />
      {state.error && <p role="alert">{state.error}</p>}
      <button disabled={pending}>ارسال</button>
    </form>
  );
}
```

همان `useActionState` فصل ۹ – فقط تابع روی سرور اجرا می‌شود. این فرم بدون JavaScript هم کار می‌کند (Progressive Enhancement واقعی).

✖ Server Function = endpoint عمومی

هر Server Function یک URL عمومی است که هر کسی می‌تواند با هر ورودی صدا بزند. همیشه داخل آن احراز هویت، مجوز و اعتبارسنجی ورودی انجام دهید – دقیقاً مثل یک REST endpoint. اسرار را در آرگومان‌ها یا مقدار بازگشتی قرار ندهید.

cache و cacheSignal — deduplication در سرور

در یک درخواست، چند Server Component ممکن است داده یکسانی بخواهند. `cache` تضمین می‌کند تابع فقط یک بار به‌ازای هر درخواست اجرا شود:

```
lib/data.ts

import { cache, cacheSignal } from 'react';

export const getUser = cache(async (id: string) => {
  // cacheSignal (React 19.2): abort وقتی رندر تمام/لغو شد، درخواست
  const signal = cacheSignal() ?? undefined;
  const res = await fetch(`${API}/users/${id}`, { signal });
  return res.json() as Promise<User>;
});

// واقعی fetch فقط یک — در سه کامپوننت مختلف
```

`cache` فقط در Server Components معنا دارد و عمرش یک درخواست است (نه کش بین درخواست‌ها؛ آن را فریم‌ورک با `'use cache'` یا مشابه ارائه می‌دهد).

Suspense و Streaming در سرور

```
app/dashboard/page.tsx

export default function Dashboard() {
  return (
    <>
      <Header />                                {/* فوری */}
      <Suspense fallback=<StatsSkeleton />>
        <SlowStats />                            {/* stream می‌شود؛ بعداً await */}
      </Suspense>
      <Suspense fallback=<FeedSkeleton />>
        <Feed />
      </Suspense>
    </>
  );
}
```

سرور HTML را **تکه‌تکه** می‌فرستد: ابتدا با shell با اسکلت‌ها، سپس هر بخش وقتی آماده شد. کاربر TTFB بسیار پایین می‌بیند و بخش‌های سریع را زودتر می‌گیرد. React 19.2 مرزهای Suspense را در SSR دسته‌ای آشکار می‌کند تا از پرش‌های پی‌درپی جلوگیری شود.

پاس دادن Promise از سرور به کلاینت

```
// Server Component
export default function Page() {
  const commentsPromise = getComments(); // بدون await!
  return (
    <Suspense fallback={<p>...</p>}>
      <Comments promise={commentsPromise} />
    </Suspense>
  );
}

// Client Component
'use client';
export function Comments({ promise }: { promise: Promise<Comment[]> }) {
  const comments = use(promise); // stream می‌شود تا suspend در کلاینت
  return comments.map(c => <li key={c.id}>{c.text}</li>);
}
```

این الگو اجازه می‌دهد رندر سرور بلاک نشود، در حالی که کامپوننت تعاملی کلاینت داده را دریافت می‌کند.

چه چیزی کجا برود؟

بگذارید در Server Component	بگذارید در Client Component
واکنشی داده، دسترسی به DB	event handler ، <code>useReducer</code> / <code>useState</code>
کتابخانه‌های سنگین (markdown parser, syntax highlighter)	<code>Effect</code> ها، API های مرورگر (<code>window</code> , <code>localStorage</code>)
کد وابسته به secret	Context provider و مصرف‌کننده‌ها
رندر محتوای استاتیک و لیست‌ها	انیمیشن، drag & drop، فرم‌های تعاملی پیچیده

قاعده: مرز `'use client'` را تا حد ممکن پایین (برگ‌ها) نگه دارید. به جای کل صفحه، فقط دکمه را کلاینتی کنید.

Partial Pre-rendering (React 19.2)

API های جدید `prerender` + `resume` اجازه می‌دهند shell استاتیک صفحه در زمان build ساخته و در CDN کش شود، و بخش‌های پویا در زمان درخواست «ادامه» یابند. Next.js این را با عنوان PPR ارائه می‌دهد: بهترین‌های استاتیک (سرعت CDN) و پویا (شخصی‌سازی) در یک صفحه.

“ سؤالات مصاحبه (با پاسخ)

Junior – تفاوت Server Component و SSR چیست؟ SSR یک Client Component را روی سرور به HTML تبدیل می‌کند اما JS آن باز هم به مرورگر می‌رود و hydrate می‌شود. Server Component هرگز به کلاینت نمی‌رود؛ فقط خروجی‌اش (در قالب RSC payload) ارسال می‌شود. می‌توان هر دو را با هم داشت.

Mid – چرا Client Component نمی‌تواند Server Component را import کند ولی می‌تواند به‌عنوان children بگیرد؟ import یعنی «این کد را در باندل من قرار بده» – که برای کد سرور معنا ندارد. اما children در سرور رندر شده و به‌صورت payload آماده به کلاینت می‌رسد؛ Client Component فقط آن را جای‌گذاری می‌کند.

Senior – RSC payload چیست و چرا به‌جای HTML از آن استفاده می‌شود؟ یک فرمت متنی stream‌پذیر که درخت React (نه HTML) را توصیف می‌کند: عناصر، props سریال‌شده، ارجاع به Client Component‌ها (به‌صورت module reference) و Promise‌های در حال انتظار. مزیت نسبت به HTML: هنگام ناوبری کلاینت، React می‌تواند درخت جدید را با درخت فعلی reconcile کند و state کامپوننت‌های کلاینت (مثل input یا اسکرول) را حفظ کند – کاری که با جایگزینی HTML ممکن نیست.

29 انتخاب فریم‌ورک: Next.js، React Router و TanStack Start

بعد از تسلط بر React، سؤال بعدی این است که آن را با چه چیزی به Production ببرید. این فصل بدون تعصب، گزینه‌های سال ۲۰۲۶ را با معیارهای واقعی مقایسه می‌کند تا برای هر پروژه انتخاب درستی داشته باشید.

اول: آیا اصلاً به فریم‌ورک نیاز دارید؟

به فریم‌ورک نیاز دارید اگر...	SPA با Vite کافی است اگر...
SEO مهم است (فروشگاه، بلاگ، لندینگ)	اپ پشت لاگین است (داشبورد، ادمین، ابزار داخلی)
می‌خواهید بک‌اند و فرانت در یک پروژه باشند	بک‌اند جداگانه و تیم بک‌اند دارید
کارایی بارگذاری اولیه در موبایل/شبکه ضعیف مهم است	کاربران اینترنت خوب دارند و TTFB حیاتی نیست
می‌خواهید از RSC، Streaming، Server Functions استفاده کنید	می‌خواهید کنترل کامل و کمترین «جادو» داشته باشید
زیرساخت Node/Edge دارید	استقرار استاتیک روی هر CDN/هاست ساده

توصیه رسمی تیم React برای پروژه جدید، استفاده از فریم‌ورک است؛ اما «ابزار داخلی با Vite» همچنان کاملاً معتبر و رایج است.

گزینه‌ها در یک نگاه

Vite SPA	TanStack Start	React Router v8 (Framework)	Next.js 16	
✗	✓ (در حال تکمیل)	✓ (تجربی → پایدار)	✓ بالغ	RSC
/ React Router TanStack Router	فایل محور، -type safe کامل	فایل محور یا config	فایل محور (App Router)	روتینگ
بستگی به روتر	✓ عالی	خوب (typegen)	متوسط	Type-safety مسیر
✗	✓	✓	✓	Server Functions
—	ساده	ساده (HTTP headers)	✓ پیشرفته (PPR , 'use cache')	کش و ISR
Vite	Vite	Vite	Turbopack	باندلر
هر CDN استاتیک	هر Node/Edge؛ adapterها	هر Node/Edge؛ adapterها	Vercel بهینه؛ Docker / Node هر جا	استقرار
کم	متوسط	متوسط (میراث Remix)	بالا تر (قواعد کش، RSC)	منحنی یادگیری
زیاد	کم اما رو به رشد	در حال رشد	بیشترین	بازار کار ایران

بالغترین پیاده‌سازی RSC با اکوسیستم عظیم. مفاهیم کلیدی: App Router (/app/)، Layout های تودرتو، (جانشین proxy.ts، Cache Components و 'use cache'، Server Actions، error.tsx / loading.tsx middleware)، Turbopack. برای این فریم‌ورک، کتاب «مرجع جامع Next.js 16» از همین نویسنده را مطالعه کنید.

```

Next.js App Router

app/
├── layout.tsx           # Root layout (Server)
├── page.tsx             # /
├── products/
│   ├── page.tsx        # /products (async Server Component)
│   ├── loading.tsx     # Suspense fallback خودکار
│   └── [id]/page.tsx    # /products/:id
└── api/.../route.ts    # Route Handlers
  
```

React Router v8 – حالت Framework |

تکامل Remix؛ روی Vite. اگر حالت Data (فصل ۱۶) را بلدید، ۸۰٪ راه را رفته‌اید: همان loader / action، فقط روی سرور اجرا می‌شوند.

```

app/routes/products.$id.tsx

import type { Route } from './+types/products.$id';

export async function loader({ params }: Route.LoaderArgs) {
  const product = await db.product.findUnique({ where: { id: params.id } });
  if (!product) throw new Response('Not Found', { status: 404 });
  return { product };
}

export default function ProductPage({ loaderData }: Route.ComponentProps) {
  return <h1>{loaderData.product.title}</h1>; // تایپ‌ها خودکار تولید می‌شوند
}
  
```

مزایا: ساده‌تر از Next.js، وابستگی کمتر به یک ارائه‌دهنده، مستندات عالی، استانداردهای وب (Request/Response). مسیر مهاجرت رسمی از SPA وجود دارد.

جدیدترین گزینه از سازندگان TanStack Query/Router. برگ برنده: **type-safety کامل** — پارامترهای مسیر، search params، loader data و حتی لینک‌ها در زمان کامپایل بررسی می‌شوند. برای تیم‌های TypeScript-محور که از TanStack Query استفاده می‌کنند، یکپارچگی بی‌نظیری دارد. هنوز جوان است؛ برای پروژه‌های سازمانی محافظه‌کارانه صبر کنید.

```
src/routes/products.$id.tsx

import { createFileRoute } from '@tanstack/react-router';
import { createServerFn } from '@tanstack/react-start';

const getProduct = createServerFn({ method: 'GET' })
  .validator((id: string) => id)
  .handler(async ({ data: id }) => db.product.findUnique({ where: { id } }));

export const Route = createFileRoute('/products/$id')({
  loader: ({ params }) => getProduct({ data: params.id }),
  component: () => {
    const product = Route.useLoaderData(); // cast تایپ دقیق، بدون
    return <h1>{product?.title}</h1>;
  },
});
```

معیارهای انتخاب



سؤال‌های تصمیم‌گیری:

۱. تیم چه چیزی بلد است؟ فریم‌ورکی که تیم می‌شناسد، از «بهترین» فریم‌ورک بهتر است.
۲. کجا مستقر می‌شوید؟ اگر فقط هاست استاتیک دارید، SPA یا خروجی استاتیک.
۳. چقدر «جادو» می‌پذیرید؟ Next.js قواعد کش پیچیده‌تری دارد؛ RR7 صریح‌تر است.
۴. بازار کار محلی؟ برای استخدام در ایران، Next.js بیشترین تقاضا را دارد.

مهاجرت از Vite SPA به فریمورک

مهارت‌های این کتاب مستقیماً منتقل می‌شوند. آنچه تغییر می‌کند:

در فریمورک RSC	در SPA
<code>await</code> مستقیم در Server Component (+ Query برای داده کلاینتی پرتغییر)	<code>useSuspenseQuery</code> در کامپوننت
دسترسی مستقیم به DB در سرور، یا Server Function	<code>fetch</code> به API جدا
همان + API متادیتای فریمورک	<code><title></code> با Metadata React
پیش‌فرض سرور؛ <code>'use client'</code> برای تعامل	همه کامپوننت‌ها کلاینت
متغیرهای سرور (بدون پیشوند) + عمومی با پیشوند	<code>import.meta.env.VITE_*</code>
ساختار پوشه‌ها	روتینگ در <code>router.tsx</code>

گام‌های عملی: (۱) ابتدا با React Router v8 حالت Data کار کنید تا loader/action در دستتان بیاید؛ (۲) کامپوننت‌های نمایشی را از هوک‌های داده جدا کنید؛ (۳) در فریمورک، صفحه به صفحه مهاجرت کنید و از الگوی children برای ترکیب Server/Client استفاده کنید.

✦ یک فریمورک را عمیق یاد بگیرید، بقیه را بشناسید

مفاهیم (Actions، Streaming، RSC، کش) مشترک‌اند. با تسلط بر یکی، یادگیری بقیه چند روز طول می‌کشد. پراکندگی بین همه، تسلط بر هیچ‌کدام است.

“ سؤالات مصاحبه (با پاسخ) ”

Junior – چرا تیم React توصیه می‌کند از فریمورک استفاده کنیم؟ چون اپ واقعی به روتینگ، واکنشی داده، code splitting، SSR/SEO و بهینه‌سازی نیاز دارد و فریمورک این‌ها را یکپارچه و آزموده‌شده ارائه می‌دهد؛ سرهم‌کردن دستی آن‌ها زمان‌بر و خطاخیز است.

Mid – تفاوت اصلی React Router v8 Framework و Next.js در مدل داده چیست؟ RR8 روی loader/action و استانداردهای Request/Response بنا شده و کش را به HTTP و کتابخانه‌ها می‌سپارد؛ Next.js لایه کش چندسطحی خودش (Data Cache، Full Route Cache، `'use cache'`) را دارد که قدرتمندتر اما پیچیده‌تر است.

Senior – چه زمانی SPA خالص انتخاب معماری بهتری از فریمورک فول‌استک است؟ وقتی: تیم بک‌اند مستقل با API قراردادمحور دارد (مثلاً چند کلاینت موبایل/وب)، اپ کاملاً پشت احراز هویت است و SEO ندارد، زیرساخت فقط CDN استاتیک است (هزینه و پیچیدگی عملیاتی کمتر)، یا اپ بسیار تعاملی است (ویرایشگر، نقشه) که مزیت RSC ناچیز است. در این موارد، SPA سادگی، استقلال از vendor و قابلیت کش کامل در CDN می‌دهد و پیچیدگی مرز سرور/کلاینت را حذف می‌کند.

30 Build، استقرار و Web Vitals

کد خوب فقط نیمی از راه است؛ باید سریع، پایدار و قابل‌مشاهده به دست کاربر برسد. این فصل از تحلیل باندل شروع می‌کند، استقرار روی CDN و سرور خودی را پوشش می‌دهد و با معیارهای Core Web Vitals و پایش تمام می‌شود.

Build بهینه با Vite

Terminal

```
npm run build          # در dist/ با hash و code splitting خروجی در
npm run preview        # تست محلی خروجی
npx vite-bundle-visualizer  # نقشه باندل: چه چیزی چقدر جا گرفته
```

vite.config.ts

```
export default defineConfig({
  plugins: [react(), babel({ presets: [reactCompilerPreset()] })],
  build: {
    target: 'es2022',
    sourcemap: 'hidden', // در سرور عمومی سرو نشود Sentry برای
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom', 'react-router'],
          query: ['@tanstack/react-query'],
        },
      },
    },
  },
});
```

کاهش حجم باندل

تکنیک	اثر
Code splitting به‌ازای مسیر (lazy در روتر)	کاربر فقط JS صفحه فعلی را می‌گیرد
React.lazy برای کامپوننت‌های سنگین	نمودار، ویرایشگر، نقشه فقط در صورت نیاز
import انتخابی	نه <code>import { debounce } from 'lodash-es'</code> نه <code>import _ from 'lodash'</code>
جایگزینی وابستگی‌های سنگین	date-fns به‌جای moment ؛ Intl به‌جای هر دو
حذف polyfill‌های غیرضروری	target مدرن؛ browserslist واقع‌بینانه
تصاویر	WebP/AVIF ، loading="lazy" ، srcset ، CDN تصویر
فونت	فونت متغیر، subset فارسی، font-display: swap ، preload

```
import { lazy, Suspense } from 'react';
const Chart = lazy(() => import('./Chart')); // chunk جدا

<Suspense fallback=<ChartSkeleton />>
  <Chart data={data} />
</Suspense>
```

✦ بودجه کارایی

یک عدد تعیین کنید (مثلاً JS اولیه > ۲۰۰KB gzip) و در CI با size-limit یا bundlesize اجبار کنید. بدون بودجه، باندل هر ماه ۱۰٪ بزرگ‌تر می‌شود.

Core Web Vitals

معیار	چه چیزی را می‌سنجد	هدف	راهکار React
LCP (Largest Contentful Paint)	سرعت نمایش بزرگ‌ترین عنصر	$> 2.5s$	SSR/SSG، preload، hero تصویر، حذف render-blocking، code splitting
INP (Interaction to Next Paint)	تأخیر پاسخ به تعامل	$> 200ms$	<code>startTransition</code> ، شکستن کار سنگین، virtualization، React Compiler
CLS (Cumulative Layout Shift)	پرش‌های چیدمان	> 0.1	Skeleton، تصاویر، ابعاد صریح هم‌اندازه، <code>font-display</code> درست

اندازه‌گیری: Lighthouse (آزمایشگاهی)، Chrome UX Report و `web-vitals` (کاربران واقعی):

```
src/lib/vitals.ts

import { onCLS, onINP, onLCP } from 'web-vitals';

function send(metric: { name: string; value: number; id: string }) {
  navigator.sendBeacon('/api/vitals', JSON.stringify(metric));
}

onCLS(send); onINP(send); onLCP(send);
```

استقرار SPA روی CDN / هاست استاتیک

SPA فقط فایل استاتیک است؛ هر جایی قابل سرو است. یک **قاعده حیاتی**: همه مسیرها باید به `index.html` برگردند (SPA fallback) وگرنه رفرش روی `/products/42` خطای ۴۰۴ می‌دهد.

پلتفرم	پیکربندی fallback
Vercel	خودکار برای Vite؛ یا <code>vercel.json</code> با <code>rewrites</code>
Netlify	فایل <code>_redirects</code> : <code>/* /index.html 200</code>
Cloudflare Pages	خودکار (SPA mode)
GitHub Pages	ترفند <code>404.html</code> یا HashRouter
هاست اشتراکی (cPanel)	<code>.htaccess</code> با <code>RewriteRule</code>
Nginx / Docker	<code>try_files \$uri /index.html</code> (پایین)

Dockerfile

```
# مرحله ۱: build
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

# مرحله ۲: سرو استاتیک
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

پیکربندی Nginx سه کار دارد: کش تهاجمی برای فایل‌های hash دار، «هرگز کش نشود» برای `index.html` و fallback مسیرهای SPA:

nginx.conf (1/2)

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # دار: کش طولانی‌hash فایل‌های
    location /assets/ {
        add_header Cache-Control "public, max-age=31536000, immutable";
    }

    # index.html: هرگز کش نشود تا نسخه جدید فوراً دیده شود
    location = /index.html {
        add_header Cache-Control "no-cache";
    }

    # SPA fallback
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

و در ادامه‌ی همان بلوک `server` ، فشرده‌سازی و هدرهای امنیتی پایه (CSP کامل را در فصل ۲۶ دیدید):

nginx.conf (2/2)

```
# فشرده‌سازی
gzip on;
gzip_types text/css application/javascript application/json image/svg+xml;

# هدرهای امنیتی
add_header X-Content-Type-Options nosniff;
add_header Referrer-Policy strict-origin-when-cross-origin;
add_header X-Frame-Options DENY;
}
```

Terminal

```
docker build --build-arg VITE_API_URL=https://api.example.com -t my-app .
docker run -p 8080:80 my-app
```

⚠ متغیرهای محیطی در زمان build

در SPA، `VITE_*` هنگام **build** در کد جای‌گذاری می‌شود، نه در زمان اجرا. برای یک image با چند محیط (staging/prod)، یا برای هر محیط جداگانه build کنید، یا الگوی «config در runtime» را پیاده کنید: یک `config.js` که Nginx سرو می‌کند و در `index.html` قبل از اپ لود می‌شود.


```

.github/workflows/ci.yml

name: CI
on: [push, pull_request]
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm test -- --run --coverage
      - run: npm run build
      - name: E2E
        run: npx playwright install --with-deps chromium && npx playwright test
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-report, path: playwright-report }

```

خط لوله: E2E → build → unit/component → typecheck → lint. هر کدام شکست بخورد، merge نمی‌شود.

استراتژی کش و نسخه‌بندی

Vite نام فایل‌های `assets/` را با hash محتوا می‌سازد (`index-a1b2c3.js`). بنابراین:

◆ `Cache-Control: immutable, max-age=1y` → `assets/*` (تغییر محتوا = نام جدید)

◆ `no-cache` → `index.html` (همیشه تازه؛ به فایل‌های جدید اشاره می‌کند)

✖ باگ «chunk قدیمی» پس از deploy

کاربری که تب باز دارد، پس از deploy جدید روی لینکی کلیک می‌کند که chunk قدیمی (حذف‌شده) را lazy-load می‌کند → خطای `Failed to fetch dynamically imported module`. راه‌حل‌ها: (۱) نگاه‌داشتن چند نسخه قبلی `assets/` روی سرور؛ (۲) گوش‌دادن به رویداد `vite:preloadError` و reload صفحه؛ (۳) Error Boundary که در این خطا `location.reload()` می‌کند. ساده‌ترین نسخه‌ی راه‌حل (۲) در `main.tsx`:

```

window.addEventListener('vite:preloadError', () => window.location.reload());

```

لایه	ابزار
خطاها	source map با Sentry / Bugsnag
کارایی واقعی (RUM)	Vercel Analytics / SpeedCurve یا web-vitals → endpoint خودتان
رفتار کاربر	PostHog / Plausible (حریم خصوصی محور)
در دسترس بودن	UptimeRobot / Better Stack
لاگ سرور (اگر دارید)	Grafana Loki / Datadog

چک لیست پیش از انتشار

- ☐ `npm run build` بدون هشدار؛ حجم باندل داخل بودجه
- ☐ SPA fallback پیکربندی شده؛ رفرش روی مسیر عمیق کار می کند
- ☐ هدرهای کش درست (`immutable` برای `assets`، `no-cache` برای HTML)
- ☐ HTTPS، هدرهای امنیتی، CSP
- ☐ Error Boundary سراسری + Sentry با source map
- ☐ Lighthouse در Performance و Accessibility روی موبایل ≥ 90
- ☐ `<meta viewport>`، `manifest.json`، `favicon`، `robots.txt`
- ☐ متغیرهای محیطی Production جدا از dev؛ هیچ secret در باندل
- ☐ `vite:preloadError` handle شده
- ☐ تست E2E جریان های حیاتی سبز است

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا رفرش روی `/products/42` در SPA خطای ۴۰۴ می‌دهد و چطور حل می‌شود؟ چون سرور فایلی به آن نام ندارد؛ روتینگ در کلاینت انجام می‌شود. سرور باید همه مسیرهای ناشناخته را به `index.html` برگرداند تا React Router مسیر را مدیریت کند.

Mid – INP چیست و در React چطور بهبود می‌یابد؟ زمان از تعامل کاربر تا نقاشی فریم بعدی. راهکارها: به‌روزرسانی‌های سنگین را با `startTransition` غیرفوری کنید، کار طولانی را بشکنید، لیست‌های بزرگ را `virtualize` کنید، از React Compiler برای حذف رندهای اضافه استفاده کنید، و کار غیرضروری را از `event handler` به بعد از `paint` منتقل کنید.

Senior – استراتژی استقرار بدون downtime و بدون شکستن کاربران فعال برای SPA چیست؟ (۱) `assets` با `hash` و کش HTML، `immutable` بدون کش؛ (۲) نگه‌داشتن N نسخه قبلی `assets` تا کاربران با تب باز `chunk`ها را پیدا کنند؛ (۳) `handler` برای `vite:preloadError` و `reload` نرم؛ (۴) اعلان «نسخه جدید موجود است» با بررسی دوره‌ای `version.json`؛ (۵) استقرار `atomic` (پوشه جدید + تعویض `symlink` یا `deploy immutable` در CDN) تا حالت نیمه‌کاره وجود نداشته باشد؛ (۶) سازگاری API با نسخه قبلی کلاینت حداقل برای یک دوره.

31 مهاجرت و ارتقا به React 19

اکثر پروژه‌های موجود روی React 18 هستند. ارتقا به ۱۹ ساده‌تر از آن چیزی است که به نظر می‌رسد – اگر ترتیب درست را رعایت کنید. این فصل یک نقشه گام‌به‌گام با codemodهای رسمی و فهرست کامل تغییرات شکننده ارائه می‌دهد.

نقشه راه ارتقا



گام ۱: React 18.3 – پل ارتقا

Terminal

```
npm install react@18.3 react-dom@18.3
```

نسخه ۱۸.۳ از نظر عملکرد با ۱۸.۲ یکسان است اما برای هر API که در ۱۹ حذف می‌شود **هشدار** می‌دهد. اپ را اجرا کنید، کنسول را بخوانید و هشدارها را رفع کنید. این کار بیشتر ارتقا را بدون ریسک انجام می‌دهد.

گام ۲: codemodهای رسمی

Terminal

```
# با هم React 19 های codemod همه
npx codemod@latest react/19/migration-recipe

# یا جداگانه:
npx codemod@latest react/19/replace-reactdom-render # ReactDOM.render → createRoot
npx codemod@latest react/19/replace-string-ref      # string refs → callback
npx codemod@latest react/19/replace-act-import      # act از react
npx codemod@latest react/19/replace-use-form-state  # useFormState → useActionState
npx codemod@latest react/prop-types-typescript      # propTypes → TS
npx types-react-codemod@latest preset-19 ./src       # تایپ‌ها
```

گام ۳: نصب React 19

Terminal

```
npm install react@19 react-dom@19
npm install -D @types/react@19 @types/react-dom@19
npm install -D eslint-plugin-react-hooks@latest
```

بعد: `npx tsc --noEmit` و `npm run lint` – خطاهای باقی‌مانده را با جدول‌های زیر رفع کنید.

تغییرات شکننده – مرجع کامل

حذف‌شده‌ها

جایگزین	حذف‌شده
<code>hydrateRoot()</code> / <code>createRoot(el).render()</code>	<code>hydrate</code> / <code>ReactDOM.render</code>
<code>root.unmount()</code>	<code>ReactDOM.unmountComponentAtNode</code>
<code>ref</code> روی عنصر	<code>ReactDOM.findDOMNode</code>
TypeScript + مقدار پیش‌فرض در destructuring	<code>defaultProps</code> , <code>propTypes</code> (توابع)
<code>useRef</code> یا <code>ref callback</code>	String refs (<code>ref="input"</code>)
<code>createContext</code>	Legacy Context (<code>getChildContext</code> , <code>contextTypes</code>)
JSX	<code>React.createFactory</code>
Testing Library	<code>react-test-renderer/shallow</code>
<code>import { act } from 'react'</code>	<code>act</code> از <code>react-dom/test-utils</code>
تابع معمولی	Module pattern factories

◆ تغییر رفتار

تغییر	اثر و راهکار
خطا هنگام رندر <code>throw</code> می‌شود (نه <code>rethrow</code>)	یک لاگ به جای دو؛ از <code>onCaughtError</code> / <code>onUncaughtError</code> استفاده کنید
<code>ref</code> <code>cleanup function</code>	اگر <code>ref callback</code> مقداری برمی‌گرداند، حالا <code>cleanup</code> تلقی می‌شود؛ <code>ref={el => (this.x = el)}</code> را به بلاک تبدیل کنید
<code>useRef</code> آرگومان اجباری (تایپ)	<code>useRef<T>(null)</code>
<code>ReactElement.props</code> از <code>any</code> به <code>unknown</code>	با <code>isValidElement<Props>(el)</code> <code>narrow</code> کنید
UMD builds حذف شد	از ESM CDN (<code>esm.sh</code>) یا باندلر
حداقل ES2020	Chrome 80+، Safari 13.1؛ برای قدیمی‌تر <code>polyfill</code>
<code><Context.Provider></code> deprecated	<code><Context value></code> (هنوز کار می‌کند)
<code>forwardRef</code> deprecated	<code>ref</code> به عنوان <code>prop</code> (هنوز کار می‌کند)
<code>element.ref</code> حذف	<code>element.props.ref</code>
<code>StrictMode: useMemo/useCallback</code>	نتیجه اولین رندر در رندر دوم استفاده می‌شود (رفتار درست‌تر)

◆ تغییرات JSX و تایپ‌ها

قبل	بعد
<code>React.FC<Props></code> با <code>children</code> ضمنی	<code>children</code> را صریح در <code>Props</code> بنویسید
<code>JSX.Element</code> سراسری	<code>React.JSX.Element</code>
<code>MutableRefObject<T></code>	<code>RefObject<T></code> (<code>current</code> قابل نوشتن است)
<code>ReactChild</code> , <code>ReactFragment</code>	<code>ReactNode</code>
<code>React.VFC</code>	تابع معمولی

گام ۴: به‌روزرسانی وابستگی‌ها

کتابخانه‌هایی که با React 19 مشکل داشتند (و نسخه سازگارشان):

کتابخانه	وضعیت
@testing-library/react	$16 \leq$
react-router	v8 (v6.28+ و v7 هم با React 19 سازگارند)
@tanstack/react-query	v5
react-hook-form	$7.54 \leq$
motion → framer-motion	$11.11 \leq$
react-redux	$9.2 \leq$
@mui/material	$6.1 \leq$
antd	+v5.22 (با @ant-design/v5-patch-for-react-19)
react-helmet	حذف کنید؛ Metadata داخلی
styled-components	v6 سازگار؛ اما برای RSC مناسب نیست

Terminal

```
npm ls react          # می‌خواهند react وابستگی‌هایی که نسخه دیگری از
npm install --legacy-peer-deps # هنوز به‌روز نشده peerDependency موقت، اگر
```

گام ۵: React Compiler – تدریجی

Terminal

```
npm install -D --save-exact babel-plugin-react-compiler@latest
```

```
vite.config.ts

babel({
  presets: [reactCompilerPreset({
    // مرحله اول: فقط پوشه‌های انتخابی
    sources: (filename) => filename.includes('src/features/products'),
  })],
})
```

مسیر پذیرش: (۱) ابتدا فقط ESLint `react-hooks` نسخه +۶ را فعال و خطاها را رفع کنید؛ (۲) کامپایلر را روی یک فیچر فعال کنید و در DevTools نشان ✨ را بررسی کنید؛ (۳) گسترش تدریجی؛ (۴) `useMemo / memo` قدیمی را در ری‌فکتورهای بعدی حذف کنید (اجباری نیست). اگر کامپوننتی رفتار عجیب داشت، `"use no memo"` بالای آن بگذارید و بعداً بررسی کنید – تقریباً همیشه یک نقض قوانین React است.

چک‌لیست پس از ارتقا

- ☐ `test` , `lint` , `typecheck` , `npm run build` همه سبز
- ☐ کنسول در `dev` بدون هشدار `deprecation`
- ☐ فرم‌ها: `useFormState` → `useActionState` (آرگومان سوم `isPending` اضافه شد)
- ☐ `forwardRef` ها به تدریج به `prop ref`
- ☐ `react-helmet` حذف و با `<title>` / `<meta>` جایگزین
- ☐ اگر RSC دارید: روی نسخه وصله‌شده امنیتی ($\leq 19.2.4$)
- ☐ Error reporting با `onUncaughtError` / `onCaughtError` متصل

➤ از ۱۶ یا ۱۷ می‌آیید؟

ابتدا به ۱۸ بروید (`createRoot` , `StrictMode` , `automatic batching` دوگانه) و اپ را پایدار کنید. پرش مستقیم از ۱۶ به ۱۹ ممکن است ولی دیباگ هم‌زمان دو مجموعه تغییر، دشوار است. کامپوننت‌های کلاسی هنوز کار می‌کنند اما هوک‌های جدید (`use` , `Actions`) فقط در توابع در دسترس‌اند.

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا ابتدا به ۱۸.۳ برویم؟ چون همان رفتار ۱۸.۲ را دارد اما برای هر API حذف شده در ۱۹ هشدار می‌دهد؛ می‌توانید بدون ریسک، مشکلات را قبل از ارتقای واقعی پیدا و رفع کنید.

Mid – تغییر رفتار ref callback در React 19 چه باگی می‌سازد؟ `ref={el => (this.el = el)}` مقدار assignment را برمی‌گرداند؛ در ۱۹ این مقدار به‌عنوان cleanup function تفسیر می‌شود و اگر تابع نباشد هشدار/خطا می‌دهد. راه‌حل: بدنه را داخل آکولاد بنویسید تا `undefined` برگردد.

Senior – استراتژی ارتقای یک monorepo بزرگ با ده‌ها پکیج و کتابخانه داخلی چیست؟ (۱) `npm ls :inventory` و لیست API‌های deprecated با `codemod` در حالت `dry-run`؛ (۲) کتابخانه‌های داخلی را ابتدا با `peerDependency ^18 || ^19` سازگار کنید (بدون استفاده از API جدید)؛ (۳) اپ‌های برگ را یکی‌یکی ارتقا دهید – از کم‌ریسک‌ترین؛ (۴) در CI ماتریس تست روی هر دو نسخه تا مهاجرت کامل؛ (۵) پس از ارتقای همه، `peerDependency` را به `^19` محدود کنید و از قابلیت‌های جدید استفاده کنید؛ (۶) `React Compiler` را آخر و فیچر به فیچر.

32 انیمیشن، View Transitions و حس «نرم بودن»

انیمیشن خوب دیده نمی‌شود؛ فقط حس می‌شود. این فصل نشان می‌دهد چه چیزی را با CSS خالص حل کنید، کجا به کتابخانه‌ای مثل Motion نیاز دارید، چطور از View Transitions مرورگر برای جابه‌جایی صفحه‌ها استفاده کنید و `<ViewTransition>` آزمایشی React چه آینده‌ای را نوید می‌دهد – همه با احترام به کاربرانی که حرکت کمتر می‌خواهند.

اصول: انیمیشن باید معنا داشته باشد

انیمیشن سه کار مفید انجام می‌دهد: **جهت‌دهی** (از کجا آمدیم، به کجا رفتیم)، **بازخورد** (کلیک شد، ذخیره شد) و **تداوم** (این کارت همان کارتی است که در لیست بود). هر انیمیشنی که یکی از این سه کار را نکند، فقط تأخیر است.

- ♦ مدت ۱۵۰ تا ۳۰۰ میلی‌ثانیه برای UI معمولی؛ بیش از ۵۰۰ms فقط برای انتقال صفحه یا داستان‌سرایی
- ♦ فقط `transform` و `opacity` را انیمیت کنید – روی compositor اجرا می‌شوند و layout را دوباره محاسبه نمی‌کنند. انیمیت‌کردن `height`، `top` یا `margin` باعث jank می‌شود
- ♦ easing طبیعی: ورود با `ease-out`، خروج با `ease-in`، جابه‌جایی با `ease-in-out`
- ♦ همیشه `prefers-reduced-motion` را رعایت کنید

i قاعده سرانگشتی

اگر انیمیشن را حذف کنید و کسی متوجه نشود، حذفش کنید. اگر حذفش کنید و کاربر گیج شود («این از کجا آمد؟»)، انیمیشن درست بوده است.

سطح ۱: CSS خالص – ۸۰٪ نیازها

React برای بیشتر انیمیشن‌ها به هیچ کتابخانه‌ای نیاز ندارد؛ کافی است `state` را به `class` یا `data-attribute` تبدیل کنید و بقیه را به CSS بسپارید.

src/components/Toast.tsx

```
import type { ReactNode } from 'react';

export function Toast({ open, children }: { open: boolean; children: ReactNode }) {
  return (
    <div className="toast" data-open={open} role="status">
      {children}
    </div>
  );
}
```

src/components/toast.css

```
.toast {
  transform: translateY(16px);
  opacity: 0;
  transition: transform 220ms cubic-bezier(0.2, 0.8, 0.2, 1), opacity 220ms ease-out;
}
.toast[data-open="true"] { transform: none; opacity: 1; }

@media (prefers-reduced-motion: reduce) {
  .toast { transition-duration: 1ms; }
}
```

با Tailwind v4 همین کار یک خط است: `transition-all duration-200 data-[open=true]:translate-y-0 data-[open=false]:translate-y-4 motion-reduce:transition-none`

♦ ورود عناصر با `@starting-style`

مشکل کلاسیک: عنصری که تازه mount شده، transition ورودی ندارد چون «حالت قبلی» نداشته. CSS مدرن این را با `@starting-style` حل می‌کند – بدون هیچ JS:

src/styles/enter.css

```
.card {
  opacity: 1; transform: none;
  transition: opacity 200ms, transform 200ms;

  @starting-style {
    opacity: 0; transform: scale(0.96);
  }
}
```

هر بار React یک `.card` جدید به DOM اضافه کند، از حالت `@starting-style` به حالت عادی انیمیت می‌شود. برای خروج هنوز CSS خالص کافی نیست (عنصر قبل از انیمیشن از DOM حذف می‌شود) – این‌جاست که به سطح ۲ می‌رسیم.

```
dialog {
  opacity: 0;
  transition:
    opacity 200ms, display 200ms allow-discrete, overlay 200ms allow-discrete;
}
dialog[open] { opacity: 1; @starting-style { opacity: 0; } }
```

با این دو ویژگی، حتی باز/بسته شدن `<dialog>` و Popover بومی هم بدون کتابخانه انیمیت می‌شود.

سطح ۲: Motion – وقتی به خروج، layout و ژست نیاز دارید

کتابخانه **Motion** (نام جدید Framer Motion؛ ایمپورت از `motion/react`، نصب با `npm install motion`) سه چیز را می‌دهد که CSS نمی‌دهد: **انیمیشن خروج** (`AnimatePresence`)، **انیمیشن layout** خودکار (FLIP) و **ژست‌ها** (drag، hover، tap). حجم آن حدود ۳۰KB است؛ فقط وقتی به این سه نیاز دارید نصبش کنید. نمونه‌ی کلاسیک – Modal با انیمیشن خروج:

```
src/components/Modal.tsx

import { AnimatePresence, motion } from 'motion/react';

const pop = {
  initial: { opacity: 0, y: 24, scale: 0.98 },
  animate: { opacity: 1, y: 0, scale: 1 },
  exit: { opacity: 0, y: 12, scale: 0.98 },
  transition: { type: 'spring', stiffness: 400, damping: 32 },
};

export function Modal({ open, onClose, children }: ModalProps) {
  return (
    <AnimatePresence>
      {open && (
        <motion.div role="dialog" aria-modal="true" className="modal" {...pop}>
          {children}
          <button onClick={onClose}>بستن</button>
        </motion.div>
      )}
    </AnimatePresence>
  );
}
```

`AnimatePresence` فرزندی را که از درخت حذف شده، تا پایان انیمیشن `exit` در DOM نگه می‌دارد – همان چیزی که با CSS خالص ممکن نبود. شرط کارکرد: فرزند مستقیم باید `key` پایدار داشته باشد.

◆ انیمیشن layout: مرتب‌سازی لیست بدون محاسبه

```
src/features/tasks/TaskList.tsx

import { motion, AnimatePresence } from 'motion/react';

export function TaskList({ tasks }: { tasks: Task[] }) {
  return (
    <ul>
      <AnimatePresence initial={false}>
        {tasks.map((t) => (
          <motion.li
            key={t.id}
            layout // فیلتر/reorder هنگام نرم جایی‌جابه
            initial={{ opacity: 0, height: 0 }}
            animate={{ opacity: 1, height: 'auto' }}
            exit={{ opacity: 0, height: 0 }}
            transition={{ duration: 0.2 }}
          >
            {t.title}
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

`layout` prop تکنیک FLIP را خودکار می‌کند: موقعیت قبل و بعد از رندر را اندازه می‌گیرد و تفاوت را با `transform` انیمیت می‌کند. با `layoutId` مشترک بین دو کامپوننت (مثلاً کارت لیست و کارت جزئیات) به `shared element` `transition` می‌رسید.

◆ React Compiler و Motion

Motion با React 19 و React Compiler سازگار است. فقط تابع‌هایی که به `onAnimationComplete` می‌دهید را مثل هر handler دیگری بنویسید — Compiler خودش `memoize` می‌کند. اگر با انیمیشن مبتنی بر `useMotionValue` کار می‌کنید، مقدار را در رندر نخوانید؛ آن را به `style` بدهید تا خارج از چرخه رندر React به‌روز شود (بدون رندر مجدد در هر فریم).

```
src/app/providers.tsx

import { MotionConfig } from 'motion/react';

<MotionConfig reducedMotion="user">
  <App />
</MotionConfig>
```

با `reducedMotion="user"` هر انیمیشن `transform` برای کاربرانی که در سیستم عامل «کاهش حرکت» را فعال کرده اند غیرفعال می شود؛ `opacity` باقی می ماند تا تغییر همچنان قابل درک باشد.

سطح ۳: View Transitions API مرورگر

برای انتقال بین صفحه ها (یا هر تغییر بزرگ DOM)، مرورگر یک API بومی دارد: `document.startViewTransition(callback)`. مرورگر از حالت فعلی اسکرین شات می گیرد، شما DOM را تغییر می دهد، و بین دو تصویر cross-fade (یا هر انیمیشن CSS دلخواه) اجرا می کند. پشتیبانی: Chrome / Edge 111+، Safari 18+، Firefox 132 – یعنی در ۲۰۲۶ همه ی مرورگرهای اصلی برای انتقال درون سندی.

♦ با React Router

React Router از v6.4 به بعد این را داخلی پشتیبانی می کند؛ کافی است prop بدهید:

```
src/components/ProductCard.tsx

import { Link } from 'react-router';

<Link to={`\products/${p.id}`} viewTransition>
  <img src={p.image} alt="" style={{ viewTransitionName: `product-${p.id}` }} />
  <h3>{p.title}</h3>
</Link>
```

src/routes/product.tsx

```
import { useLoaderData } from 'react-router';

export default function Product() {
  const p = useLoaderData() as ProductData; // فصل ۱۶ همان الگوی
  return (
    <article>
      <img
        src={p.image} alt={p.title}
        style={{ viewTransitionName: `product-${p.id}` }}
      />
      { /* ... */ }
    </article>
  );
}
```

چون تصویر در هر دو صفحه یک `view-transition-name` دارد، مرورگر آن را از موقعیت کوچک در لیست به موقعیت بزرگ در صفحه جزئیات **morph** می‌کند — اثر «Hero» بدون یک خط محاسبه. شرط مهم: در هر لحظه فقط یک عنصر در صفحه می‌تواند نام مشخصی داشته باشد (برای همین `product-${id}` است، نه `product`).

◆ سفارشی‌سازی با CSS

src/styles/transitions.css

```
::view-transition-old(root) { animation: 180ms ease-in both fade-out; }
::view-transition-new(root) {
  animation: 220ms ease-out both fade-in, 220ms ease-out both slide-up;
}

@keyframes fade-out { to { opacity: 0; } }
@keyframes fade-in { from { opacity: 0; } }
@keyframes slide-up { from { transform: translateY(12px); } }

@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*),
  ::view-transition-old(*),
  ::view-transition-new(*) { animation: none !important; }
}
```

```
src/hooks/useViewTransition.ts

import { flushSync } from 'react-dom';

export function useViewTransition() {
  return (update: () => void) => {
    const reduce = matchMedia('(prefers-reduced-motion: reduce)').matches;
    if (!document.startViewTransition || reduce) {
      update();
      return;
    }
    document.startViewTransition(() => flushSync(update));
  };
}
```

`flushSync` لازم است تا React تغییر state را **همزمان** داخل `callback commit` کند؛ در غیر این صورت مرورگر قبل از به‌روزرسانی DOM اسکرین‌شات «بعد» را می‌گیرد. مثلاً برای تغییر تم: `withTransition(() => .setTheme('dark'))`

⚠️ آبخار انیمیشن و INP

`startViewTransition` کل صفحه را تا پایان انیمیشن **غیرقابل‌تعامل** می‌کند. انیمیشن‌های بلند (بیش از ۳۰۰ms) مستقیماً INP را بدتر می‌کنند. برای فهرست‌های طولانی یا صفحات سنگین، `view-transition-name: none` را روی بخش‌های بی‌ربط بگذارید تا اسکرین‌شات سبک‌تر شود.

آینده: <ViewTransition> در React (آزمایشی)

تیم React در حال ساخت یک کامپوننت بومی است که همین API را با مدل Transition خودش یکپارچه می‌کند. وضعیت در زمان نگارش: فقط در کانال‌های `canary` و `experimental`؛ در React 19.2 پایدار وجود ندارد و ایمپورت آن خطا می‌دهد. Next.js با فلگ `experimental.viewTransition` آن را در دسترس می‌گذارد (اما «توصیه‌نشده برای production»).

```
app/Gallery.tsx (React canary)

// ⚠️ نمی‌شود export در 19.2 پایدار - react@canary فقط در
import { ViewTransition, startTransition, addTransitionType } from 'react';

function Gallery({ photos, onOpen }: Props) {
  return (
    <ViewTransition default="fade" enter="slide-in" exit="slide-out">
      <ul>
        {photos.map((p) => (
          <ViewTransition key={p.id} name={`photo-${p.id}`}>
            <li>
              onClick={() => {
                startTransition(() => {
                  addTransitionType('open-photo');
                  onOpen(p.id);
                })
              }
            </li>
          </ViewTransition>
        ))}
      </ul>
    </ViewTransition>
  );
}
```

ایده‌ها:

- ◆ `<ViewTransition>` چه چیزی انیمیت شود را مشخص می‌کند؛ props مثل `enter` / `exit` / `update` `share` نام کلاس CSS می‌گیرند
- ◆ انیمیشن فقط برای به‌روزرسانی‌هایی فعال می‌شود که داخل `startTransition` (یا `Suspense reveal`) یا `navigation` (روتر) اتفاق بیفتند – به‌روزرسانی‌های همگام عمداً انیمیت نمی‌شوند
- ◆ `addTransitionType('nav-back')` علت انتقال را مشخص می‌کند تا مثلاً «برگشت» جهت مخالف «رفتن» انیمیت شود

- ◆ کامپوننت باید بالای اولین گره DOM خروجی بنشیند؛ اگر یک `<div>` بین `<ViewTransition>` و محتوا باشد، انیمیشن `enter/exit` کار نمی‌کند

✗ در production منتظر بمانید

API نهایی ممکن است تغییر کند. برای اپ‌های واقعی امروز از `viewTransition` React Router یا `useViewTransition` بالا استفاده کنید؛ وقتی `<ViewTransition>` پایدار شد، مهاجرت کوچک است چون همان `view-transition-name` ها و همان CSS کار می‌کنند.

کدام ابزار برای کدام کار؟

نیاز	ابزار	چرا
hover، focus، تغییر state ساده	CSS transition + data-attribute	صفر KB، روی compositor
ورود عنصر تازه	@starting-style	بدون JS، بدون کتابخانه
باز/بسته شدن Popover/ <dialog>	transition-behavior: allow-discrete	بومی و دسترسی‌پذیر
خروج عنصر، لیست قابل مرتب‌سازی، drag	Motion (layout , AnimatePresence)	تنها راه تمیز برای exit و FLIP
انتقال بین صفحه‌ها، Hero image	View Transitions (viewTransition روتر)	بومی، بدون محاسبه، cross-fade رایگان
اسکرول محور (parallax)، (reveal)	CSS animation-timeline: scroll() یا Motion whileInView	CSS بومی در Chrome / Edge؛ Motion برای همه
میکرو-انیمیشن SVG / عدد	Motion animate() یا WAAP	کنترل فریم‌به‌فریم

دسترسی‌پذیری و انیمیشن

- ◆ `prefers-reduced-motion` را در سه سطح رعایت کنید: CSS (`@media`)، Motion (`MotionConfig`)، View Transitions (شرط در هوک)
- ◆ انیمیشن نباید تنها راه انتقال اطلاعات باشد؛ «ذخیره شد» را با متن (و `role="status"`) هم بگویید، نه فقط با چشمک سبز
- ◆ فوکوس را بعد از انتقال صفحه مدیریت کنید (فصل ۲۵)؛ انیمیشن زیبا با فوکوس گم‌شده، برای کاربر کیبورد فاجعه است

♦ از انیمیشن‌های چشم‌کزن با فرکانس بالا (بیش از ۳ بار در ثانیه) پرهیز کنید – معیار WCAG 2.3.1

“ سؤالات مصاحبه (با پاسخ)

Junior – چرا انیمیت‌کردن height یا top باعث lag می‌شود؟ چون این ویژگی‌ها layout را تغییر می‌دهند و مرورگر باید در هر فریم موقعیت همه‌ی عناصر را دوباره محاسبه کند (reflow). `transform` و `opacity` روی لایه‌ی compositor اجرا می‌شوند و به layout دست نمی‌زنند؛ برای همین ۶۰fps پایدار می‌مانند.

Mid – `AnimatePresence` چطور انیمیشن خروج را ممکن می‌کند در حالی که `React` عنصر را از درخت حذف کرده؟ `AnimatePresence` فرزندان قبلی خود را (بر اساس `key`) به خاطر می‌سپارد؛ وقتی فرزندی در رندر جدید غایب است، آن را همچنان رندر می‌کند و انیمیشن `exit` را اجرا می‌کند و فقط پس از پایان، واقعاً حذف می‌کند. به همین دلیل `key` پایدار ضروری است.

Senior – تفاوت مدل `<ViewTransition>` آزمایشی `React` با فراخوانی مستقیم `document.startViewTransition` چیست؟ فراخوانی مستقیم به `flushSync` نیاز دارد و با `Suspense`، رندر هم‌روند و به‌روزرسانی‌های قطع‌شده هماهنگ نیست. `React` انیمیشن را فقط برای `Transition`‌ها (`startTransition`، `Suspense reveal`، `navigation`) فعال می‌کند، `snapshot` را دقیقاً در لحظه‌ی `commit` می‌گیرد، چند به‌روزرسانی هم‌زمان را در یک `view transition` ادغام می‌کند و با `addTransitionType` امکان انیمیشن متفاوت بر اساس علت را می‌دهد – بدون این‌که بلوک همگام به رندر تزریق کنید.

≡ جمع‌بندی فصل

- ♦ CSS اول: `transition` + `data-attribute`، `@starting-style` برای ورود، `allow-discrete` برای dialog
- ♦ Motion فقط برای خروج، layout/FLIP و ژست‌ها؛ با `MotionConfig reducedMotion="user"`
- ♦ View Transitions مرورگر برای انتقال صفحه؛ در React Router با `prop viewTransition` و `view-transition-name` یکتا
- ♦ `<ViewTransition>` React هنوز canary است – API را بشناسید، در production صبر کنید
- ♦ `prefers-reduced-motion` و مدیریت فوکوس اختیاری نیستند

📌 تمرین

۱. به پروژه فصل ۱۸ (مدیریت وظایف) انیمیشن ورود/خروج آیت‌ها با `AnimatePresence` و `layout` اضافه کنید؛ سپس همان را فقط با `@starting-style` + `transition` (بدون خروج) پیاده کنید و تفاوت حجم باندل را با `npx vite-bundle-visualizer` مقایسه کنید. ۲. در پروژه روتینگ فصل ۱۶، انتقال لیست → جزئیات را با `viewTransition` و `view-transition-name` روی تصویر بسازید و با `prefers-reduced-motion` تست کنید.

33 نکات و ترفندهای طلایی

این فصل مجموعه‌ای فشرده از درس‌هایی است که معمولاً فقط با ساعت‌ها دیباگ و بازبینی کد یاد گرفته می‌شوند. هر نکته را با «چرا» بخوانید، نه فقط «چه».

| مدل ذهنی و state

۱. **key** را برای ریست **state** استفاده کنید، نه **Effect**.

```
// ✗ useEffect(() => setDraft(''), [userId]);  
// ✓  
<ProfileEditor key={userId} user={user} />
```

۲. **state** مشتق‌شده را ذخیره نکنید؛ محاسبه کنید. اگر **total** از **items** به‌دست می‌آید، در رندر بنویسید **const** `total = items.reduce(...)` حتی **useMemo** هم لازم نیست.

۳. **state** را تا حد ممکن پایین نگه دارید. **state** جستجو در **SearchBox** باشد نه در **App**. هر سطحی که بالا می‌رود، کامپوننت‌های بیشتری رندر می‌شوند.

۴. برای حالت‌های چندگانه از **union** استفاده کنید نه چند **boolean**.

```
// ✗ isLoading, isError, isSuccess (ترکیب، ۵ تا غیرممکن)  
// ✓ status: 'idle' | 'loading' | 'error' | 'success'
```

۵. برای هر «سیستم خارجی» یک **هوک** بسازید. **useMediaQuery**، **useGeolocation**، **useWebSocket** — کامپوننت نباید بداند چطور **subscribe** می‌شود.

| Effect و داده

۶. اگر **Effect** فقط **setState** می‌کند، احتمالاً اشتباه است. یا مقدار مشتق‌شده است (محاسبه در رندر) یا واکنش به رویداد است (در **handler**).

۷. **Race condition** را با **AbortController** یا **ignore flag** حل کنید — یا بهتر: از کتابخانه داده استفاده کنید که این را داخلی حل کرده.

۸. **queryKey** را مثل **URL** طراحی کنید: `['products', { category, page }]`. سلسله‌مراتبی، قابل **invalidate** جزئی: `invalidateQueries({ queryKey: ['products'] })` همه صفحات را باطل می‌کند.

۹. **staleTime** را تنظیم کنید. پیش‌فرض **TanStack Query** صفر است؛ یعنی هر **mount** یک **refetch**. برای داده‌های نسبتاً ثابت `staleTime: 5 * 60_000`.

۱۰. Promise را قبل از رندر بسازید. برای `use()` در loader، والد، یا با `useMemo` — هرگز مستقیماً در بدنه رندر.

کامپوننت و الگو

۱۱. کامپوننت داخل کامپوننت تعریف نکنید. هر رندر والد = نوع جدید = unmount/mount کامل فرزند = از دست رفتن state و کارایی.

۱۲. `children` را به `props` پیکربندی ترجیح دهید. `<Card><CardHeader/>...</Card>` انعطاف‌پذیرتر از `<Card>` `headerTitle` `headerIcon` `headerAction` `/>`

۱۳. Boolean prop را پیش‌فرض `false` طراحی کنید. `<Button disabled>` طبیعی است؛ `<Button>` `enabled={false}` نه.

۱۴. از `ComponentProps<'button'>` به ارث ببرید تا کامپوننت سفارشی همه ویژگی‌های بومی (`data`، `*-aria`، `onFocus`) را بدون تعریف دستی بپذیرد.

۱۵. Early return برای حالت‌های استثنا. اول `if (!user) return <Login/>`، بعد مسیر اصلی. تودرتویی کمتر، خوانایی بیشتر.

کارایی

۱۶. ابتدا Profile کنید، بعد بهینه کنید. ۹۰٪ «مشکلات کارایی» که توسعه‌دهندگان حدس می‌زنند، در Profiler وجود ندارند.

۱۷. رندر مجدد به‌خودی‌خود بد نیست. رندری که DOM را تغییر نمی‌دهد، ارزان است. مشکل، رندر گران و مکرر است.

۱۸. Context را بر اساس فرکانس تغییر تقسیم کنید. `AuthContext` (به‌ندرت) جدا از `NotificationContext` (مکرر).

۱۹. لیست بالای ۱۰۰۰ آیتم با DOM سنگین را `virtualize` کنید. `@tanstack/react-virtual` چند خط است.

۲۰. ورودی کاربر همیشه فوری، بقیه در `transition`. `setQuery(v)` بیرون، `startTransition(() =>` `setResults(...))` داخل.

TypeScript

۲۱. `as` را حداقل کنید. هر `as` یک دروغ بالقوه به کامپایلر است. به‌جای `data as User`، با `Zod parse` کنید یا `type guard` بنویسید.

۲۲. `satisfies` برای اشیاء پیکربندی. نوع را بررسی می‌کند بدون این‌که literal types را از دست بدهید.

۲۳. `noUncheckedIndexedAccess` را فعال کنید. `arr[0]` می‌شود `T | undefined` — همان‌طور که واقعاً هست. صدها باگ runtime را جلوگیری می‌کند.

۲۴. پیام خطای React را کامل بخوانید. React 19 پیام‌های بسیار دقیقی دارد، اغلب با لینک به راه‌حل. «Cannot update a component while rendering a different component» دقیقاً می‌گوید کجا.

۲۵. «React DevTools → Highlight updates when components render». در چند ثانیه می‌بینید کدام بخش UI بی‌دلیل چشمک می‌زند.

ترفند: لاگ رندر با نام کامپوننت

```
function useRenderCount(name: string) {
  const count = useRef(0);
  count.current++;
  if (import.meta.env.DEV) console.debug(`[render] ${name} #${count.current}`);
}
```

در بررسی کارایی موقتاً به کامپوننت مشکوک اضافه کنید.

قطعه‌کدهای پرکاربرد

الگوهای کوچک اما پرتکرار

```
// ۱) تبدیل اعداد فارسی ورودی به انگلیسی قبل از اعتبارسنجی
const toEnDigits = (s: string) =>
  s.replace(/[۰-۹]/g, (d) => String('۰۱۲۳۴۵۶۷۸۹'.indexOf(d)));

// ۲) فقط اگر چاره‌ای نیست id کلید ترکیبی پایدار برای آیتم بدون
const key = `${item.category}-${item.slug}`;

// ۳) رندر شرطی چندحالتی بدون تودرتویی
const views = {
  idle: <Idle />, loading: <Spinner />, error: <Err />, success: <Data />,
} as const;
return views[status];

// ۴) با پیام واضح env دسترسی امن به
const env = (k: keyof ImportMetaEnv) => {
  const v = import.meta.env[k];
  if (!v) throw new Error(`متغیر محیطی ${k} تنظیم نشده`);
  return v;
};

// ۵) در SSR mismatch بدون) های مرورگر API برای «mount هوک» فقط بعد از
const useMounted = () =>
  useSyncExternalStore(() => () => {}, () => true, () => false);
```

≡ جمع‌بندی فصل

اگر فقط پنج چیز را به خاطر بسپارید: (۱) UI تابعی از state است و رندر یعنی اجرای مجدد تابع؛ (۲) Effect برای همگام‌سازی با بیرون است، نه برای محاسبه؛ (۳) داده سرور را به کتابخانه بسپارید؛ (۴) ترکیب (children) قبل از Context، Context قبل از store؛ (۵) اول اندازه بگیرید، بعد بهینه کنید – و بگذارید React Compiler کار خودش را بکند.

34 کارگاه مینی پروژه‌ها

دانستن کافی نیست؛ باید ساخت. این هشت پروژه طوری طراحی شده‌اند که هر کدام چند مفهوم کتاب را ترکیب کند و در ۲ تا ۶ ساعت قابل انجام باشد. برای هر پروژه: هدف، مفاهیم، مشخصات، نکات پیاده‌سازی و «چالش اضافه» آمده است.

۱. شمارنده پیشرفته و Undo/Redo

مفاهیم: `useReducer`، نا تغییرپذیری، الگوی تاریخچه، `TypeScript union`.

مشخصات: شمارنده با افزایش/کاهش/ریست، گام قابل تنظیم، دکمه‌های Undo و Redo (Ctrl+Z / Ctrl+Shift+Z)، نمایش تاریخچه ۱۰ عمل آخر.

```
type HistoryState<T> = { past: T[]; present: T; future: T[] };
// undo: present → future, past.pop() → present
// redo: present → past, future.shift() → present
```

نکته: reducer را جنریک بنویسید (`historyReducer<T>`) تا در پروژه ۶ دوباره استفاده کنید. **چالش:** ذخیره تاریخچه در `sessionStorage` و بازیابی پس از رفرش.

۲. جستجوی زنده با debounce و transition

مفاهیم: `useTransition`، `useDeferredValue`، `AbortController`، `Suspense`، `TanStack Query`.

مشخصات: input جستجو روی API `https://dummyjson.com/products/search?q=`؛ نتایج با تأخیر `debounce ۳۰۰ms`؛ نمایش «در حال جستجو...» بدون پرش layout؛ تاریخچه ۵ جستجوی اخیر در `localStorage`؛ حالت خالی و خطا.

نکته: input همیشه فوری باشد — مقدار `deferred` را به `query` بدهید. **چالش:** highlight کردن عبارت جستجو در نتایج بدون `dangerouslySetInnerHTML` (با `split` و `<mark>`).

۳. فرم چند مرحله‌ای ثبت نام

مفاهیم: `useReducer`، `Zod`، `useActionState` برای wizard، حفظ state در URL.

مشخصات: سه مرحله (اطلاعات شخصی → آدرس → بازبینی)؛ اعتبارسنجی هر مرحله با schema جدا (`.pick`)؛ نمایش خطای هر فیلد زیر آن؛ مرحله فعلی در `step=2`؟ تا رفرش آن را حفظ کند؛ دکمه «ویرایش» در بازبینی که به مرحله مربوطه برمی‌گردد.

نکته: اعداد فارسی موبایل را قبل از regex به انگلیسی تبدیل کنید. **چالش:** ذخیره خودکار پیش‌نویس هر ۲ ثانیه با نشانگر «ذخیره شد ✓».

۴. کانبان با Drag & Drop و Optimistic UI

مفاهیم: `useOptimistic` ، `@dnd-kit` ، Zustand ، الگوی normalize شده.

مشخصات: سه ستون (To Do / Doing / Done)؛ کشیدن کارت بین ستون‌ها و تغییر ترتیب داخل ستون؛ API شبیه‌سازی‌شده با ۵۰۰ms تأخیر و ۱۰٪ خطا؛ حرکت کارت فوراً دیده شود و در خطا برگردد؛ افزودن/حذف کارت.

```
type Board = {
  columns: Record<ColumnId, { id: ColumnId; title: string; cardIds: string[] }>;
  cards: Record<string, Card>;
};
```

نکته: state را normalize کنید؛ جابه‌جایی فقط `cardIds` دو ستون را تغییر می‌دهد. **چالش:** پشتیبانی کیبورد برای drag (dnd-kit دارد) و اعلام حرکت با `aria-live`.

۵. داشبورد با نمودار، فیلتر URL و Suspense تودرتو

مفاهیم: React Router loader ، `<Await>` ، Suspense چندلایه ، `lazy` ، search params ، `<Activity>`.

مشخصات: صفحه با ۴ ویجت (KPIها، نمودار فروش، جدول سفارش‌ها، فید فعالیت)؛ هر ویجت Suspense و ErrorBoundary خودش؛ فیلتر بازه زمانی در URL؛ نمودار (Recharts) lazy-load شود؛ تب «گزارش‌ها» با `<Activity>` پنهان شود تا state فیلترهایش حفظ بماند.

نکته: loader فقط Promiseها را برگرداند (بدون await) تا shell فوراً رندر شود. **چالش:** export جدول به CSV با Blob و بررسی INP در Chrome Performance.

۶. ویرایشگر Markdown با پیش‌نمایش زنده

مفاهیم: `useDeferredValue` ، Web Worker ، `dangerouslySetInnerHTML` امن با `DOMPurify` ، `<dialog>` ، Undo/Redo از پروژه ۱.

مشخصات: دو ستون (متن / پیش‌نمایش)؛ پارس Markdown با `marked` در Web Worker تا تایپ گیر نکند؛ پاک‌سازی HTML با `DOMPurify`؛ نوار ابزار (بولد، عنوان، لینک) با حفظ موقعیت cursor؛ ذخیره در IndexedDB modal تنظیمات با `( idb-keyval )` . `<dialog>`

نکته: پیش‌نمایش را از `deferredText` بسازید؛ textarea از `text` . **چالش:** پشتیبانی RTL خودکار برای هر پاراگراف با `dir="auto"` و شمارش کلمات فارسی صحیح.

۷. چت بلادرنگ با WebSocket

مفاهیم: `useEffect` + cleanup ، `useEffectEvent` ، `useSyncExternalStore` ، `useOptimistic` ، virtualization ، `useLayoutEffect` برای اسکرول.

مشخصات: اتصال به سرور echo WebSocket عمومی `wss://echo.websocket.org` یا سرور کوچک Node خودتان؛ نمایش وضعیت اتصال؛ ارسال پیام با Optimistic UI و نشانگر «در حال ارسال/ارسال‌شده/ناموفق»؛ reconnect خودکار با backoff؛ اسکروول خودکار به پایین فقط اگر کاربر پایین است؛ لیست virtualize شده برای ۱۰۰۰+ پیام.

نکته: «نمایش notification با تم فعلی هنگام اتصال» را با `useEffectEvent` بنویسید تا تغییر تم reconnect سازد. **چالش:** نشانگر «در حال تایپ...» با throttle و اعلان مرورگر برای پیام جدید در تب غیرفعال.

۸. فروشگاه کامل (پروژه پایانی)

مفاهیم: همه‌چیز: معماری React Router v8، feature-based، Zustand، TanStack Query، loader/action، Actions برای فرم‌ها، Tailwind + shadcn، تست با Docker، Vitest/MSW/Playwright.

مشخصات:

- ♦ صفحات: خانه، لیست محصولات (فیلتر/مرتب‌سازی/صفحه‌بندی در URL)، جزئیات محصول، سبد، پرداخت (فرم چندمرحله‌ای)، ورود/ثبت‌نام، پروفایل و سفارش‌ها (محافظت‌شده).
 - ♦ API: `https://dummyjson.com` (محصولات، auth، سبد) یا `json-server` محلی.
 - ♦ سبد در Zustand با `persist`؛ افزودن با Optimistic badge.
 - ♦ Metadata هر صفحه با `<title>` / `<meta>` React 19.
 - ♦ ErrorBoundary سه‌لایه، Skeleton برای هر بخش، `<Activity>` برای تب‌های پروفایل.
 - ♦ تست: reducer سبد (unit)، فرم پرداخت (component + MSW)، جریان خرید (Playwright).
 - ♦ Docker + Nginx با SPA fallback و هدرهای کش؛ CI با GitHub Actions.
- معیار پذیرش:** Lighthouse موبایل ≤ 90 در Performance و Accessibility؛ JS اولیه $> 200\text{KB}$ gzip؛ همه تست‌ها سبز.

چالش: مهاجرت همین پروژه به React Router v8 حالت Framework و مقایسه LCP قبل و بعد.

نحوه ارائه پروژه‌ها (برای رزومه)

مورد	توضیح
README قوی	اسکرین‌شات/GIF، لینک دمو، تصمیم‌های معماری، نحوه اجرا
commit‌های معنادار	<code>feat(cart): optimistic add with rollback</code> نه <code>fix stuff</code>
دمو زنده	Vercel/Netlify/Cloudflare Pages رایگان
تست و CI badge	نشان می‌دهد حرفه‌ای کار می‌کنید
یک «تصمیم سخت» را مستند کنید	«چرا Zustand به جای Context» – مصاحبه‌کننده‌ها عاشق این‌اند

♦ ترتیب پیشنهادی

۱ → ۲ → ۳ → ۵ → ۴ → ۶ → ۷ → ۸. اگر وقت کم دارید، ۲، ۳، ۴ و ۸ بیشترین پوشش را می‌دهند. پروژه ۸ به‌تنهایی می‌تواند نمونه‌کار اصلی رزومه شما باشد.

35 بهترین شیوه‌ها، اشتباهات رایج و منابع

این فصل جمع‌بندی کل کتاب در قالب یک مرجع سریع است: اشتباهاتی که در بازبینی کد بارها می‌بینیم، شیوه‌های درست در برابر آن‌ها، و منابعی که برای عمیق‌تر شدن باید دنبال کنید.

۲۰ اشتباه رایج و راه‌حل

♦ ۱ تا ۱۰ – بنیادها، Hooks و فرم‌ها

#	اشتباه	راه‌حل	فصل
۱	تغییر مستقیم state (<code>arr.push</code>)	کپی جدید: <code>spread</code> ، <code>filter</code> ، <code>map</code> ، <code>toSorted</code>	۶
۲	<code>setCount(count + 1)</code> چند بار پشت‌سرهم	فرم تابعی: <code>setCount(c => c + 1)</code>	۶
۳	<code>index</code> به‌عنوان <code>key</code> در لیست پویا	شناسه پایدار از داده	۴
۴	<code>{items.length && <List/>}</code> → چاپ «»	<code>{items.length > 0 && ...}</code>	۴
۵	تعریف کامپوننت داخل کامپوننت	انتقال به سطح ماژول	۵
۶	کپی props در state (<code>useState(props.x)</code>)	مستقیم از props؛ یا <code>key</code> برای ریست	۶
۷	<code>useEffect</code> برای محاسبه مقدار مشتق‌شده	محاسبه در بدنه رندر	۱۱
۸	<code>fetch</code> در Effect بدون <code>cleanup</code> → <code>race</code>	<code>AbortController</code> ؛ بهتر: <code>TanStack Query</code>	۱۱
۹	دروغ در آرایه وابستگی‌ها / <code>eslint-disable</code>	رفع علت؛ <code>useEffectEvent</code> برای منطق رویدادی	۱۱
۱۰	<code>use(fetch(...))</code> مستقیم در رندر	<code>Promise</code> پایدار از والد/loader/کش	۱۲

#	اشتباه	راه حل	فصل
۱۱	<code>useFormStatus</code> در همان کامپوننت فرم	انتقال دکمه به کامپوننت فرزند	۹
۱۲	<code>addOptimistic</code> خارج از <code>transition</code>	داخل <code>startTransition</code> یا <code><form action></code>	۱۰
۱۳	یک Context بزرگ برای همه چیز	تقسیم بر اساس فرکانس تغییر؛ <code>state/dispatch</code> جدا	۱۳
۱۴	داده سرور در <code>Redux/Zustand</code>	<code>TanStack Query</code> ؛ <code>store</code> فقط برای <code>client state</code>	۲۰
۱۵	<code>useMemo / memo</code> همه جا بدون اندازه گیری	<code>React Compiler</code> ؛ <code>Profiler</code>	۱۹
۱۶	یک <code>Error Boundary</code> فقط دور <code><App></code>	مرزهای دانه ریز به ازای هر ویجت	۲۴
۱۷	<code>localStorage</code> در <code>JWT</code>	کوکی <code>httpOnly + CSRF defense</code>	۲۶
۱۸	<code><div onClick></code> به جای <code><button></code>	عنصر معنایی	۲۵
۱۹	<code>React.FC</code> و <code>any</code> در تایپ ها	تابع معمولی + <code>props</code> تایپ شده؛ <code>narrow + unknown</code>	۱۵
۲۰	تست پیاده سازی (<code>state</code> داخلی، نام کلاس)	تست رفتار با <code>Testing Library</code>	۲۷

| بهترین شیوه ها – خلاصه اجرایی

◆ ساختار و کد

- ◆ `feature-based`؛ هر فیچر با `index.ts` به عنوان API عمومی.
- ◆ کامپوننت کوچک (> ۱۵۰ خط)، منطق در هوک، نما در کامپوننت.
- ◆ `TypeScript strict` + `noUncheckedIndexedAccess`؛ `Zod` در مرزهای سیستم.
- ◆ `ESLint` با `react-hooks` + `v6` و `jsx-a11y`؛ `Prettier`؛ `husky`.

◆ state و داده

- ◆ `state` را دسته بندی کنید: محلی / مشترک / سراسری / سرور / URL / فرم.
- ◆ پیش فرض `useState`؛ `useReducer` برای منطق پیچیده؛ `Context` برای کم تغییر؛ `Zustand` برای پرتغییر؛ `TanStack Query` برای سرور.
- ◆ فرم ها با `Zod` + `Actions`؛ `RHF` فقط برای فرم های بسیار پیچیده.
- ◆ `Optimistic UI` برای تعاملات پرتکرار و کم ریسک.

◆ کارایی و UX

- ◆ React Compiler فعال؛ memoization دستی فقط با شواهد Profiler.
- ◆ ورودی فوری، بقیه در transition؛ <Activity> برای حفظ state تب‌ها.
- ◆ Suspense + ErrorBoundary به‌ازای هر بخش مستقل.
- ◆ code splitting به‌ازای مسیر؛ virtualization برای لیست بزرگ؛ بودجه باندل در CI.

◆ کیفیت و امنیت

- ◆ هرم تست: unit برای منطق، component با E2E، MSW، Testing Library فقط جریان‌های حیاتی.
- ◆ بدون dangerouslySetInnerHTML خام؛ توکن در کوکی httpOnly؛ CSP؛ npm audit در CI.
- ◆ دسترسی‌پذیری: HTML معنایی، کیبورد، فوکوس، aria-live؛ axe در CI.
- ◆ RTL با ویژگی‌های منطقی؛ Intl برای اعداد/تاریخ فارسی.

| چک‌لیست بازبینی کد (Code Review)

- ☐ آیا این state واقعاً لازم است یا مشتق‌شده است؟
- ☐ آیا این Effect با یک «سیستم خارجی» همگام می‌شود؟ cleanup دارد؟
- ☐ آیا key پایدار و یکتاست؟
- ☐ آیا کامپوننت یک مسئولیت دارد؟ قابل‌تست است؟
- ☐ آیا props تایپ دقیق دارند (نه any، نه object)؟
- ☐ آیا حالت‌های loading / error / empty مدیریت شده‌اند؟
- ☐ آیا با کیبورد قابل‌استفاده است؟ label دارد؟
- ☐ آیا داده کاربر بدون پاک‌سازی رندر می‌شود؟
- ☐ آیا تست رفتار (نه پیاده‌سازی) اضافه شده؟
- ☐ آیا نام‌ها «چرا» را می‌گویند نه فقط «چه»؟

منابع برای ادامه مسیر

رسمی

منبع	چرا
react.dev/learn	بهترین نقطه شروع؛ مدل ذهنی درست با مثال تعاملی
react.dev/reference	مرجع کامل API با نکات و تله‌ها
react.dev/blog	اعلام نسخه‌ها، RFC، React Labs
react.dev/learn/react-compiler	راهنمای کامپایلر و Rules of React
github.com/reactjs/rfcs	آینده React را زودتر ببینید

کتابخانه‌ها و ابزار

منبع	موضوع
tanstack.com/query	مستندات + مقالات TkDodo (بهترین محتوای server state)
reactrouter.com	راهنمای Data و Framework mode
vite.dev	پیکربندی و پلاگین‌ها
ui.shadcn.com	کامپوننت‌های قابل مالکیت
testing-library.com	فلسفه و API تست
playwright.dev	تست E2E

◆ افراد و وبلاگ‌ها

منبع	موضوع
overreacted.io — Dan Abramov	مدل ذهنی عمیق React، RSC
epicreact.dev — Kent C. Dodds	الگوها، تست، آموزش ساختاریافته
joshwcomeau.com — Josh Comeau	React + CSS با توضیحات بصری عالی
developerway.com — Nadia Makarevich	کارایی و رندر مجدد به صورت دقیق
tkdodo.eu/blog — TkDodo	state management و TanStack Query
buildui.com — Sam Selikoff	انیمیشن و UI حرفه‌ای

◆ خبرنامه و به روز ماندن

◆ **This Week in React** (Sébastien Lorber) — هفتگی، جامع‌ترین خلاصه اکوسیستم.

◆ **React Status** — هفتگی، کوتاه.

◆ **Bytes** (ui.dev) — سبک و طنزآمیز، JavaScript و React.

◆ **React Conf** (سالانه، ویدئوها رایگان) و **React Summit**.

◆ مسیر بعدی

۱. **Next.js 16** — کتاب «مرجع جامع Next.js 16» از همین نویسنده؛ RSC و فول‌استک در عمل.

۲. **React Native / Expo** — همان React برای موبایل؛ ۸۰٪ دانش شما منتقل می‌شود.

۳. **معماری Front-End در مقیاس** — Micro-frontends، Design System، Monorepo (Turborepo/Nx).

۴. **کارایی عمیق** — Chrome Performance، React Performance Tracks، Web Vitals در production.

۵. **مشارکت متن‌باز** — [good first issue](#) های **React Router**، **TanStack**، **shadcn** در **good first issue**.

i سخن پایانی

React در ۱۳ سال از «کتابخانه View فیسبوک» به پلتفرمی برای دو کامپیوتر (سرور و کلاینت) تبدیل شد، اما هسته‌اش تغییر نکرده: **UI تابعی از state است**. اگر این کتاب یک چیز به شما داده باشد، امیدوارم مدل ذهنی درست باشد — چون API‌ها تغییر می‌کنند، ولی مدل ذهنی درست سال‌ها کار می‌کند. موفق باشید. 🍀

36 پرسش‌های مصاحبه (Senior تا Junior)

این فصل مکمل جعبه‌های «سؤالات مصاحبه» در طول کتاب است: سؤالاتی که در مصاحبه‌های واقعی Front-End در ایران و خارج پرسیده می‌شوند، با پاسخ‌هایی به اندازه‌ای که در ۱ تا ۲ دقیقه بتوانید بیان کنید. برای هر پاسخ، ارجاع به فصل مربوطه آمده تا در صورت نیاز عمیق‌تر شوید.

| مبانی (Junior)

۱. **React چیست و چه تفاوتی با فریم‌ورک دارد؟** کتابخانه‌ای برای ساخت UI با کامپوننت‌های اعلانی. برخلاف فریم‌ورک (Angular)، روتینگ، fetch و ساختار پروژه را تحمیل نمی‌کند؛ این‌ها را اکوسیستم (React Router، TanStack Query، Next.js) فراهم می‌کند. (فصل ۱)

۲. **JSX چیست و مرورگر چطور آن را می‌فهمد؟** افزونه نحوی که به فراخوانی `jsx()` تبدیل می‌شود و یک شیء React Element برمی‌گرداند. مرورگر JSX را نمی‌فهمد؛ Vite/Babel آن را در build تبدیل می‌کند. (فصل ۴)

۳. **تفاوت props و state؟** props ورودی از والد و فقط خواندنی؛ state حافظه داخلی که کامپوننت خودش تغییر می‌دهد. تغییر هرکدام رندر مجدد می‌سازد. (فصل ۵ و ۶)

۴. **چرا key لازم است و چرا index بد است؟** key هویت آیتم را بین رندها حفظ می‌کند تا React state و DOM را درست نگه دارد. index با تغییر ترتیب/حذف، state را به آیتم اشتباه می‌چسباند. (فصل ۴)

۵. **چرا state را مستقیم تغییر نمی‌دهیم؟** React با `Object.is` مرجع را مقایسه می‌کند؛ mutation مرجع را عوض نمی‌کند و رندر رخ نمی‌دهد. علاوه بر آن، ناتیغیرپذیری برای memo، undo و دیباگ لازم است. (فصل ۶)

۶. **Controlled و Uncontrolled چه فرقی دارند؟** Controlled: مقدار در React state و با `onChange + value`. Uncontrolled: مقدار در DOM و با `ref + defaultValue` یا `FormData`. React 19 با Actions کار با Uncontrolled را بسیار ساده کرد. (فصل ۶ و ۹)

۷. **Fragment چیست؟** راهی برای برگرداندن چند عنصر بدون افزودن wrapper به DOM: `<>...</>` یا `<Fragment key>`. (فصل ۴)

۸. **useEffect چه زمانی اجرا می‌شود؟** بعد از commit و paint. با `[]` فقط پس از mount؛ با `[a]` هر بار `a` تغییر کند؛ بدون آرایه پس از هر رندر. cleanup قبل از اجرای بعدی و در unmount. (فصل ۱۱)

۹. **تفاوت useEffect و useLayoutEffect؟** `useLayoutEffect` همگام و قبل از paint (برای اندازه‌گیری DOM بدون پرش)؛ `useEffect` بعد از paint. (فصل ۱۱)

۱۰. **StrictMode چه می‌کند؟** در dev کامپوننت را دو بار رندر و Effect را دو بار اجرا می‌کند تا ناخالصی و cleanup ناقص را آشکار کند. در production اثری ندارد. (فصل ۲)

۱۱. `useRef` چه تفاوتی با `useState` دارد؟ تغییر `ref.current` رندر نمی‌سازد و در رندر نباید خوانده شود؛ برای DOM، تایمر و مقادیر غیر-UI. (فصل ۱۱)

۱۲. **Lifting State Up** یعنی چه؟ انتقال state به نزدیک‌ترین والد مشترک وقتی دو کامپوننت خواهر به آن نیاز دارند. (فصل ۶)

هوک‌ها و React 19 (Mid)

۱۳. قوانین هوک چیست و چرا؟ فقط در سطح بالا و فقط از کامپوننت/هوک. چون React هوک‌ها را با ترتیب فراخوانی شناسایی می‌کند. (فصل ۸)

۱۴. **Actions** در **React 19** چیست؟ تابع `async` داخل `transition` که React برایش `pending`، خطا، `optimistic` و ریست فرم را مدیریت می‌کند. از طریق `<form action>`، `startTransition(async)` و `useActionState`. (فصل ۹)

۱۵. `useActionState` چه برمی‌گرداند؟ `[state, formAction, isPending]` : نتیجه آخرین Action، تابعی برای `<form action>`، و وضعیت Action `pending`. با `(prevState, formData)` صدا زده می‌شود. (فصل ۹)

۱۶. `useFormStatus` کجا کار می‌کند؟ فقط در فرزندان `<form>`؛ وضعیت فرم والد را می‌خواند. در همان کامپوننتی که فرم را رندر می‌کند همیشه `pending: false` است. (فصل ۹)

۱۷. `useOptimistic` چطور `rollback` می‌کند؟ مقدار خوش‌بینانه فقط تا پایان `transition` زنده است؛ سپس به مقدار واقعی (props) برمی‌گردد. اگر سرور موفق بود و والد داده جدید داد، UI ثابت می‌ماند؛ اگر نه، برمی‌گردد. (فصل ۱۰)

۱۸. `use API` چیست و چه تفاوتی با هوک دارد؟ `Promise` یا `Context` را در رندر می‌خواند؛ برخلاف هوک‌ها داخل شرط و حلقه مجاز است. با `Promise`، کامپوننت `suspend` می‌شود. (فصل ۸ و ۱۲)

۱۹. `useEffectEvent` چه مشکلی را حل می‌کند؟ جداکردن منطق «رویدادی» از `Effect` تا مقادیر استفاده‌شده در آن وابستگی نشوند و `Effect` بی‌دلیل دوباره اجرا نشود؛ همیشه آخرین `props/state` را می‌بیند. (فصل ۱۱)

۲۰. `ref` به‌عنوان `prop` در **React 19** یعنی چه؟ دیگر نیازی به `forwardRef` نیست؛ `ref` مثل هر `prop` به کامپوننت تابعی می‌رسد. `forwardRef` deprecated است. (فصل ۵)

۲۱. **Suspense** چطور کار می‌کند؟ وقتی کامپوننتی داخلش `suspend` شود (در `Promise`، `use`، `lazy`، یا کتابخانه)، `fallback` نشان می‌دهد و پس از آماده‌شدن محتوا را جایگزین می‌کند. با `transition` می‌تواند به‌جای UI، `fallback` قدیمی را نگه دارد. (فصل ۱۲)

۲۲. چرا `use(fetch())` مستقیم اشتباه است؟ هر رندر `Promise` جدید می‌سازد `suspend` بی‌پایان. `Promise` باید پایدار (از والد، `loader` یا کش) باشد. (فصل ۱۲)

۲۳. **Error Boundary** چه چیزهایی را نمی‌گیرد؟ خطای async، event handler خارج از Actions/Suspense، خطای خودش، و SSR. (فصل ۲۴)

۲۴. **useTransition** و **useDeferredValue**؟ هر دو به‌روزرسانی را غیرفوری می‌کنند. اولی وقتی خودتان setState می‌زنید و **isPending** می‌خواهید؛ دومی برای مقداری که از بیرون می‌آید. (فصل ۱۹)

۲۵. **<Activity>** چیست؟ (۱۹.۲) پنهان کردن بخشی از UI با حفظ state و cleanup Effect، DOMها و اولویت پایین برای به‌روزرسانی‌ها؛ برای تب‌ها و پیش‌رندر. (فصل ۱۹)

۲۶. **Context** چه زمانی رندر می‌سازد و راه بهینه‌سازی؟ هر تغییر **value** همه مصرف‌کننده‌ها را رندر می‌کند. راه‌ها: تقسیم Context، جداکردن value، state/dispatch پایدار (useMemo یا Compiler). (فصل ۱۳)

۲۷. **useReducer** یا **useState**؟ reducer وقتی چند مقدار با هم تغییر می‌کنند، منطق انتقال پیچیده است یا می‌خواهید بدون React تست کنید. (فصل ۱۳)

۲۸. **useSyncExternalStore** برای چیست؟ اتصال امن به store بیرونی (Zustand، window.matchMedia) بدون tearing در Concurrent Rendering؛ با **getServerSnapshot** برای SSR. (فصل ۱۴)

۲۹. تفاوت **TanStack Query** و **Redux**؟ Query برای **server state** (کش، invalidation، refetch)؛ Redux برای **client state**. قراردادن داده سرور در Redux یعنی بازنویسی دستی همه این‌ها. (فصل ۲۰)

۳۰. **Loader** در **React Router** چه مزیتی دارد؟ واکنشی قبل از رندر و موازی برای مسیرهای تودرتو (بدون آبشار)، مدیریت خطای یکپارچه، revalidation پس از action. (فصل ۱۶)

| معماری و عمق (Senior)

۳۱. **Reconciliation** و **Fiber** را توضیح دهید. Reconciliation الگوریتم مقایسه درخت جدید با قبلی: نوع متفاوت → جایگزینی کامل؛ نوع یکسان → به‌روزرسانی props؛ لیست‌ها با **key**. Fiber ساختار داده‌ای است که هر واحد کار را نگه می‌دارد و اجازه می‌دهد رندر قطع، اولویت‌بندی و از سر گرفته شود – پایه Concurrent Rendering. (فصل ۶ و ۱۹)

۳۲. **React Compiler** چطور کار می‌کند و چرا به **Rules of React** وابسته است؟ کد را به HIR تبدیل، وابستگی هر عبارت را تحلیل و برای هر واحد reactive اسلات کش می‌سازد. فرضش ناتغییرپذیری و خلوص است؛ کد متخلف را رد می‌کند. (فصل ۱۹)

۳۳. **Server Component** با **SSR** چه فرقی دارد؟ SSR کامپوننت کلاینتی را به HTML تبدیل می‌کند ولی JS آن باز هم ارسال و hydrate می‌شود. Server Component هرگز به کلاینت نمی‌رود؛ فقط خروجی‌اش (RSC payload) می‌آید. می‌توان هر دو را داشت. (فصل ۲۸)

۳۴. `'use client'` دقیقاً چه می‌کند؟ یک مرز ماژول تعریف می‌کند: آن فایل و همه `import`هایش وارد باندل کلاینت می‌شوند. Client Component نمی‌تواند Server Component را `import` کند اما می‌تواند به‌عنوان `children` بگیرد. (فصل ۲۸)

۳۵. **RSC payload چرا به‌جای HTML؟** درخت React را (نه HTML) با ارجاع به ماژول‌های کلاینت و Promise‌های در جریان توصیف می‌کند؛ هنگام ناوبری، React آن را با درخت فعلی `reconcile` می‌کند و `state` کلاینت حفظ می‌شود. (فصل ۲۸)

۳۶. **Server Function چه ریسک امنیتی دارد؟** هر Server Function یک endpoint عمومی است؛ باید احراز هویت، مجوز و اعتبارسنجی ورودی داخلش انجام شود – دقیقاً مثل REST. آسیب‌پذیری‌های RSC در ۲۰۲۵–۲۶ اهمیت به‌روزرسانی سریع را نشان داد. (فصل ۲۶ و ۲۸)

۳۷. **Hydration mismatch چیست و چطور جلوگیری می‌کنید؟** وقتی HTML سرور با اولین رندر کلاینت متفاوت است (تاریخ، `window`، `random`). راه‌ها: `useId` به‌جای `random`، `useSyncExternalStore` با `getServerSnapshot`، رندر مقادیر وابسته به مرورگر فقط پس از `mount`، `suppressHydrationWarning` فقط برای موارد ناگزیر. (فصل ۲۵ و ۲۸)

۳۸. **چطور INP یک صفحه سنگین React را بهبود می‌دهید؟** Profile → شناسایی handler/رندر طولانی → `startTransition` برای به‌روزرسانی غیرفوری، شکستن کار، React Compiler، `virtualization`، حذف کار از مسیر بحرانی تعامل، `<Activity>` برای بخش‌های پنهان. (فصل ۱۹ و ۳۰)

۳۹. **tearing چیست؟** در Concurrent Rendering، بخش‌های مختلف درخت ممکن است در زمان‌های مختلف رندر شوند؛ اگر `store` بیرونی بین آن‌ها تغییر کند، UI ناسازگار می‌شود. `useSyncExternalStore` با رندر همگام هنگام تغییر، این را حل می‌کند. (فصل ۱۴)

۴۰. **استراتژی state برای اپ متوسط؟** URL برای فیلتر/صفحه، TanStack Query برای سرور، Zustand (۱–۳ store) برای client state پرتغییر، Context برای تم/auth، `useState` برای بقیه (اکثریت). (فصل ۲۰)

۴۱. **چطور معماری را در برابر فرسایش حفظ می‌کنید؟** تبدیل قواعد به ابزار: ESLint boundaries، `index.ts` API عمومی، TypeScript strict، تست معماری، ADR، بازبینی با چک‌لیست. (فصل ۲۱)

۴۲. **Compound Components را چطور پیاده می‌کنید و چرا؟** `state` مشترک در Context، زیرکامپوننت‌ها به‌عنوان property (`Tabs.Trigger`)؛ انعطاف ساختاری بدون انفجار `props`؛ ARIA با `useId`. (فصل ۲۲)

۴۳. **چطور یک SPA را بدون شکستن کاربران فعال deploy می‌کنید؟** `assets` با `hash` و HTML، `immutable` بدون کش، نگاه‌داشتن نسخه‌های قبلی `assets`، handler برای `vite:preloadError`، `deploy atomic`، سازگاری API. (فصل ۳۰)

۴۴. **CSP با nonce در SPA چه چالشی دارد؟** HTML استاتیک در CDN نمی‌تواند `nonce` پویا داشته باشد؛ باید HTML در edge رندر شود یا از `hash` استفاده شود؛ کتابخانه‌های `inline`-تزریق‌کننده می‌شکنند. (فصل ۲۶)

۴۵. کی SPA خالص بهتر از فریم‌ورک فول‌استک است؟ اپ پشت لاگین بدون SEO، بک‌اند مستقل چندکلاینته، زیرساخت فقط CDN، اپ فوق‌تعاملی که RSC مزیت کمی دارد. سادگی و استقلال از vendor در برابر مزایای RSC. (فصل ۲۹)

سؤالات عملی (Live Coding) رایج

سؤال	مفاهیم آزموده‌شده
یک <code>useDebounce</code> بنویس	<code>Effect</code> ، <code>cleanup</code> ، جنریک
لیست با فیلتر و مرتب‌سازی از API	<code>state</code> ، <code>fetch</code> مشتق‌شده، <code>key</code>
فرم با اعتبارسنجی و <code>pending</code>	<code>Actions</code> یا <code>UX</code> ، <code>controlled</code> ، خطا
Modal دسترس‌پذیر	<code>Portal</code> / <code><dialog></code> ، فوکوس، <code>Escape</code>
شمارنده با <code>Undo</code>	<code>reducer</code> ، ناتغییرپذیری
<code>Compound</code> به صورت <code>Tabs</code>	<code>Context</code> ، <code>ARIA</code>
«چرا این کامپوننت دوباره رندر می‌شود؟» (کد داده می‌شود)	مدل رندر، مرجع شیء، <code>Context</code>
«این <code>Effect</code> چه باگی دارد؟»	وابستگی‌ها، <code>cleanup</code> ، <code>race condition</code>

نحوه پاسخ‌دادن

ابتدا مفهوم را در یک جمله بگویید، بعد چرا، بعد یک مثال کوتاه، و در پایان تله رایج یا جایگزین. مصاحبه‌کننده‌های سنیور دنبال مدل ذهنی‌اند، نه حفظیات. اگر چیزی را نمی‌دانید، بگویید «مطمئن نیستم، ولی حدسم این است که...» و استدلال کنید.

37 واژه‌نامه فارسی-انگلیسی

مرجع سریع مهم‌ترین اصطلاحات React و اکوسیستم آن. برای مرور پیش از مصاحبه یا وقتی در مستندات انگلیسی به واژه‌ای برخوردید که معادل فارسی‌اش را می‌خواهید.

مفاهیم هسته

♦ مدل رندر

اصطلاح	معادل و توضیح
Component	کامپوننت؛ تابعی که props می‌گیرد و React Element برمی‌گرداند
Props	ویژگی‌ها؛ ورودی فقط‌خواندنی کامپوننت از والد
State	وضعیت؛ حافظه داخلی کامپوننت که تغییرش رندر مجدد می‌سازد
Render	رندر؛ اجرای تابع کامپوننت برای محاسبه خروجی (نه لمس DOM)
Commit	کامیت؛ اعمال تفاوت‌های محاسبه‌شده روی DOM واقعی
Reconciliation	آشتی‌دهی؛ الگوریتم مقایسه درخت جدید و قبلی برای یافتن کمترین تغییر
Fiber	ساختار داده داخلی React برای هر واحد کار؛ پایه رندر قابل‌قطع
Virtual DOM	نمایش سبک درخت UI در حافظه؛ اصطلاح قدیمی‌تر برای همان ایده

اصطلاح	معادل و توضیح
JSX	افزونه نحوی شبیه HTML که به <code>jsx()</code> تبدیل می‌شود
React Element	شیء ساده‌ای که خروجی JSX است و «چه چیزی» را توصیف می‌کند
Key	کلید؛ شناسه پایدار آیتم‌های لیست برای حفظ هویت بین رندها
Fragment	قطعه؛ گروه‌بندی عناصر بدون افزودن wrapper به DOM
Pure Component / Purity	خلوص؛ خروجی یکسان با ورودی یکسان و بدون side effect در رندر
Immutability	نا تغییرپذیری؛ ساخت کپی جدید به جای تغییر شیء موجود
Lifting State Up	بالا بردن state به والد مشترک
Composition	ترکیب؛ ساخت UI پیچیده از کامپوننت‌های ساده (به جای وراثت)
Children	فرزندان؛ prop ویژه برای محتوای بین تگ باز و بسته
Controlled / Uncontrolled	کنترل‌شده (مقدار در state) / کنترل‌نشده (مقدار در DOM)
Prop Drilling	عبور دادن props از چند لایه میانی که به آن نیاز ندارند
Strict Mode	حالت سخت‌گیرانه؛ رندر و Effect دوگانه در dev برای یافتن باگ

اصطلاح	معادل و توضیح
Hook	هوک؛ تابعی با پیشوند <code>use</code> که به کامپوننت تابعی قابلیت (state, Effect...) می‌دهد
Rules of Hooks	قوانین هوک؛ فقط در سطح بالا و فقط از کامپوننت/هوک
Rules of React	قوانین React؛ مجموعه گسترده‌تر (خلوص، ناتغییرپذیری...) که Compiler به آن نیاز دارد
Custom Hook	هوک سفارشی؛ استخراج منطق stateful قابل استفاده مجدد
Effect	اثر؛ همگام‌سازی کامپوننت با سیستم خارجی پس از commit
Cleanup	پاک‌سازی؛ تابع بازگشتی Effect که قبل از اجرای بعدی و در unmount اجرا می‌شود
Dependency Array	آرایه وابستگی‌ها؛ مقادیر reactive که تغییرشان Effect را دوباره اجرا می‌کند
Effect Event (<code>useEffectEvent</code>)	رویداد Effect؛ منطق غیر-reactive که از داخل Effect صدا زده می‌شود (۱۹.۲)
Ref	مرجع؛ مقدار قابل تغییر بدون رندر یا دسترسی به DOM
Ref as Prop	<code>ref</code> به عنوان prop معمولی در React 19؛ جایگزین <code>forwardRef</code>
Context	زمینه؛ انتقال داده به عمق درخت بدون props
Provider	تأمین‌کننده؛ در React 19 خود <code><Context value></code>
Reducer	کاهنده؛ تابع خالص <code>(state, action) => newState</code>

اصطلاح	معادل و توضیح
Action (React 19)	تابع async داخل transition با مدیریت خودکار pending/خطا
useActionState	هوک state + pending برای Action فرم
useFormStatus	وضعیت pending فرم والد (از react-dom)
useOptimistic	به روزرسانی خوش بینانه با بازگشت خودکار
use	خواندن Promise یا Context در رندر؛ مجاز در شرط
Transition	انتقال؛ به روزرسانی غیرفوری و قابل قطع
useDeferredValue	مقدار عقب افتاده برای رندر کم اولویت
useSyncExternalStore	اتصال امن به store بیرونی بدون tearing
useId	شناسه یکتای پایدار سرور/کلاینت برای ARIA
<Activity>	پنهان کردن UI با حفظ state و اولویت پایین (۱۹.۲)
cache / cacheSignal	حذف تکرار فراخوانی در یک درخواست سرور (RSC)
Document Metadata	تگ های <title> / <meta> / <link> که React 19 به <head> منتقل می کند
Owner Stack	پشته مالک؛ ابزار dev برای دیدن کدام کامپوننت این را رندر کرده (۱۹.۱)
<ViewTransition>	انتقال نما؛ انیمیشن بین دو حالت UI روی View Transitions API مرورگر – در React هنوز آزمایشی (فصل ۳۲)

داده، Suspense و Concurrent

◆ Suspense و رندر همروند

اصطلاح	معادل و توضیح
Suspense	تعلیق؛ نمایش fallback تا آماده شدن داده/کد زیردرخت
Suspend	معلق شدن؛ وقتی کامپوننت Promise ناتمام را throw می کند
Fallback	جایگزین موقت؛ محتوای نمایشی در حین انتظار (Skeleton)
Error Boundary	مرز خطا؛ کامپوننتی که خطای زیردرخت را می گیرد
Streaming	جریان سازی؛ ارسال تکه تکه HTML/payload از سرور
Batching	دسته بندی؛ ادغام چند setState در یک رندر
Concurrent Rendering	رندر همروند؛ رندر قابل قطع با اولویت بندی
Tearing	پارگی؛ ناسازگاری UI وقتی store بیرونی وسط رندر تغییر کند
Waterfall	آبشار؛ درخواست های متوالی که هر کدام منتظر قبلی است
Race Condition	شرایط مسابقه؛ رسیدن پاسخ قدیمی بعد از جدید

◆ داده و کش

اصطلاح	معادل و توضیح
Server State	داده ای که منبع حقیقتش سرور است و کش می شود
Stale Time	مدت تازگی داده در کش قبل از نیاز به refetch
Invalidation	باطل سازی؛ علامت گذاری کش برای واکشی مجدد
Optimistic UI	رابط خوش بینانه؛ نمایش نتیجه قبل از تأیید سرور
Deduplication	حذف تکرار؛ ادغام درخواست های همزمان یکسان
Prefetch	پیش واکشی؛ گرفتن داده قبل از نیاز (hover، مسیر بعدی)
Loader / Action (Router)	تابع واکشی/تغییر داده در سطح مسیر
Revalidation	اعتبارسنجی مجدد؛ اجرای دوباره loaderها پس از تغییر

Server Components و فریمورک

Server Components ♦

اصطلاح	معادل و توضیح
RSC (React Server Components)	کامپوننت‌های سرور؛ فقط روی سرور اجرا و بدون JS کلاینت
Client Component	کامپوننت کلاینت؛ با <code>'use client'</code> ؛ در سرور و مرورگر اجرا می‌شود
<code>'use client'</code> / <code>'use server'</code>	دایرکتیو مرز کلاینت / علامت Server Function
Server Function (Server Action)	تابع سرور قابل فراخوانی از کلاینت؛ endpoint خودکار
RSC Payload	خروجی سریال‌شده درخت سرور برای کلاینت
Serializable	سریال‌پذیر؛ داده‌ای که می‌تواند از مرز سرور/کلاینت عبور کند

رندر سرور و فریمورک ♦

اصطلاح	معادل و توضیح
SSR	رندر سمت سرور؛ تولید HTML اولیه روی سرور
SSG	تولید استاتیک؛ HTML در زمان build
ISR	بازتولید افزایشی استاتیک؛ صفحه استاتیک با به‌روزرسانی دوره‌ای
Hydration	آب‌دهی؛ اتصال event handler به HTML سرور در کلاینت
Hydration Mismatch	ناهمخوانی HTML سرور و رندر اولیه کلاینت
Partial Pre-rendering (PPR)	پیش‌رندر جزئی؛ shell استاتیک + بخش‌های پویا در یک صفحه
Progressive Enhancement	بهبود تدریجی؛ کارکرد پایه بدون JS، بهتر با JS
Islands	جزیره‌ها؛ نواحی تعاملی کوچک در صفحه‌ای عمدتاً استاتیک
App Router	روتر مبتنی بر پوشه <code>app/</code> در Next.js با RSC
Framework Mode	حالت فریمورک React Router (SSR)، جانشین (Remix)
Middleware (Router)	میان‌افزار؛ کدی که پیش از loader/action مسیر اجرا می‌شود (احراز هویت، لاگ)؛ در React Router v8 پیش‌فرض
Edge Runtime	اجرای کد نزدیک کاربر روی شبکه CDN

اصطلاح	معادل و توضیح
React Compiler	کامپایلر React؛ memoization خودکار در زمان build (۱.۰)
Memoization	یادسپاری؛ کش نتیجه بر اساس ورودی (<code>memo</code> , <code>useMemo</code>)
Code Splitting	تقسیم کد؛ شکستن باندل به chunkهای lazy
Lazy Loading	بارگذاری تنبل؛ دانلود کد/داده فقط هنگام نیاز
Tree Shaking	حذف کد استفاده نشده در build
Bundle	باندل؛ فایل(های) JS نهایی
Chunk	تکه؛ بخشی از باندل که جداگانه بارگذاری می شود
HMR	جایگزینی داغ ماژول؛ به روزرسانی بدون رفرش در dev
Codemod	تغییر خودکار کد با اسکریپت (<code>npx codemod react/19/...</code>) برای مهاجرت
Virtualization	مجازی سازی؛ رندر فقط آیتم های داخل viewport
Core Web Vitals	معیارهای کلیدی تجربه کاربر: LCP, INP, CLS
LCP	زمان نمایش بزرگترین عنصر محتوایی
INP	تأخیر تعامل تا نقاشی بعدی
CLS	مجموع پرش های چیدمان
Profiler	پروفایلر؛ ابزار اندازه گیری رندر در DevTools
Performance Tracks	ردهای React در Chrome Performance (۱۹.۲)

♦ تست و الگوها

اصطلاح	معادل و توضیح
Testing Library	کتابخانه تست مبتنی بر رفتار کاربر
MSW (Mock Service Worker)	شبیه‌سازی شبکه در سطح درخواست
E2E (End-to-End)	تست سرتاسری در مرورگر واقعی (Playwright)
Headless UI	کامپوننت بدون استایل؛ فقط منطق و دسترسی‌پذیری
Compound Components	کامپوننت‌های مرکب با state مشترک از طریق Context
Render Prop	تابع به‌عنوان prop برای سفارشی‌کردن رندر
Slot / <code>asChild</code>	جای‌گذاری؛ اعمال رفتار/استایل روی عنصر فرزند
Portal	دروازه؛ رندر در جای دیگر DOM با حفظ جایگاه در درخت React
Discriminated Union	اتحاد تمایز یافته؛ تایپ union با فیلد تشخیص‌دهنده
Zod	کتابخانه schema برای اعتبارسنجی + استخراج تایپ

♦ امنیت

اصطلاح	معادل و توضیح
CSP	سیاست امنیت محتوا؛ هدر محدودکننده منابع مجاز
XSS	تزریق اسکریپت؛ اجرای کد مهاجم در مرورگر کاربر
CSRF	جعل درخواست بین‌سایتی؛ سوءاستفاده از کوکی کاربر
httpOnly Cookie	کوکی غیرقابل‌خواندن با JavaScript

♦ دسترسی‌پذیری و بین‌المللی‌سازی

اصطلاح	معادل و توضیح
a11y	دسترسی‌پذیری (accessibility)
i18n / l10n	بین‌المللی‌سازی / بومی‌سازی
RTL	راست‌به‌چپ
Logical Properties	ویژگی‌های منطقی CSS (inline-start) مستقل از جهت
ARIA	مجموعه ویژگی‌های دسترسی‌پذیری برای فناوری‌های کمکی
Live Region	ناحیه زنده؛ اعلام خودکار تغییرات به screen reader
Focus Trap	تله فوکوس؛ نگه‌داشتن فوکوس داخل modal
Roving tabindex	فقط یک آیتم در ترتیب Tab؛ Arrow بین آیتم‌ها

پایان مرجع

موفق باشید

اگر این کتاب برایتان مفید بود، آن را با توسعه‌دهندگان دیگر به اشتراک بگذارید و در مخزن گیت‌هاب با یک **Star** از آن حمایت کنید. بازخوردها و پیشنهادهای شما، نسخه‌های بعدی را بهتر می‌کند.

React 19.2

TypeScript

Vite

React Compiler

Production Ready

مخزن گیت‌هاب github.com/rezaian-dev/react-19-persian-guide

مرجع مکمل github.com/rezaian-dev/nextjs-16-persian-guide

گردآوری و تدوین: **محمد رضا رضائیان** · Front-End Developer · نسخه ۲۰۲۶ · ویرایش ۱.۰.۳